

Rust by Example

[Rust](#) is a modern systems programming language focusing on safety, speed, and concurrency. It accomplishes these goals by being memory safe without using garbage collection.

Rust by Example (RBE) is a collection of runnable examples that illustrate various Rust concepts and standard libraries. To get even more out of these examples, don't forget to [install Rust locally](#) and check out the [official docs](#). Additionally for the curious, you can also [check out the source code for this site](#).

Now let's begin!

- [Hello World](#) - Start with a traditional Hello World program.
- [Primitives](#) - Learn about signed integers, unsigned integers and other primitives.
- [Custom Types](#) - `struct` and `enum`.
- [Variable Bindings](#) - mutable bindings, scope, shadowing.
- [Types](#) - Learn about changing and defining types.
- [Conversion](#)
- [Expressions](#)
- [Flow of Control](#) - `if / else`, `for`, and others.
- [Functions](#) - Learn about Methods, Closures and Higher Order Functions.
- [Modules](#) - Organize code using modules
- [Crates](#) - A crate is a compilation unit in Rust. Learn to create a library.
- [Cargo](#) - Go through some basic features of the official Rust package management tool.
- [Attributes](#) - An attribute is metadata applied to some module, crate or item.
- [Generics](#) - Learn about writing a function or data type which can work for multiple types of arguments.
- [Scoping rules](#) - Scopes play an important part in ownership, borrowing, and lifetimes.
- [Traits](#) - A trait is a collection of methods defined for an unknown type: `self`

- [Macros](#)
- [Error handling](#) - Learn Rust way of handling failures.
- [Std library types](#) - Learn about some custom types provided by `std` library.
- [Std misc](#) - More custom types for file handling, threads.
- [Testing](#) - All sorts of testing in Rust.
- [Unsafe Operations](#)
- [Compatibility](#)
- [Meta](#) - Documentation, Benchmarking.

Hello World

This is the source code of the traditional Hello World program.

```
1 // This is a comment, and is ignored by the compiler.
2 // You can test this code by clicking the "Run" button over there ->
3 // or if you prefer to use your keyboard, you can use the "Ctrl + Enter"
4 // shortcut.
5
6 // This code is editable, feel free to hack it!
7 // You can always return to the original code by clicking the "Reset" but
8
9 // This is the main function.
10 fn main() {
11     // Statements here are executed when the compiled binary is called.
12
13     // Print text to the console.
14     println!("Hello World!");
15 }
```

`println!` is a *macro* that prints text to the console.

A binary can be generated using the Rust compiler: `rustc`.

```
$ rustc hello.rs
```

`rustc` will produce a `hello` binary that can be executed.

```
$ ./hello
Hello World!
```

Activity

Click 'Run' above to see the expected output. Next, add a new line with a second `println!` macro so that the output shows:

```
Hello World!
I'm a Rustacean!
```

Comments

Any program requires comments, and Rust supports a few different varieties:

- *Regular comments* which are ignored by the compiler:
 - `//` Line comments which go to the end of the line.
 - `/*` Block comments which go to the closing delimiter. `*/`
- *Doc comments* which are parsed into HTML library [documentation](#):
 - `///` Generate library docs for the following item.
 - `//!` Generate library docs for the enclosing item.

```
1 fn main() {
2     // This is an example of a line comment.
3     // There are two slashes at the beginning of the line.
4     // And nothing written after these will be read by the compiler.
5
6     // println!("Hello, world!");
7
8     // Run it. See? Now try deleting the two slashes, and run it again.
9
10    /*
11     * This is another type of comment, a block comment. In general,
12     * line comments are the recommended comment style. But block comment
13     * are extremely useful for temporarily disabling chunks of code.
14     * /* Block comments can be /* nested, */ */ so it takes only a few
15     * keystrokes to comment out everything in this main() function.
16     * /*/*/* Try it yourself! /*/*/*
17     */
18
19    /*
20     Note: The previous column of `*` was entirely for style. There's
21     no actual need for it.
22     */
23
24    // You can manipulate expressions more easily with block comments
25    // than with line comments. Try deleting the comment delimiters
26    // to change the result:
27    let x = 5 + /* 90 + */ 5;
28    println!("Is `x` 10 or 100? x = {}", x);
29 }
```

See also:

[Library documentation](#)

Formatted print

Printing is handled by a series of `macros` defined in `std::fmt` some of which include:

- `format!` : write formatted text to `String`
- `print!` : same as `format!` but the text is printed to the console (`io::stdout`).
- `println!` : same as `print!` but a newline is appended.
- `eprint!` : same as `print!` but the text is printed to the standard error (`io::stderr`).
- `eprintln!` : same as `eprint!` but a newline is appended.

All parse text in the same fashion. As a plus, Rust checks formatting correctness at compile time.

```
1 fn main() {
2     // In general, the `{}` will be automatically replaced with any
3     // arguments. These will be stringified.
4     println!("{}", days, 31);
5
6     // Positional arguments can be used. Specifying an integer inside `{}`
7     // determines which additional argument will be replaced. Arguments s
8     // at 0 immediately after the format string.
9     println!("{0}, this is {1}. {1}, this is {0}", "Alice", "Bob");
10
11     // As can named arguments.
12     println!("{subject} {verb} {object}",
13              object="the lazy dog",
14              subject="the quick brown fox",
15              verb="jumps over");
16
17     // Different formatting can be invoked by specifying the format chara
18     // after a `:`.
19     println!("Base 10:          {}", 69420); // 69420
20     println!("Base 2 (binary):    {:b}", 69420); // 10000111100101100
21     println!("Base 8 (octal):     {:o}", 69420); // 207454
22     println!("Base 16 (hexadecimal): {:x}", 69420); // 10f2c
23     println!("Base 16 (hexadecimal): {:X}", 69420); // 10F2C
24
25     // You can right-justify text with a specified width. This will
26     // output "    1". (Four white spaces and a "1", for a total width of
27     println!("{number:>5}", number=1);
28
29     // You can pad numbers with extra zeroes,
30     println!("{number:0>5}", number=1); // 00001
31     // and left-adjust by flipping the sign. This will output "10000".
32     println!("{number:0<5}", number=1); // 10000
33
34     // You can use named arguments in the format specifier by appending a
35     println!("{number:0>width$}", number=1, width=5);
36
37     // Rust even checks to make sure the correct number of arguments are
38     println!("My name is {0}, {1} {0}", "Bond");
39     // FIXME ^ Add the missing argument: "James"
40
41     // Only types that implement fmt::Display can be formatted with `{}`.
42     // defined types do not implement fmt::Display by default.
43
44     #[allow(dead_code)] // disable `dead_code` which warn against unused
45     struct Structure(i32);
46
47     // This will not compile because `Structure` does not implement
48     // fmt::Display.
49     // println!("This struct `{}` won't print...", Structure(3));
50     // TODO ^ Try uncommenting this line
51
52     // For Rust 1.58 and above, you can directly capture the argument from
53     // surrounding variable. Just like the above, this will output
54     // "    1", 4 white spaces and a "1".
55     let number: f64 = 1.0;
56     let width: usize = 5;
57     println!("{number:>width$}");
```

`std::fmt` contains many `traits` which govern the display of text. The base form of two important ones are listed below:

- `fmt::Debug`: Uses the `{:?}` marker. Format text for debugging purposes.
- `fmt::Display`: Uses the `{}` marker. Format text in a more elegant, user friendly fashion.

Here, we used `fmt::Display` because the `std` library provides implementations for these types. To print text for custom types, more steps are required.

Implementing the `fmt::Display` trait automatically implements the `ToString` trait which allows us to `convert` the type to `String`.

In *line 43*, `#[allow(dead_code)]` is an `attribute` which only apply to the module after it.

Activities

- Fix the issue in the above code (see `FIXME`) so that it runs without error.
- Try uncommenting the line that attempts to format the `Structure` struct (see `TODO`)
- Add a `println!` macro call that prints: `Pi is roughly 3.142` by controlling the number of decimal places shown. For the purposes of this exercise, use `let pi = 3.141592` as an estimate for `pi`. (Hint: you may need to check the `std::fmt` documentation for setting the number of decimals to display)

See also:

`std::fmt`, `macros`, `struct`, `traits`, and `dead_code`

Debug

All types which want to use `std::fmt` formatting traits require an implementation to be printable. Automatic implementations are only provided for types such as in the `std` library. All others *must* be manually implemented somehow.

The `fmt::Debug` trait makes this very straightforward. *All* types can derive (automatically create) the `fmt::Debug` implementation. This is not true for `fmt::Display` which must be manually implemented.

```
// This structure cannot be printed either with `fmt::Display` or
// with `fmt::Debug`.
struct UnPrintable(i32);

// The `derive` attribute automatically creates the implementation
// required to make this `struct` printable with `fmt::Debug`.
#[derive(Debug)]
struct DebugPrintable(i32);
```

All `std` library types are automatically printable with `{:?}` too:

```
1 // Derive the `fmt::Debug` implementation for `Structure`. `Structure`
2 // is a structure which contains a single `i32`.
3 #[derive(Debug)]
4 struct Structure(i32);
5
6 // Put a `Structure` inside of the structure `Deep`. Make it printable
7 // also.
8 #[derive(Debug)]
9 struct Deep(Structure);
10
11 fn main() {
12     // Printing with `{:?}` is similar to with `{}`.
13     println!("{:?} months in a year.", 12);
14     println!("{1:?} {0:?} is the {actor:?} name.",
15             "Slater",
16             "Christian",
17             actor="actor's");
18
19     // `Structure` is printable!
20     println!("Now {:?} will print!", Structure(3));
21
22     // The problem with `derive` is there is no control over how
23     // the results look. What if I want this to just show a `7`?
24     println!("Now {:?} will print!", Deep(Structure(7)));
25 }
```

So `fmt::Debug` definitely makes this printable but sacrifices some elegance. Rust also provides "pretty printing" with `{:#?}`.


```
1  #[derive(Debug)]
2  struct Person<'a> {
3      name: &'a str,
4      age: u8
5  }
6
7  fn main() {
8      let name = "Peter";
9      let age = 27;
10     let peter = Person { name, age };
11
12     // Pretty print
13     println!("{:#?}", peter);
14 }
```

One can manually implement `fmt::Display` to control the display.

See also:

[attributes](#), [derive](#), [std::fmt](#), and [struct](#)

Display

`fmt::Debug` hardly looks compact and clean, so it is often advantageous to customize the output appearance. This is done by manually implementing `fmt::Display`, which uses the `{}` print marker. Implementing it looks like this:

```
// Import (via `use`) the `fmt` module to make it available.
use std::fmt;

// Define a structure for which `fmt::Display` will be implemented. This is
// a tuple struct named `Structure` that contains an `i32`.
struct Structure(i32);

// To use the `{}` marker, the trait `fmt::Display` must be implemented
// manually for the type.
impl fmt::Display for Structure {
    // This trait requires `fmt` with this exact signature.
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        // Write strictly the first element into the supplied output
        // stream: `f`. Returns `fmt::Result` which indicates whether the
        // operation succeeded or failed. Note that `write!` uses syntax
        which
        // is very similar to `println!`.
        write!(f, "{}", self.0)
    }
}
```

`fmt::Display` may be cleaner than `fmt::Debug` but this presents a problem for the `std` library. How should ambiguous types be displayed? For example, if the `std` library implemented a single style for all `Vec<T>`, what style should it be? Would it be either of these two?

- `Vec<path>`: `:/etc:/home/username:/bin` (split on `:`)
- `Vec<number>`: `1,2,3` (split on `,`)

No, because there is no ideal style for all types and the `std` library doesn't presume to dictate one. `fmt::Display` is not implemented for `Vec<T>` or for any other generic containers. `fmt::Debug` must then be used for these generic cases.

This is not a problem though because for any new *container* type which is *not* generic, `fmt::Display` can be implemented.

```
1 use std::fmt; // Import `fmt`
2
3 // A structure holding two numbers. `Debug` will be derived so the result
4 // be contrasted with `Display`.
5 #[derive(Debug)]
6 struct MinMax(i64, i64);
7
8 // Implement `Display` for `MinMax`.
9 impl fmt::Display for MinMax {
10     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
11         // Use `self.number` to refer to each positional data point.
12         write!(f, "({}, {})", self.0, self.1)
13     }
14 }
15
16 // Define a structure where the fields are nameable for comparison.
17 #[derive(Debug)]
18 struct Point2D {
19     x: f64,
20     y: f64,
21 }
22
23 // Similarly, implement `Display` for `Point2D`.
24 impl fmt::Display for Point2D {
25     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
26         // Customize so only `x` and `y` are denoted.
27         write!(f, "x: {}, y: {}", self.x, self.y)
28     }
29 }
30
31 fn main() {
32     let minmax = MinMax(0, 14);
33
34     println!("Compare structures:");
35     println!("Display: {}", minmax);
36     println!("Debug: {:?}", minmax);
37
38     let big_range = MinMax(-300, 300);
39     let small_range = MinMax(-3, 3);
40
41     println!("The big range is {big} and the small is {small}",
42             small = small_range,
43             big = big_range);
44
45     let point = Point2D { x: 3.3, y: 7.2 };
46
47     println!("Compare points:");
48     println!("Display: {}", point);
49     println!("Debug: {:?}", point);
50
51     // Error. Both `Debug` and `Display` were implemented, but `{b}`
52     // requires `fmt::Binary` to be implemented. This will not work.
53     // println!("What does Point2D look like in binary: {b}?", point);
54 }
```

So, `fmt::Display` has been implemented but `fmt::Binary` has not, and therefore

cannot be used. `std::fmt` has many such [traits](#) and each requires its own implementation. This is detailed further in [std::fmt](#).

Activity

After checking the output of the above example, use the `Point2D` struct as a guide to add a `Complex` struct to the example. When printed in the same way, the output should be:

```
Display: 3.3 + 7.2i
Debug: Complex { real: 3.3, imag: 7.2 }
```

See also:

[derive](#), [std::fmt](#), [macros](#), [struct](#), [trait](#), and [use](#)

Testcase: List

Implementing `fmt::Display` for a structure where the elements must each be handled sequentially is tricky. The problem is that each `write!` generates a `fmt::Result`. Proper handling of this requires dealing with *all* the results. Rust provides the `?` operator for exactly this purpose.

Using `?` on `write!` looks like this:

```
// Try `write!` to see if it errors. If it errors, return
// the error. Otherwise continue.
write!(f, "{}", value)?;
```

With `?` available, implementing `fmt::Display` for a `Vec` is straightforward:

```
1 use std::fmt; // Import the `fmt` module.
2
3 // Define a structure named `List` containing a `Vec`.
4 struct List(Vec<i32>);
5
6 impl fmt::Display for List {
7     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
8         // Extract the value using tuple indexing,
9         // and create a reference to `vec`.
10        let vec = &self.0;
11
12        write!(f, "[")?;
13
14        // Iterate over `v` in `vec` while enumerating the iteration
15        // count in `count`.
16        for (count, v) in vec.iter().enumerate() {
17            // For every element except the first, add a comma.
18            // Use the ? operator to return on errors.
19            if count != 0 { write!(f, ", ")?; }
20            write!(f, "{}", v)?;
21        }
22
23        // Close the opened bracket and return a fmt::Result value.
24        write!(f, "]")
25    }
26 }
27
28 fn main() {
29     let v = List(vec![1, 2, 3]);
30     println!("{}", v);
31 }
```

Activity

Try changing the program so that the index of each element in the vector is also printed.

The new output should look like this:

```
[0: 1, 1: 2, 2: 3]
```

See also:

[for](#), [ref](#), [Result](#), [struct](#), [?](#), and [vec!](#)

Formatting

We've seen that formatting is specified via a *format string*:

- `format!("{}", foo) -> "3735928559"`
- `format!("{:X}", foo) -> "0xDEADBEEF"`
- `format!("{:o}", foo) -> "0o33653337357"`

The same variable (`foo`) can be formatted differently depending on which *argument type* is used: `x` vs `o` vs *unspecified*.

This formatting functionality is implemented via traits, and there is one trait for each argument type. The most common formatting trait is `Display`, which handles cases where the argument type is left unspecified: `{}` for instance.

```

1 use std::fmt::{self, Formatter, Display};
2
3 struct City {
4     name: &'static str,
5     // Latitude
6     lat: f32,
7     // Longitude
8     lon: f32,
9 }
10
11 impl Display for City {
12     // `f` is a buffer, and this method must write the formatted string i
13     fn fmt(&self, f: &mut Formatter) -> fmt::Result {
14         let lat_c = if self.lat >= 0.0 { 'N' } else { 'S' };
15         let lon_c = if self.lon >= 0.0 { 'E' } else { 'W' };
16
17         // `write!` is like `format!`, but it will write the formatted st
18         // into a buffer (the first argument).
19         write!(f, "{}: {:.3}°{} {:.3}°{}",
20             self.name, self.lat.abs(), lat_c, self.lon.abs(), lon_c)
21     }
22 }
23
24 #[derive(Debug)]
25 struct Color {
26     red: u8,
27     green: u8,
28     blue: u8,
29 }
30
31 fn main() {
32     for city in [
33         City { name: "Dublin", lat: 53.347778, lon: -6.259722 },
34         City { name: "Oslo", lat: 59.95, lon: 10.75 },
35         City { name: "Vancouver", lat: 49.25, lon: -123.1 },
36     ] {
37         println!("{}", city);
38     }
39     for color in [
40         Color { red: 128, green: 255, blue: 90 },
41         Color { red: 0, green: 3, blue: 254 },
42         Color { red: 0, green: 0, blue: 0 },
43     ] {
44         // Switch this to use {} once you've added an implementation
45         // for fmt::Display.
46         println!("{}", color);
47     }
48 }

```

You can view a [full list of formatting traits](#) and their argument types in the `std::fmt` documentation.

Activity

Add an implementation of the `fmt::Display` trait for the `color` struct above so that the output displays as:

```
RGB (128, 255, 90) 0x80FF5A
RGB (0, 3, 254) 0x0003FE
RGB (0, 0, 0) 0x000000
```

Three hints if you get stuck:

- The formula for calculating a color in the RGB color space is: $RGB = (R \times 65536) + (G \times 256) + B$, (when R is RED, G is GREEN and B is BLUE). For more see [RGB color format & calculation](#).
- You [may need to list each color more than once](#).
- You can [pad with zeros to a width of 2](#) with `:0>2`.

See also:

[std::fmt](#)

Primitives

Rust provides access to a wide variety of `primitives`. A sample includes:

Scalar Types

- Signed integers: `i8`, `i16`, `i32`, `i64`, `i128` and `isize` (pointer size)
- Unsigned integers: `u8`, `u16`, `u32`, `u64`, `u128` and `usize` (pointer size)
- Floating point: `f32`, `f64`
- `char` Unicode scalar values like `'a'`, `'α'` and `'∞'` (4 bytes each)
- `bool` either `true` or `false`
- The unit type `()`, whose only possible value is an empty tuple: `()`

Despite the value of a unit type being a tuple, it is not considered a compound type because it does not contain multiple values.

Compound Types

- Arrays like `[1, 2, 3]`
- Tuples like `(1, true)`

Variables can always be *type annotated*. Numbers may additionally be annotated via a *suffix* or *by default*. Integers default to `i32` and floats to `f64`. Note that Rust can also infer types from context.

```
1 fn main() {
2     // Variables can be type annotated.
3     let logical: bool = true;
4
5     let a_float: f64 = 1.0; // Regular annotation
6     let an_integer   = 5i32; // Suffix annotation
7
8     // Or a default will be used.
9     let default_float = 3.0; // `f64`
10    let default_integer = 7;   // `i32`
11
12    // A type can also be inferred from context.
13    let mut inferred_type = 12; // Type i64 is inferred from another line
14    inferred_type = 4294967296i64;
15
16    // A mutable variable's value can be changed.
17    let mut mutable = 12; // Mutable `i32`
18    mutable = 21;
19
20    // Error! The type of a variable can't be changed.
21    mutable = true;
22
23    // Variables can be overwritten with shadowing.
24    let mutable = true;
25 }
```

See also:

[the std library](#), [mut](#), [inference](#), and [shadowing](#)

Literals and operators

Integers `1`, floats `1.2`, characters `'a'`, strings `"abc"`, booleans `true` and the unit type `()` can be expressed using literals.

Integers can, alternatively, be expressed using hexadecimal, octal or binary notation using these prefixes respectively: `0x`, `0o` or `0b`.

Underscores can be inserted in numeric literals to improve readability, e.g. `1_000` is the same as `1000`, and `0.000_001` is the same as `0.000001`.

Rust also supports scientific [E-notation](#), e.g. `1e6`, `7.6e-4`. The associated type is `f64`.

We need to tell the compiler the type of the literals we use. For now, we'll use the `u32` suffix to indicate that the literal is an unsigned 32-bit integer, and the `i32` suffix to indicate that it's a signed 32-bit integer.

The operators available and their precedence in Rust are similar to other [C-like languages](#).

```
1 fn main() {
2     // Integer addition
3     println!("1 + 2 = {}", 1u32 + 2);
4
5     // Integer subtraction
6     println!("1 - 2 = {}", 1i32 - 2);
7     // TODO ^ Try changing `1i32` to `1u32` to see why the type is import
8
9     // Scientific notation
10    println!("1e4 is {}, -2.5e-3 is {}", 1e4, -2.5e-3);
11
12    // Short-circuiting boolean logic
13    println!("true AND false is {}", true && false);
14    println!("true OR false is {}", true || false);
15    println!("NOT true is {}", !true);
16
17    // Bitwise operations
18    println!("0011 AND 0101 is {:04b}", 0b0011u32 & 0b0101);
19    println!("0011 OR 0101 is {:04b}", 0b0011u32 | 0b0101);
20    println!("0011 XOR 0101 is {:04b}", 0b0011u32 ^ 0b0101);
21    println!("1 << 5 is {}", 1u32 << 5);
22    println!("0x80 >> 2 is 0x{:x}", 0x80u32 >> 2);
23
24    // Use underscores to improve readability!
25    println!("One million is written as {}", 1_000_000u32);
26 }
```

Tuples

A tuple is a collection of values of different types. Tuples are constructed using parentheses `()`, and each tuple itself is a value with type signature `(T1, T2, ...)`, where `T1`, `T2` are the types of its members. Functions can use tuples to return multiple values, as tuples can hold any number of values.

```
1 // Tuples can be used as function arguments and as return values.
2 fn reverse(pair: (i32, bool)) -> (bool, i32) {
3     // `let` can be used to bind the members of a tuple to variables.
4     let (int_param, bool_param) = pair;
5
6     (bool_param, int_param)
7 }
8
9 // The following struct is for the activity.
10 #[derive(Debug)]
11 struct Matrix(f32, f32, f32, f32);
12
13 fn main() {
14     // A tuple with a bunch of different types.
15     let long_tuple = (1u8, 2u16, 3u32, 4u64,
16                     -1i8, -2i16, -3i32, -4i64,
17                     0.1f32, 0.2f64,
18                     'a', true);
19
20     // Values can be extracted from the tuple using tuple indexing.
21     println!("Long tuple first value: {}", long_tuple.0);
22     println!("Long tuple second value: {}", long_tuple.1);
23
24     // Tuples can be tuple members.
25     let tuple_of_tuples = ((1u8, 2u16, 2u32), (4u64, -1i8), -2i16);
26
27     // Tuples are printable.
28     println!("tuple of tuples: {:?}", tuple_of_tuples);
29
30     // But long Tuples (more than 12 elements) cannot be printed.
31     //let too_long_tuple = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13);
32     //println!("Too long tuple: {:?}", too_long_tuple);
33     // TODO ^ Uncomment the above 2 lines to see the compiler error
34
35     let pair = (1, true);
36     println!("Pair is {:?}", pair);
37
38     println!("The reversed pair is {:?}", reverse(pair));
39
40     // To create one element tuples, the comma is required to tell them a
41     // from a literal surrounded by parentheses.
42     println!("One element tuple: {:?}", (5u32,));
43     println!("Just an integer: {:?}", (5u32));
44
45     // Tuples can be destructured to create bindings.
46     let tuple = (1, "hello", 4.5, true);
47
48     let (a, b, c, d) = tuple;
49     println!("{:?}", "{:?}", "{:?}", "{:?}", a, b, c, d);
50
51     let matrix = Matrix(1.1, 1.2, 2.1, 2.2);
52     println!("{:?}", matrix);
53 }
```

Activity

1. *Recap*: Add the `fmt::Display` trait to the `Matrix` struct in the above example, so that if you switch from printing the debug format `{:?}` to the display format `{}`, you see the following output:

```
( 1.1 1.2 )
( 2.1 2.2 )
```

You may want to refer back to the example for [print display](#).

2. Add a `transpose` function using the `reverse` function as a template, which accepts a matrix as an argument, and returns a matrix in which two elements have been swapped. For example:

```
println!("Matrix:\n{}", matrix);
println!("Transpose:\n{}", transpose(matrix));
```

Results in the output:

```
Matrix:
( 1.1 1.2 )
( 2.1 2.2 )
Transpose:
( 1.1 2.1 )
( 1.2 2.2 )
```

Arrays and Slices

An array is a collection of objects of the same type T , stored in contiguous memory.

Arrays are created using brackets `[]`, and their length, which is known at compile time, is part of their type signature `[T; length]`.

Slices are similar to arrays, but their length is not known at compile time. Instead, a slice is a two-word object; the first word is a pointer to the data, the second word the length of the slice. The word size is the same as `usize`, determined by the processor architecture, e.g. 64 bits on an x86-64. Slices can be used to borrow a section of an array and have the type signature `&[T]`.


```
1 use std::mem;
2
3 // This function borrows a slice.
4 fn analyze_slice(slice: &[i32]) {
5     println!("First element of the slice: {}", slice[0]);
6     println!("The slice has {} elements", slice.len());
7 }
8
9 fn main() {
10     // Fixed-size array (type signature is superfluous).
11     let xs: [i32; 5] = [1, 2, 3, 4, 5];
12
13     // All elements can be initialized to the same value.
14     let ys: [i32; 500] = [0; 500];
15
16     // Indexing starts at 0.
17     println!("First element of the array: {}", xs[0]);
18     println!("Second element of the array: {}", xs[1]);
19
20     // `len` returns the count of elements in the array.
21     println!("Number of elements in array: {}", xs.len());
22
23     // Arrays are stack allocated.
24     println!("Array occupies {} bytes", mem::size_of_val(&xs));
25
26     // Arrays can be automatically borrowed as slices.
27     println!("Borrow the whole array as a slice.");
28     analyze_slice(&xs);
29
30     // Slices can point to a section of an array.
31     // They are of the form [starting_index..ending_index].
32     // `starting_index` is the first position in the slice.
33     // `ending_index` is one more than the last position in the slice.
34     println!("Borrow a section of the array as a slice.");
35     analyze_slice(&ys[1 .. 4]);
36
37     // Example of empty slice `&[]`:
38     let empty_array: [u32; 0] = [];
39     assert_eq!(&empty_array, &[]);
40     assert_eq!(&empty_array, &[][..]); // Same but more verbose
41
42     // Arrays can be safely accessed using `.get`, which returns an
43     // `Option`. This can be matched as shown below, or used with
44     // `.expect()` if you would like the program to exit with a nice
45     // message instead of happily continue.
46     for i in 0..xs.len() + 1 { // Oops, one element too far!
47         match xs.get(i) {
48             Some(xval) => println!("{}", i, xval),
49             None => println!("Slow down! {} is too far!", i),
50         }
51     }
52
53     // Out of bound indexing on array causes compile time error.
54     //println!("{}", xs[5]);
55     // Out of bound indexing on slice causes runtime error.
56     //println!("{}", xs[..][5]);
57 }
```

Custom Types

Rust custom data types are formed mainly through the two keywords:

- `struct` : define a structure
- `enum` : define an enumeration

Constants can also be created via the `const` and `static` keywords.

Structures

There are three types of structures ("structs") that can be created using the `struct` keyword:

- Tuple structs, which are, basically, named tuples.
- The classic [C structs](#)
- Unit structs, which are field-less, are useful for generics.

```
1 // An attribute to hide warnings for unused code.
2 #![allow(dead_code)]
3
4 #[derive(Debug)]
5 struct Person {
6     name: String,
7     age: u8,
8 }
9
10 // A unit struct
11 struct Unit;
12
13 // A tuple struct
14 struct Pair(i32, f32);
15
16 // A struct with two fields
17 struct Point {
18     x: f32,
19     y: f32,
20 }
21
22 // Structs can be reused as fields of another struct
23 struct Rectangle {
24     // A rectangle can be specified by where the top left and bottom right
25     // corners are in space.
26     top_left: Point,
27     bottom_right: Point,
28 }
29
30 fn main() {
31     // Create struct with field init shorthand
32     let name = String::from("Peter");
33     let age = 27;
34     let peter = Person { name, age };
35
36     // Print debug struct
37     println!("{:?}", peter);
38
39     // Instantiate a `Point`
40     let point: Point = Point { x: 10.3, y: 0.4 };
41
42     // Access the fields of the point
43     println!("point coordinates: ({}, {})", point.x, point.y);
44
45     // Make a new point by using struct update syntax to use the fields of
46     // other one
47     let bottom_right = Point { x: 5.2, ..point };
48
49     // `bottom_right.y` will be the same as `point.y` because we used that
50     // from `point`
51     println!("second point: ({}, {})", bottom_right.x, bottom_right.y);
52
53     // Destructure the point using a `let` binding
54     let Point { x: left_edge, y: top_edge } = point;
55
56     let _rectangle = Rectangle {
57         // struct instantiation is an expression too
```

Activity

1. Add a function `rect_area` which calculates the area of a `Rectangle` (try using nested destructuring).
2. Add a function `square` which takes a `Point` and a `f32` as arguments, and returns a `Rectangle` with its top left corner on the point, and a width and height corresponding to the `f32`.

See also

[attributes](#) , and [destructuring](#)

Enums

The `enum` keyword allows the creation of a type which may be one of a few different variants. Any variant which is valid as a `struct` is also valid in an `enum`.

```
1 // Create an `enum` to classify a web event. Note how both
2 // names and type information together specify the variant:
3 // `PageLoad != PageUnload` and `KeyPress(char) != Paste(String)`.
4 // Each is different and independent.
5 enum WebEvent {
6     // An `enum` variant may either be `unit-like`,
7     PageLoad,
8     PageUnload,
9     // like tuple structs,
10    KeyPress(char),
11    Paste(String),
12    // or c-like structures.
13    Click { x: i64, y: i64 },
14 }
15
16 // A function which takes a `WebEvent` enum as an argument and
17 // returns nothing.
18 fn inspect(event: WebEvent) {
19     match event {
20         WebEvent::PageLoad => println!("page loaded"),
21         WebEvent::PageUnload => println!("page unloaded"),
22         // Destructure `c` from inside the `enum` variant.
23         WebEvent::KeyPress(c) => println!("pressed '{}'.", c),
24         WebEvent::Paste(s) => println!("pasted \"{}\".", s),
25         // Destructure `Click` into `x` and `y`.
26         WebEvent::Click { x, y } => {
27             println!("clicked at x={}, y={}.", x, y);
28         },
29     }
30 }
31
32 fn main() {
33     let pressed = WebEvent::KeyPress('x');
34     // `to_owned()` creates an owned `String` from a string slice.
35     let pasted = WebEvent::Paste("my text".to_owned());
36     let click = WebEvent::Click { x: 20, y: 80 };
37     let load = WebEvent::PageLoad;
38     let unload = WebEvent::PageUnload;
39
40     inspect(pressed);
41     inspect(pasted);
42     inspect(click);
43     inspect(load);
44     inspect(unload);
45 }
46
```

Type aliases

If you use a type alias, you can refer to each enum variant via its alias. This might be useful if the enum's name is too long or too generic, and you want to rename it.

```
1 enum VeryVerboseEnumOfThingsToDoWithNumbers {
2     Add,
3     Subtract,
4 }
5
6 // Creates a type alias
7 type Operations = VeryVerboseEnumOfThingsToDoWithNumbers;
8
9 fn main() {
10     // We can refer to each variant via its alias, not its long and incon
11     // name.
12     let x = Operations::Add;
13 }
```

The most common place you'll see this is in `impl` blocks using the `self` alias.

```
1 enum VeryVerboseEnumOfThingsToDoWithNumbers {
2     Add,
3     Subtract,
4 }
5
6 impl VeryVerboseEnumOfThingsToDoWithNumbers {
7     fn run(&self, x: i32, y: i32) -> i32 {
8         match self {
9             Self::Add => x + y,
10            Self::Subtract => x - y,
11        }
12    }
13 }
```

To learn more about enums and type aliases, you can read the [stabilization report](#) from when this feature was stabilized into Rust.

See also:

[match](#), [fn](#), and [string](#), "Type alias enum variants" RFC

use

The `use` declaration can be used so manual scoping isn't needed:

```
1 // An attribute to hide warnings for unused code.
2 #![allow(dead_code)]
3
4 enum Status {
5     Rich,
6     Poor,
7 }
8
9 enum Work {
10     Civilian,
11     Soldier,
12 }
13
14 fn main() {
15     // Explicitly `use` each name so they are available without
16     // manual scoping.
17     use crate::Status::{Poor, Rich};
18     // Automatically `use` each name inside `Work`.
19     use crate::Work::*;
20
21     // Equivalent to `Status::Poor`.
22     let status = Poor;
23     // Equivalent to `Work::Civilian`.
24     let work = Civilian;
25
26     match status {
27         // Note the lack of scoping because of the explicit `use` above.
28         Rich => println!("The rich have lots of money!"),
29         Poor => println!("The poor have no money..."),
30     }
31
32     match work {
33         // Note again the lack of scoping.
34         Civilian => println!("Civilians work!"),
35         Soldier => println!("Soldiers fight!"),
36     }
37 }
```

See also:

[match](#) and [use](#)

C-like

`enum` can also be used as C-like enums.

```
1 // An attribute to hide warnings for unused code.
2 #![allow(dead_code)]
3
4 // enum with implicit discriminator (starts at 0)
5 enum Number {
6     Zero,
7     One,
8     Two,
9 }
10
11 // enum with explicit discriminator
12 enum Color {
13     Red = 0xff0000,
14     Green = 0x00ff00,
15     Blue = 0x0000ff,
16 }
17
18 fn main() {
19     // `enums` can be cast as integers.
20     println!("zero is {}", Number::Zero as i32);
21     println!("one is {}", Number::One as i32);
22
23     println!("roses are #{:06x}", Color::Red as i32);
24     println!("violets are #{:06x}", Color::Blue as i32);
25 }
```

See also:

[casting](#)

Testcase: linked-list

A common way to implement a linked-list is via `enums` :

```
1 use crate::List::*;
2
3 enum List {
4     // Cons: Tuple struct that wraps an element and a pointer to the next
5     Cons(u32, Box<List>),
6     // Nil: A node that signifies the end of the linked list
7     Nil,
8 }
9
10 // Methods can be attached to an enum
11 impl List {
12     // Create an empty list
13     fn new() -> List {
14         // `Nil` has type `List`
15         Nil
16     }
17
18     // Consume a list, and return the same list with a new element at its
19     fn prepend(self, elem: u32) -> List {
20         // `Cons` also has type List
21         Cons(elem, Box::new(self))
22     }
23
24     // Return the length of the list
25     fn len(&self) -> u32 {
26         // `self` has to be matched, because the behavior of this method
27         // depends on the variant of `self`
28         // `self` has type `&List`, and `*self` has type `List`, matching
29         // concrete type `T` is preferred over a match on a reference `&T`
30         // after Rust 2018 you can use self here and tail (with no ref) b
31         // rust will infer &s and ref tail.
32         // See https://doc.rust-lang.org/edition-guide/rust-2018/ownership
33         match *self {
34             // Can't take ownership of the tail, because `self` is borrow
35             // instead take a reference to the tail
36             Cons(_, ref tail) => 1 + tail.len(),
37             // Base Case: An empty list has zero length
38             Nil => 0
39         }
40     }
41
42     // Return representation of the list as a (heap allocated) string
43     fn stringify(&self) -> String {
44         match *self {
45             Cons(head, ref tail) => {
46                 // `format!` is similar to `print!`, but returns a heap
47                 // allocated string instead of printing to the console
48                 format!("{}", {}, head, tail.stringify())
49             },
50             Nil => {
51                 format!("{}", "Nil")
52             },
53         }
54     }
55 }
56
57 fn main() {
```

See also:

[Box](#) and [methods](#)

constants

Rust has two different types of constants which can be declared in any scope including global. Both require explicit type annotation:

- `const` : An unchangeable value (the common case).
- `static` : A possibly mutable variable with `'static` lifetime. The static lifetime is inferred and does not have to be specified. Accessing or modifying a mutable static variable is `unsafe`.

```
1 // Globals are declared outside all other scopes.
2 static LANGUAGE: &str = "Rust";
3 const THRESHOLD: i32 = 10;
4
5 fn is_big(n: i32) -> bool {
6     // Access constant in some function
7     n > THRESHOLD
8 }
9
10 fn main() {
11     let n = 16;
12
13     // Access constant in the main thread
14     println!("This is {}", LANGUAGE);
15     println!("The threshold is {}", THRESHOLD);
16     println!("{}", n, if is_big(n) { "big" } else { "small" });
17
18     // Error! Cannot modify a `const`.
19     THRESHOLD = 5;
20     // FIXME ^ Comment out this line
21 }
```

See also:

[The `const` / `static` RFC](#), `'static` lifetime

Variable Bindings

Rust provides type safety via static typing. Variable bindings can be type annotated when declared. However, in most cases, the compiler will be able to infer the type of the variable from the context, heavily reducing the annotation burden.

Values (like literals) can be bound to variables, using the `let` binding.

```
1 fn main() {
2     let an_integer = 1u32;
3     let a_boolean = true;
4     let unit = ();
5
6     // copy `an_integer` into `copied_integer`
7     let copied_integer = an_integer;
8
9     println!("An integer: {:?}", copied_integer);
10    println!("A boolean: {:?}", a_boolean);
11    println!("Meet the unit value: {:?}", unit);
12
13    // The compiler warns about unused variable bindings; these warnings
14    // be silenced by prefixing the variable name with an underscore
15    let _unused_variable = 3u32;
16
17    let noisy_unused_variable = 2u32;
18    // FIXME ^ Prefix with an underscore to suppress the warning
19    // Please note that warnings may not be shown in a browser
20 }
```

Mutability

Variable bindings are immutable by default, but this can be overridden using the `mut` modifier.

```
1 fn main() {
2     let _immutable_binding = 1;
3     let mut mutable_binding = 1;
4
5     println!("Before mutation: {}", mutable_binding);
6
7     // Ok
8     mutable_binding += 1;
9
10    println!("After mutation: {}", mutable_binding);
11
12    // Error! Cannot assign a new value to an immutable variable
13    _immutable_binding += 1;
14 }
```

The compiler will throw a detailed diagnostic about mutability errors.

Scope and Shadowing

Variable bindings have a scope, and are constrained to live in a *block*. A block is a collection of statements enclosed by braces `{}`.

```
1 fn main() {
2     // This binding lives in the main function
3     let long_lived_binding = 1;
4
5     // This is a block, and has a smaller scope than the main function
6     {
7         // This binding only exists in this block
8         let short_lived_binding = 2;
9
10        println!("inner short: {}", short_lived_binding);
11    }
12    // End of the block
13
14    // Error! `short_lived_binding` doesn't exist in this scope
15    println!("outer short: {}", short_lived_binding);
16    // FIXME ^ Comment out this line
17
18    println!("outer long: {}", long_lived_binding);
19 }
```

Also, [variable shadowing](#) is allowed.

```
1 fn main() {
2     let shadowed_binding = 1;
3
4     {
5         println!("before being shadowed: {}", shadowed_binding);
6
7         // This binding *shadows* the outer one
8         let shadowed_binding = "abc";
9
10        println!("shadowed in inner block: {}", shadowed_binding);
11    }
12    println!("outside inner block: {}", shadowed_binding);
13
14    // This binding *shadows* the previous binding
15    let shadowed_binding = 2;
16    println!("shadowed in outer block: {}", shadowed_binding);
17 }
```


Declare first

It's possible to declare variable bindings first, and initialize them later. However, this form is seldom used, as it may lead to the use of uninitialized variables.

```
1 fn main() {
2     // Declare a variable binding
3     let a_binding;
4
5     {
6         let x = 2;
7
8         // Initialize the binding
9         a_binding = x * x;
10    }
11
12    println!("a binding: {}", a_binding);
13
14    let another_binding;
15
16    // Error! Use of uninitialized binding
17    println!("another binding: {}", another_binding);
18    // FIXME ^ Comment out this line
19
20    another_binding = 1;
21
22    println!("another binding: {}", another_binding);
23 }
```

The compiler forbids use of uninitialized variables, as this would lead to undefined behavior.

Freezing

When data is bound by the same name immutably, it also *freezes*. *Frozen* data can't be modified until the immutable binding goes out of scope:

```
1 fn main() {
2     let mut _mutable_integer = 7i32;
3
4     {
5         // Shadowing by immutable `_mutable_integer`
6         let _mutable_integer = _mutable_integer;
7
8         // Error! `_mutable_integer` is frozen in this scope
9         _mutable_integer = 50;
10        // FIXME ^ Comment out this line
11
12        // `_mutable_integer` goes out of scope
13    }
14
15    // Ok! `_mutable_integer` is not frozen in this scope
16    _mutable_integer = 3;
17 }
```

Types

Rust provides several mechanisms to change or define the type of primitive and user defined types. The following sections cover:

- [Casting](#) between primitive types
- Specifying the desired type of [literals](#)
- Using [type inference](#)
- [Aliasing](#) types

Casting

Rust provides no implicit type conversion (coercion) between primitive types. But, explicit type conversion (casting) can be performed using the `as` keyword.

Rules for converting between integral types follow C conventions generally, except in cases where C has undefined behavior. The behavior of all casts between integral types is well defined in Rust.

```
1 // Suppress all warnings from casts which overflow.
2 #![allow(overflowing_literals)]
3
4 fn main() {
5     let decimal = 65.4321_f32;
6
7     // Error! No implicit conversion
8     let integer: u8 = decimal;
9     // FIXME ^ Comment out this line
10
11     // Explicit conversion
12     let integer = decimal as u8;
13     let character = integer as char;
14
15     // Error! There are limitations in conversion rules.
16     // A float cannot be directly converted to a char.
17     let character = decimal as char;
18     // FIXME ^ Comment out this line
19
20     println!("Casting: {} -> {} -> {}", decimal, integer, character);
21
22     // when casting any value to an unsigned type, T,
23     // T::MAX + 1 is added or subtracted until the value
24     // fits into the new type
25
26     // 1000 already fits in a u16
27     println!("1000 as a u16 is: {}", 1000 as u16);
28
29     // 1000 - 256 - 256 - 256 = 232
30     // Under the hood, the first 8 least significant bits (LSB) are kept,
31     // while the rest towards the most significant bit (MSB) get truncate
32     println!("1000 as a u8 is : {}", 1000 as u8);
33     // -1 + 256 = 255
34     println!(" -1 as a u8 is : {}", (-1i8) as u8);
35
36     // For positive numbers, this is the same as the modulus
37     println!("1000 mod 256 is : {}", 1000 % 256);
38
39     // When casting to a signed type, the (bitwise) result is the same as
40     // first casting to the corresponding unsigned type. If the most sign
41     // bit of that value is 1, then the value is negative.
42
43     // Unless it already fits, of course.
44     println!(" 128 as a i16 is: {}", 128 as i16);
45
46     // In boundary case 128 value in 8-bit two's complement representatio
47     println!(" 128 as a i8 is : {}", 128 as i8);
48
49     // repeating the example above
50     // 1000 as u8 -> 232
51     println!("1000 as a u8 is : {}", 1000 as u8);
52     // and the value of 232 in 8-bit two's complement representation is -
53     println!(" 232 as a i8 is : {}", 232 as i8);
54
55     // Since Rust 1.45, the `as` keyword performs a *saturating cast*
56     // when casting from float to int. If the floating point value exceed
57     // the upper bound or is less than the lower bound, the returned valu
```


Literals

Numeric literals can be type annotated by adding the type as a suffix. As an example, to specify that the literal `42` should have the type `i32`, write `42i32`.

The type of unsuffixed numeric literals will depend on how they are used. If no constraint exists, the compiler will use `i32` for integers, and `f64` for floating-point numbers.

```
1 fn main() {
2     // Suffixed literals, their types are known at initialization
3     let x = 1u8;
4     let y = 2u32;
5     let z = 3f32;
6
7     // Unsuffixed literals, their types depend on how they are used
8     let i = 1;
9     let f = 1.0;
10
11     // `size_of_val` returns the size of a variable in bytes
12     println!("size of `x` in bytes: {}", std::mem::size_of_val(&x));
13     println!("size of `y` in bytes: {}", std::mem::size_of_val(&y));
14     println!("size of `z` in bytes: {}", std::mem::size_of_val(&z));
15     println!("size of `i` in bytes: {}", std::mem::size_of_val(&i));
16     println!("size of `f` in bytes: {}", std::mem::size_of_val(&f));
17 }
```

There are some concepts used in the previous code that haven't been explained yet, here's a brief explanation for the impatient readers:

- `std::mem::size_of_val` is a function, but called with its *full path*. Code can be split in logical units called *modules*. In this case, the `size_of_val` function is defined in the `mem` module, and the `mem` module is defined in the `std` *crate*. For more details, see [modules](#) and [crates](#).

Inference

The type inference engine is pretty smart. It does more than looking at the type of the value expression during an initialization. It also looks at how the variable is used afterwards to infer its type. Here's an advanced example of type inference:

```
1 fn main() {
2     // Because of the annotation, the compiler knows that `elem` has type
3     let elem = 5u8;
4
5     // Create an empty vector (a growable array).
6     let mut vec = Vec::new();
7     // At this point the compiler doesn't know the exact type of `vec`, i
8     // just knows that it's a vector of something (`Vec<_>`).
9
10    // Insert `elem` in the vector.
11    vec.push(elem);
12    // Aha! Now the compiler knows that `vec` is a vector of `u8`s (`Vec<u8>`)
13    // TODO ^ Try commenting out the `vec.push(elem)` line
14
15    println!("{:?}", vec);
16 }
```

No type annotation of variables was needed, the compiler is happy and so is the programmer!

Aliasing

The `type` statement can be used to give a new name to an existing type. Types must have `UpperCamelCase` names, or the compiler will raise a warning. The exception to this rule are the primitive types: `usize`, `f32`, etc.

```
1 // `NanoSecond`, `Inch`, and `U64` are new names for `u64`.
2 type NanoSecond = u64;
3 type Inch = u64;
4 type U64 = u64;
5
6 fn main() {
7     // `NanoSecond` = `Inch` = `U64` = `u64`.
8     let nanoseconds: NanoSecond = 5 as U64;
9     let inches: Inch = 2 as U64;
10
11     // Note that type aliases *don't* provide any extra type safety, beca
12     // aliases are *not* new types
13     println!("{}", nanoseconds + {} inches = {} unit?",
14               nanoseconds,
15               inches,
16               nanoseconds + inches);
17 }
```

The main use of aliases is to reduce boilerplate; for example the `io::Result<T>` type is an alias for the `Result<T, io::Error>` type.

See also:

[Attributes](#)

Conversion

Primitive types can be converted to each other through [casting](#).

Rust addresses conversion between custom types (i.e., `struct` and `enum`) by the use of [traits](#). The generic conversions will use the [From](#) and [Into](#) traits. However there are more specific ones for the more common cases, in particular when converting to and from `String`s.

From and Into

The `From` and `Into` traits are inherently linked, and this is actually part of its implementation. If you are able to convert type A from type B, then it should be easy to believe that we should be able to convert type B to type A.

From

The `From` trait allows for a type to define how to create itself from another type, hence providing a very simple mechanism for converting between several types. There are numerous implementations of this trait within the standard library for conversion of primitive and common types.

For example we can easily convert a `str` into a `String`

```
let my_str = "hello";
let my_string = String::from(my_str);
```

We can do similar for defining a conversion for our own type.

```
1 use std::convert::From;
2
3 #[derive(Debug)]
4 struct Number {
5     value: i32,
6 }
7
8 impl From<i32> for Number {
9     fn from(item: i32) -> Self {
10         Number { value: item }
11     }
12 }
13
14 fn main() {
15     let num = Number::from(30);
16     println!("My number is {:?}", num);
17 }
```

Into

The `Into` trait is simply the reciprocal of the `From` trait. That is, if you have implemented the `From` trait for your type, `Into` will call it when necessary.

Using the `Into` trait will typically require specification of the type to convert into as the

compiler is unable to determine this most of the time. However this is a small trade-off considering we get the functionality for free.

```
1 use std::convert::Into;
2
3 #[derive(Debug)]
4 struct Number {
5     value: i32,
6 }
7
8 impl Into<Number> for i32 {
9     fn into(self) -> Number {
10         Number { value: self }
11     }
12 }
13
14 fn main() {
15     let int = 5;
16     // Try removing the type annotation
17     let num: Number = int.into();
18     println!("My number is {:?}", num);
19 }
```

TryFrom and TryInto

Similar to [From and Into](#), [TryFrom](#) and [TryInto](#) are generic traits for converting between types. Unlike [From / Into](#), the [TryFrom / TryInto](#) traits are used for fallible conversions, and as such, return [Result](#)s.

```
1 use std::convert::TryFrom;
2 use std::convert::TryInto;
3
4 #[derive(Debug, PartialEq)]
5 struct EvenNumber(i32);
6
7 impl TryFrom<i32> for EvenNumber {
8     type Error = ();
9
10    fn try_from(value: i32) -> Result<Self, Self::Error> {
11        if value % 2 == 0 {
12            Ok(EvenNumber(value))
13        } else {
14            Err(())
15        }
16    }
17 }
18
19 fn main() {
20     // TryFrom
21
22     assert_eq!(EvenNumber::try_from(8), Ok(EvenNumber(8)));
23     assert_eq!(EvenNumber::try_from(5), Err(()));
24
25     // TryInto
26
27     let result: Result<EvenNumber, ()> = 8i32.try_into();
28     assert_eq!(result, Ok(EvenNumber(8)));
29     let result: Result<EvenNumber, ()> = 5i32.try_into();
30     assert_eq!(result, Err(()));
31 }
```

To and from Strings

Converting to String

To convert any type to a `String` is as simple as implementing the `ToString` trait for the type. Rather than doing so directly, you should implement the `fmt::Display` trait which automatically provides `ToString` and also allows printing the type as discussed in the section on `println!`.

```
1 use std::fmt;
2
3 struct Circle {
4     radius: i32
5 }
6
7 impl fmt::Display for Circle {
8     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
9         write!(f, "Circle of radius {}", self.radius)
10    }
11 }
12
13 fn main() {
14     let circle = Circle { radius: 6 };
15     println!("{}", circle.to_string());
16 }
```

Parsing a String

One of the more common types to convert a string into a number. The idiomatic approach to this is to use the `parse` function and either to arrange for type inference or to specify the type to parse using the 'turbofish' syntax. Both alternatives are shown in the following example.

This will convert the string into the type specified as long as the `FromStr` trait is implemented for that type. This is implemented for numerous types within the standard library. To obtain this functionality on a user defined type simply implement the `FromStr` trait for that type.

```
1 fn main() {
2     let parsed: i32 = "5".parse().unwrap();
3     let turbo_parsed = "10".parse::<i32>().unwrap();
4
5     let sum = parsed + turbo_parsed;
6     println!("Sum: {:?}", sum);
7 }
```

Expressions

A Rust program is (mostly) made up of a series of statements:

```
1 fn main() {  
2     // statement  
3     // statement  
4     // statement  
5 }
```

There are a few kinds of statements in Rust. The most common two are declaring a variable binding, and using a `; with an expression:`

```
1 fn main() {  
2     // variable binding  
3     let x = 5;  
4  
5     // expression;  
6     x;  
7     x + 1;  
8     15;  
9 }
```

Blocks are expressions too, so they can be used as values in assignments. The last expression in the block will be assigned to the place expression such as a local variable. However, if the last expression of the block ends with a semicolon, the return value will be `()`.

```
1 fn main() {  
2     let x = 5u32;  
3  
4     let y = {  
5         let x_squared = x * x;  
6         let x_cube = x_squared * x;  
7  
8         // This expression will be assigned to `y`  
9         x_cube + x_squared + x  
10    };  
11  
12    let z = {  
13        // The semicolon suppresses this expression and `()` is assigned  
14        2 * x;  
15    };  
16  
17    println!("x is {:?}", x);  
18    println!("y is {:?}", y);  
19    println!("z is {:?}", z);  
20 }
```

Flow of Control

An integral part of any programming language are ways to modify control flow: `if / else` , `for` , and others. Let's talk about them in Rust.

if/else

Branching with `if - else` is similar to other languages. Unlike many of them, the boolean condition doesn't need to be surrounded by parentheses, and each condition is followed by a block. `if - else` conditionals are expressions, and, all branches must return the same type.

```
1 fn main() {
2     let n = 5;
3
4     if n < 0 {
5         print!("{}", is negative", n);
6     } else if n > 0 {
7         print!("{}", is positive", n);
8     } else {
9         print!("{}", is zero", n);
10    }
11
12    let big_n =
13        if n < 10 && n > -10 {
14            println!("{}", and is a small number, increase ten-fold");
15
16            // This expression returns an `i32`.
17            10 * n
18        } else {
19            println!("{}", and is a big number, halve the number");
20
21            // This expression must return an `i32` as well.
22            n / 2
23            // TODO ^ Try suppressing this expression with a semicolon.
24        };
25    // ^ Don't forget to put a semicolon here! All `let` bindings need
26
27    println!("{}", -> {}", n, big_n);
28 }
```

loop

Rust provides a `loop` keyword to indicate an infinite loop.

The `break` statement can be used to exit a loop at anytime, whereas the `continue` statement can be used to skip the rest of the iteration and start a new one.

```
1 fn main() {
2     let mut count = 0u32;
3
4     println!("Let's count until infinity!");
5
6     // Infinite loop
7     loop {
8         count += 1;
9
10        if count == 3 {
11            println!("three");
12
13            // Skip the rest of this iteration
14            continue;
15        }
16
17        println!("{}", count);
18
19        if count == 5 {
20            println!("OK, that's enough");
21
22            // Exit this loop
23            break;
24        }
25    }
26 }
```

Nesting and labels

It's possible to `break` or `continue` outer loops when dealing with nested loops. In these cases, the loops must be annotated with some `'label'`, and the label must be passed to the `break` / `continue` statement.

```
1  #![allow(unreachable_code, unused_labels)]
2
3  fn main() {
4      'outer: loop {
5          println!("Entered the outer loop");
6
7          'inner: loop {
8              println!("Entered the inner loop");
9
10             // This would break only the inner loop
11             //break;
12
13             // This breaks the outer loop
14             break 'outer;
15         }
16
17         println!("This point will never be reached");
18     }
19
20     println!("Exited the outer loop");
21 }
```

Returning from loops

One of the uses of a `loop` is to retry an operation until it succeeds. If the operation returns a value though, you might need to pass it to the rest of the code: put it after the `break`, and it will be returned by the `loop` expression.

```
1 fn main() {  
2     let mut counter = 0;  
3  
4     let result = loop {  
5         counter += 1;  
6  
7         if counter == 10 {  
8             break counter * 2;  
9         }  
10    };  
11  
12    assert_eq!(result, 20);  
13 }
```

while

The `while` keyword can be used to run a loop while a condition is true.

Let's write the infamous [FizzBuzz](#) using a `while` loop.

```
1 fn main() {
2     // A counter variable
3     let mut n = 1;
4
5     // Loop while `n` is less than 101
6     while n < 101 {
7         if n % 15 == 0 {
8             println!("fizzbuzz");
9         } else if n % 3 == 0 {
10            println!("fizz");
11        } else if n % 5 == 0 {
12            println!("buzz");
13        } else {
14            println!("{}", n);
15        }
16
17        // Increment counter
18        n += 1;
19    }
20 }
```

for loops

for and range

The `for in` construct can be used to iterate through an `Iterator`. One of the easiest ways to create an iterator is to use the range notation `a..b`. This yields values from `a` (inclusive) to `b` (exclusive) in steps of one.

Let's write FizzBuzz using `for` instead of `while`.

```
1 fn main() {
2     // `n` will take the values: 1, 2, ..., 100 in each iteration
3     for n in 1..101 {
4         if n % 15 == 0 {
5             println!("fizzbuzz");
6         } else if n % 3 == 0 {
7             println!("fizz");
8         } else if n % 5 == 0 {
9             println!("buzz");
10        } else {
11            println!("{}", n);
12        }
13    }
14 }
```

Alternatively, `a..=b` can be used for a range that is inclusive on both ends. The above can be written as:

```
1 fn main() {
2     // `n` will take the values: 1, 2, ..., 100 in each iteration
3     for n in 1..=100 {
4         if n % 15 == 0 {
5             println!("fizzbuzz");
6         } else if n % 3 == 0 {
7             println!("fizz");
8         } else if n % 5 == 0 {
9             println!("buzz");
10        } else {
11            println!("{}", n);
12        }
13    }
14 }
```

for and iterators

The `for in` construct is able to interact with an `Iterator` in several ways. As discussed

in the section on the [Iterator](#) trait, by default the `for` loop will apply the `into_iter` function to the collection. However, this is not the only means of converting collections into iterators.

`into_iter`, `iter` and `iter_mut` all handle the conversion of a collection into an iterator in different ways, by providing different views on the data within.

- `iter` - This borrows each element of the collection through each iteration. Thus leaving the collection untouched and available for reuse after the loop.

```
1 fn main() {
2     let names = vec!["Bob", "Frank", "Ferris"];
3
4     for name in names.iter() {
5         match name {
6             &"Ferris" => println!("There is a rustacean among us!"),
7             // TODO ^ Try deleting the & and matching just "Ferris"
8             _ => println!("Hello {}", name),
9         }
10    }
11
12    println!("names: {:?}", names);
13 }
```

- `into_iter` - This consumes the collection so that on each iteration the exact data is provided. Once the collection has been consumed it is no longer available for reuse as it has been 'moved' within the loop.

```
1 fn main() {
2     let names = vec!["Bob", "Frank", "Ferris"];
3
4     for name in names.into_iter() {
5         match name {
6             "Ferris" => println!("There is a rustacean among us!"),
7             _ => println!("Hello {}", name),
8         }
9     }
10
11    println!("names: {:?}", names);
12    // FIXME ^ Comment out this line
13 }
```

- `iter_mut` - This mutably borrows each element of the collection, allowing for the collection to be modified in place.

```
1 fn main() {
2     let mut names = vec!["Bob", "Frank", "Ferris"];
3
4     for name in names.iter_mut() {
5         *name = match name {
6             &mut "Ferris" => "There is a rustacean among us!",
7             _ => "Hello",
8         }
9     }
10
11     println!("names: {:?}", names);
12 }
```

In the above snippets note the type of `match` branch, that is the key difference in the types of iteration. The difference in type then of course implies differing actions that are able to be performed.

See also:

[Iterator](#)

match

Rust provides pattern matching via the `match` keyword, which can be used like a C `switch`. The first matching arm is evaluated and all possible values must be covered.

```
1 fn main() {
2     let number = 13;
3     // TODO ^ Try different values for `number`
4
5     println!("Tell me about {}", number);
6     match number {
7         // Match a single value
8         1 => println!("One!"),
9         // Match several values
10        2 | 3 | 5 | 7 | 11 => println!("This is a prime"),
11        // TODO ^ Try adding 13 to the list of prime values
12        // Match an inclusive range
13        13..=19 => println!("A teen"),
14        // Handle the rest of cases
15        _ => println!("Ain't special"),
16        // TODO ^ Try commenting out this catch-all arm
17    }
18
19    let boolean = true;
20    // Match is an expression too
21    let binary = match boolean {
22        // The arms of a match must cover all the possible values
23        false => 0,
24        true => 1,
25        // TODO ^ Try commenting out one of these arms
26    };
27
28    println!("{}", boolean, binary);
29 }
```

Destructuring

A `match` block can destructure items in a variety of ways.

- [Destructuring Tuples](#)
- [Destructuring Arrays and Slices](#)
- [Destructuring Enums](#)
- [Destructuring Pointers](#)
- [Destructuring Structures](#)

tuples

Tuples can be destructured in a `match` as follows:

```
1 fn main() {
2     let triple = (0, -2, 3);
3     // TODO ^ Try different values for `triple`
4
5     println!("Tell me about {:?}", triple);
6     // Match can be used to destructure a tuple
7     match triple {
8         // Destructure the second and third elements
9         (0, y, z) => println!("First is `0`, `y` is {:?}, and `z` is {:?}", y, z),
10        (1, ..) => println!("First is `1` and the rest doesn't matter"),
11        (.., 2) => println!("last is `2` and the rest doesn't matter"),
12        (3, .., 4) => println!("First is `3`, last is `4`, and the rest is {:?}", ..),
13        // `..` can be used to ignore the rest of the tuple
14        _ => println!("It doesn't matter what they are"),
15        // `_` means don't bind the value to a variable
16    }
17 }
```

See also:

[Tuples](#)

arrays/slices

Like tuples, arrays and slices can be destructured this way:

```

1 fn main() {
2     // Try changing the values in the array, or make it a slice!
3     let array = [1, -2, 6];
4
5     match array {
6         // Binds the second and the third elements to the respective vari
7         [0, second, third] =>
8             println!("array[0] = 0, array[1] = {}, array[2] = {}", second
9
10        // Single values can be ignored with _
11        [1, _, third] => println!(
12            "array[0] = 1, array[2] = {} and array[1] was ignored",
13            third
14        ),
15
16        // You can also bind some and ignore the rest
17        [-1, second, ..] => println!(
18            "array[0] = -1, array[1] = {} and all the other ones were ign
19            second
20        ),
21        // The code below would not compile
22        // [-1, second] => ...
23
24        // Or store them in another array/slice (the type depends on
25        // that of the value that is being matched against)
26        [3, second, tail @ ..] => println!(
27            "array[0] = 3, array[1] = {} and the other elements were {:?}
28            second, tail
29        ),
30
31        // Combining these patterns, we can, for example, bind the first
32        // last values, and store the rest of them in a single array
33        [first, middle @ .., last] => println!(
34            "array[0] = {}, middle = {:?}, array[2] = {}",
35            first, middle, last
36        ),
37    }
38 }

```

See also:

[Arrays and Slices](#) and [Binding](#) for @ sigil

enums

An `enum` is destructured similarly:

```
1 // `allow` required to silence warnings because only
2 // one variant is used.
3 #[allow(dead_code)]
4 enum Color {
5     // These 3 are specified solely by their name.
6     Red,
7     Blue,
8     Green,
9     // These likewise tie `u32` tuples to different names: color models.
10    RGB(u32, u32, u32),
11    HSV(u32, u32, u32),
12    HSL(u32, u32, u32),
13    CMY(u32, u32, u32),
14    CMYK(u32, u32, u32, u32),
15 }
16
17 fn main() {
18     let color = Color::RGB(122, 17, 40);
19     // TODO ^ Try different variants for `color`
20
21     println!("What color is it?");
22     // An `enum` can be destructured using a `match`.
23     match color {
24         Color::Red => println!("The color is Red!"),
25         Color::Blue => println!("The color is Blue!"),
26         Color::Green => println!("The color is Green!"),
27         Color::RGB(r, g, b) =>
28             println!("Red: {}, green: {}, and blue: {}!", r, g, b),
29         Color::HSV(h, s, v) =>
30             println!("Hue: {}, saturation: {}, value: {}!", h, s, v),
31         Color::HSL(h, s, l) =>
32             println!("Hue: {}, saturation: {}, lightness: {}!", h, s, l),
33         Color::CMY(c, m, y) =>
34             println!("Cyan: {}, magenta: {}, yellow: {}!", c, m, y),
35         Color::CMYK(c, m, y, k) =>
36             println!("Cyan: {}, magenta: {}, yellow: {}, key (black): {}!",
37                 c, m, y, k),
38         // Don't need another arm because all variants have been examined
39     }
40 }
```

See also:

[#\[allow\(...\)\]](#), [color models](#) and [enum](#)

pointers/ref

For pointers, a distinction needs to be made between destructuring and dereferencing as they are different concepts which are used differently from languages like C/C++.

- Dereferencing uses `*`
- Destructuring uses `&`, `ref`, and `ref mut`

```
1 fn main() {
2     // Assign a reference of type `i32`. The `&` signifies there
3     // is a reference being assigned.
4     let reference = &4;
5
6     match reference {
7         // If `reference` is pattern matched against `&val`, it results
8         // in a comparison like:
9         // `&i32`
10        // `&val`
11        // ^ We see that if the matching `&`s are dropped, then the `i32`
12        // should be assigned to `val`.
13        &val => println!("Got a value via destructuring: {:?}", val),
14    }
15
16    // To avoid the `&`, you dereference before matching.
17    match *reference {
18        val => println!("Got a value via dereferencing: {:?}", val),
19    }
20
21    // What if you don't start with a reference? `reference` was a `&`
22    // because the right side was already a reference. This is not
23    // a reference because the right side is not one.
24    let _not_a_reference = 3;
25
26    // Rust provides `ref` for exactly this purpose. It modifies the
27    // assignment so that a reference is created for the element; this
28    // reference is assigned.
29    let ref _is_a_reference = 3;
30
31    // Accordingly, by defining 2 values without references, references
32    // can be retrieved via `ref` and `ref mut`.
33    let value = 5;
34    let mut mut_value = 6;
35
36    // Use `ref` keyword to create a reference.
37    match value {
38        ref r => println!("Got a reference to a value: {:?}", r),
39    }
40
41    // Use `ref mut` similarly.
42    match mut_value {
43        ref mut m => {
44            // Got a reference. Gotta dereference it before we can
45            // add anything to it.
46            *m += 10;
47            println!("We added 10. `mut_value`: {:?}", m);
48        },
49    }
50 }
```

See also:

[The ref pattern](#)

structs

Similarly, a `struct` can be destructured as shown:

```
1 fn main() {
2     struct Foo {
3         x: (u32, u32),
4         y: u32,
5     }
6
7     // Try changing the values in the struct to see what happens
8     let foo = Foo { x: (1, 2), y: 3 };
9
10    match foo {
11        Foo { x: (1, b), y } => println!("First of x is 1, b = {}, y = {",
12
13        // you can destructure structs and rename the variables,
14        // the order is not important
15        Foo { y: 2, x: i } => println!("y is 2, i = {:?}", i),
16
17        // and you can also ignore some variables:
18        Foo { y, .. } => println!("y = {}, we don't care about x", y),
19        // this will give an error: pattern does not mention field `x`
20        // Foo { y } => println!("y = {}", y),
21    }
22
23    let faa = Foo { x: (1, 2), y: 3 };
24
25    // You do not need a match block to destructure structs:
26    let Foo { x : x0, y: y0 } = faa;
27    println!("Outside: x0 = {x0:?}, y0 = {y0}");
28 }
```

See also:

[Structs](#)

Guards

A *match guard* can be added to filter the arm.

```
1  #[allow(dead_code)]
2  enum Temperature {
3      Celsius(i32),
4      Fahrenheit(i32),
5  }
6
7  fn main() {
8      let temperature = Temperature::Celsius(35);
9      // ^ TODO try different values for `temperature`
10
11     match temperature {
12         Temperature::Celsius(t) if t > 30 => println!("{}", C is above 30 Ce
13         // The `if condition` part ^ is a guard
14         Temperature::Celsius(t) => println!("{}", C is below 30 Celsius", t)
15
16         Temperature::Fahrenheit(t) if t > 86 => println!("{}", F is above 86
17         Temperature::Fahrenheit(t) => println!("{}", F is below 86 Fahrenhei
18     }
19 }
```

Note that the compiler won't take guard conditions into account when checking if all patterns are covered by the match expression.

```
1  fn main() {
2      let number: u8 = 4;
3
4      match number {
5          i if i == 0 => println!("Zero"),
6          i if i > 0 => println!("Greater than zero"),
7          // _ => unreachable!("Should never happen."),
8          // TODO ^ uncomment to fix compilation
9      }
10 }
```

See also:

[Tuples Enums](#)

Binding

Indirectly accessing a variable makes it impossible to branch and use that variable without re-binding. `match` provides the `@` sigil for binding values to names:

```
1 // A function `age` which returns a `u32`.
2 fn age() -> u32 {
3     15
4 }
5
6 fn main() {
7     println!("Tell me what type of person you are");
8
9     match age() {
10         0 => println!("I haven't celebrated my first birthday
11 // Could `match` 1 ..= 12 directly but then what age
12 // would the child be? Instead, bind to `n` for the
13 // sequence of 1 ..= 12. Now the age can be reported.
14 n @ 1 ..= 12 => println!("I'm a child of age {:?}", n),
15 n @ 13 ..= 19 => println!("I'm a teen of age {:?}", n),
16 // Nothing bound. Return the result.
17 n          => println!("I'm an old person of age {:?}", n),
18     }
19 }
```

You can also use binding to "destructure" enum variants, such as `Option`:

```
1 fn some_number() -> Option<u32> {
2     Some(42)
3 }
4
5 fn main() {
6     match some_number() {
7         // Got `Some` variant, match if its value, bound to `n`,
8         // is equal to 42.
9         Some(n @ 42) => println!("The Answer: {}", n),
10        // Match any other number.
11        Some(n)      => println!("Not interesting... {}", n),
12        // Match anything else (`None` variant).
13        -            => (),
14    }
15 }
```

See also:

[functions](#), [enums](#) and [Option](#)

if let

For some use cases, when matching enums, `match` is awkward. For example:

```
// Make `optional` of type `Option<i32>`
let optional = Some(7);

match optional {
    Some(i) => {
        println!("This is a really long string and `{:?}`", i);
        // ^ Needed 2 indentations just so we could destructure
        // `i` from the option.
    },
    _ => {},
    // ^ Required because `match` is exhaustive. Doesn't it seem
    // like wasted space?
};
```

`if let` is cleaner for this use case and in addition allows various failure options to be specified:

```
1 fn main() {
2     // All have type `Option<i32>`
3     let number = Some(7);
4     let letter: Option<i32> = None;
5     let emoticon: Option<i32> = None;
6
7     // The `if let` construct reads: "if `let` destructures `number` into
8     // `Some(i)`, evaluate the block (`{}`)".
9     if let Some(i) = number {
10         println!("Matched {:?}!", i);
11     }
12
13     // If you need to specify a failure, use an else:
14     if let Some(i) = letter {
15         println!("Matched {:?}!", i);
16     } else {
17         // Destructure failed. Change to the failure case.
18         println!("Didn't match a number. Let's go with a letter!");
19     }
20
21     // Provide an altered failing condition.
22     let i_like_letters = false;
23
24     if let Some(i) = emoticon {
25         println!("Matched {:?}!", i);
26     } // Destructure failed. Evaluate an `else if` condition to see if the
27     // alternate failure branch should be taken:
28     } else if i_like_letters {
29         println!("Didn't match a number. Let's go with a letter!");
30     } else {
31         // The condition evaluated false. This branch is the default:
32         println!("I don't like letters. Let's go with an emoticon :)!");
33     }
34 }
```

In the same way, `if let` can be used to match any enum value:

```
1 // Our example enum
2 enum Foo {
3     Bar,
4     Baz,
5     Qux(u32)
6 }
7
8 fn main() {
9     // Create example variables
10    let a = Foo::Bar;
11    let b = Foo::Baz;
12    let c = Foo::Qux(100);
13
14    // Variable a matches Foo::Bar
15    if let Foo::Bar = a {
16        println!("a is foobar");
17    }
18
19    // Variable b does not match Foo::Bar
20    // So this will print nothing
21    if let Foo::Bar = b {
22        println!("b is foobar");
23    }
24
25    // Variable c matches Foo::Qux which has a value
26    // Similar to Some() in the previous example
27    if let Foo::Qux(value) = c {
28        println!("c is {}", value);
29    }
30
31    // Binding also works with `if let`
32    if let Foo::Qux(value @ 100) = c {
33        println!("c is one hundred");
34    }
35 }
```

Another benefit is that `if let` allows us to match non-parameterized enum variants. This is true even in cases where the enum doesn't implement or derive `PartialEq`. In such cases `if Foo::Bar == a` would fail to compile, because instances of the enum cannot be equated, however `if let` will continue to work.

Would you like a challenge? Fix the following example to use `if let`:

```
1 // This enum purposely neither implements nor derives PartialEq.
2 // That is why comparing Foo::Bar == a fails below.
3 enum Foo {Bar}
4
5 fn main() {
6     let a = Foo::Bar;
7
8     // Variable a matches Foo::Bar
9     if Foo::Bar == a {
10        // ^-- this causes a compile-time error. Use `if let` instead.
11        println!("a is foobar");
12    }
13 }
```

See also:

[enum](#), [Option](#), and the [RFC](#)

let-else

 stable since: rust 1.65

 you can target specific edition by compiling like this `rustc --edition=2021 main.rs`

With `let - else`, a refutable pattern can match and bind variables in the surrounding scope like a normal `let`, or else diverge (e.g. `break`, `return`, `panic!`) when the pattern doesn't match.

```

use std::str::FromStr;

fn get_count_item(s: &str) -> (u64, &str) {
    let mut it = s.split(' ');
    let (Some(count_str), Some(item)) = (it.next(), it.next()) else {
        panic!("Can't segment count item pair: '{s}'");
    };
    let Ok(count) = u64::from_str(count_str) else {
        panic!("Can't parse integer: '{count_str}'");
    };
    (count, item)
}

fn main() {
    assert_eq!(get_count_item("3 chairs"), (3, "chairs"));
}

```

The scope of name bindings is the main thing that makes this different from `match` or `if let - else` expressions. You could previously approximate these patterns with an unfortunate bit of repetition and an outer `let`:

```

let (count_str, item) = match (it.next(), it.next()) {
    (Some(count_str), Some(item)) => (count_str, item),
    _ => panic!("Can't segment count item pair: '{s}'"),
};
let count = if let Ok(count) = u64::from_str(count_str) {
    count
} else {
    panic!("Can't parse integer: '{count_str}'");
};

```

See also:

[option](#), [match](#), [if let](#) and the [let-else RFC](#).

while let

Similar to `if let`, `while let` can make awkward `match` sequences more tolerable. Consider the following sequence that increments `i`:

```
// Make `optional` of type `Option<i32>`
let mut optional = Some(0);

// Repeatedly try this test.
loop {
    match optional {
        // If `optional` deconstructs, evaluate the block.
        Some(i) => {
            if i > 9 {
                println!("Greater than 9, quit!");
                optional = None;
            } else {
                println!("`i` is `{:?}`. Try again.", i);
                optional = Some(i + 1);
            }
            // ^ Requires 3 indentations!
        },
        // Quit the loop when the destructure fails:
        _ => { break; }
        // ^ Why should this be required? There must be a better way!
    }
}
```

Using `while let` makes this sequence much nicer:

```
1 fn main() {
2     // Make `optional` of type `Option<i32>`
3     let mut optional = Some(0);
4
5     // This reads: "while `let` deconstructs `optional` into
6     // `Some(i)`, evaluate the block (`{}`)". Else `break`.
7     while let Some(i) = optional {
8         if i > 9 {
9             println!("Greater than 9, quit!");
10            optional = None;
11        } else {
12            println!("`i` is `{:?}`. Try again.", i);
13            optional = Some(i + 1);
14        }
15        // ^ Less rightward drift and doesn't require
16        // explicitly handling the failing case.
17    }
18    // ^ `if let` had additional optional `else`/`else if`
19    // clauses. `while let` does not have these.
20 }
```


See also:

[enum](#), [Option](#), and the [RFC](#)

Functions

Functions are declared using the `fn` keyword. Its arguments are type annotated, just like variables, and, if the function returns a value, the return type must be specified after an arrow `->`.

The final expression in the function will be used as return value. Alternatively, the `return` statement can be used to return a value earlier from within the function, even from inside loops or `if` statements.

Let's rewrite FizzBuzz using functions!

```
1 // Unlike C/C++, there's no restriction on the order of function definiti
2 fn main() {
3     // We can use this function here, and define it somewhere later
4     fizzbuzz_to(100);
5 }
6
7 // Function that returns a boolean value
8 fn is_divisible_by(lhs: u32, rhs: u32) -> bool {
9     // Corner case, early return
10    if rhs == 0 {
11        return false;
12    }
13
14    // This is an expression, the `return` keyword is not necessary here
15    lhs % rhs == 0
16 }
17
18 // Functions that "don't" return a value, actually return the unit type `
19 fn fizzbuzz(n: u32) -> () {
20     if is_divisible_by(n, 15) {
21         println!("fizzbuzz");
22     } else if is_divisible_by(n, 3) {
23         println!("fizz");
24     } else if is_divisible_by(n, 5) {
25         println!("buzz");
26     } else {
27         println!("{}", n);
28     }
29 }
30
31 // When a function returns `()` , the return type can be omitted from the
32 // signature
33 fn fizzbuzz_to(n: u32) {
34     for n in 1..=n {
35         fizzbuzz(n);
36     }
37 }
```

Associated functions & Methods

Some functions are connected to a particular type. These come in two forms: associated functions, and methods. Associated functions are functions that are defined on a type generally, while methods are associated functions that are called on a particular instance of a type.

```
1 struct Point {
2     x: f64,
3     y: f64,
4 }
5
6 // Implementation block, all `Point` associated functions & methods go i
7 impl Point {
8     // This is an "associated function" because this function is associa
9     // a particular type, that is, Point.
10    //
11    // Associated functions don't need to be called with an instance.
12    // These functions are generally used like constructors.
13    fn origin() -> Point {
14        Point { x: 0.0, y: 0.0 }
15    }
16
17    // Another associated function, taking two arguments:
18    fn new(x: f64, y: f64) -> Point {
19        Point { x: x, y: y }
20    }
21 }
22
23 struct Rectangle {
24     p1: Point,
25     p2: Point,
26 }
27
28 impl Rectangle {
29     // This is a method
30     // `&self` is sugar for `self: &Self`, where `Self` is the type of t
31     // caller object. In this case `Self` = `Rectangle`
32     fn area(&self) -> f64 {
33         // `self` gives access to the struct fields via the dot operator
34         let Point { x: x1, y: y1 } = self.p1;
35         let Point { x: x2, y: y2 } = self.p2;
36
37         // `abs` is a `f64` method that returns the absolute value of th
38         // caller
39         ((x1 - x2) * (y1 - y2)).abs()
40     }
41
42     fn perimeter(&self) -> f64 {
43         let Point { x: x1, y: y1 } = self.p1;
44         let Point { x: x2, y: y2 } = self.p2;
45
46         2.0 * ((x1 - x2).abs() + (y1 - y2).abs())
47     }
48
49     // This method requires the caller object to be mutable
50     // `&mut self` desugars to `self: &mut Self`
51     fn translate(&mut self, x: f64, y: f64) {
52         self.p1.x += x;
53         self.p2.x += x;
54
55         self.p1.y += y;
56         self.p2.y += y;
57     }
58 }
```


Closures

Closures are functions that can capture the enclosing environment. For example, a closure that captures the `x` variable:

```
|val| val + x
```

The syntax and capabilities of closures make them very convenient for on the fly usage. Calling a closure is exactly like calling a function. However, both input and return types *can* be inferred and input variable names *must* be specified.

Other characteristics of closures include:

- using `||` instead of `()` around input variables.
- optional body delimitation `{ }` for a single expression (mandatory otherwise).
- the ability to capture the outer environment variables.

```
1 fn main() {
2     let outer_var = 42;
3
4     // A regular function can't refer to variables in the enclosing enviro
5     //fn function(i: i32) -> i32 { i + outer_var }
6     // TODO: uncomment the line above and see the compiler error. The com
7     // suggests that we define a closure instead.
8
9     // Closures are anonymous, here we are binding them to references
10    // Annotation is identical to function annotation but is optional
11    // as are the `{}` wrapping the body. These nameless functions
12    // are assigned to appropriately named variables.
13    let closure_annotated = |i: i32| -> i32 { i + outer_var };
14    let closure_inferred  = |i      |          i + outer_var ;
15
16    // Call the closures.
17    println!("closure_annotated: {}", closure_annotated(1));
18    println!("closure_inferred: {}", closure_inferred(1));
19    // Once closure's type has been inferred, it cannot be inferred again
20    //println!("cannot reuse closure_inferred with another type: {}", clo
21    // TODO: uncomment the line above and see the compiler error.
22
23    // A closure taking no arguments which returns an `i32`.
24    // The return type is inferred.
25    let one = || 1;
26    println!("closure returning one: {}", one());
27
28 }
```

Capturing

Closures are inherently flexible and will do what the functionality requires to make the closure work without annotation. This allows capturing to flexibly adapt to the use case, sometimes moving and sometimes borrowing. Closures can capture variables:

- by reference: `&T`
- by mutable reference: `&mut T`
- by value: `T`

They preferentially capture variables by reference and only go lower when required.

```
1 fn main() {
2     use std::mem;
3
4     let color = String::from("green");
5
6     // A closure to print `color` which immediately borrows (`&`) `color`
7     // stores the borrow and closure in the `print` variable. It will rem
8     // borrowed until `print` is used the last time.
9     //
10    // `println!` only requires arguments by immutable reference so it do
11    // impose anything more restrictive.
12    let print = || println!("`color`: {}", color);
13
14    // Call the closure using the borrow.
15    print();
16
17    // `color` can be borrowed immutably again, because the closure only
18    // an immutable reference to `color`.
19    let _reborrow = &color;
20    print();
21
22    // A move or reborrow is allowed after the final use of `print`
23    let _color_moved = color;
24
25
26    let mut count = 0;
27    // A closure to increment `count` could take either `&mut count` or `
28    // but `&mut count` is less restrictive so it takes that. Immediately
29    // borrows `count`.
30    //
31    // A `mut` is required on `inc` because a `&mut` is stored inside. Th
32    // calling the closure mutates the closure which requires a `mut`.
33    let mut inc = || {
34        count += 1;
35        println!("`count`: {}", count);
36    };
37
38    // Call the closure using a mutable borrow.
39    inc();
40
41    // The closure still mutably borrows `count` because it is called lat
42    // An attempt to reborrow will lead to an error.
43    // let _reborrow = &count;
44    // ^ TODO: try uncommenting this line.
45    inc();
46
47    // The closure no longer needs to borrow `&mut count`. Therefore, it
48    // possible to reborrow without an error
49    let _count_reborrowed = &mut count;
50
51
52    // A non-copy type.
53    let movable = Box::new(3);
54
55    // `mem::drop` requires `T` so this must take by value. A copy type
56    // would copy into the closure leaving the original untouched.
57    // A non-copy must move and so `movable` immediately moves into
```


Using `move` before vertical pipes forces closure to take ownership of captured variables:

```
1 fn main() {
2     // `Vec` has non-copy semantics.
3     let haystack = vec![1, 2, 3];
4
5     let contains = move |needle| haystack.contains(needle);
6
7     println!("{}", contains(&1));
8     println!("{}", contains(&4));
9
10    // println!("There're {} elements in vec", haystack.len());
11    // ^ Uncommenting above line will result in compile-time error
12    // because borrow checker doesn't allow re-using variable after it
13    // has been moved.
14
15    // Removing `move` from closure's signature will cause closure
16    // to borrow _haystack_ variable immutably, hence _haystack_ is still
17    // available and uncommenting above line will not cause an error.
18 }
```

See also:

[Box](#) and [std::mem::drop](#)

As input parameters

While Rust chooses how to capture variables on the fly mostly without type annotation, this ambiguity is not allowed when writing functions. When taking a closure as an input parameter, the closure's complete type must be annotated using one of a few `traits`, and they're determined by what the closure does with captured value. In order of decreasing restriction, they are:

- `Fn` : the closure uses the captured value by reference (`&T`)
- `FnMut` : the closure uses the captured value by mutable reference (`&mut T`)
- `FnOnce` : the closure uses the captured value by value (`T`)

On a variable-by-variable basis, the compiler will capture variables in the least restrictive manner possible.

For instance, consider a parameter annotated as `FnOnce`. This specifies that the closure *may* capture by `&T`, `&mut T`, or `T`, but the compiler will ultimately choose based on how the captured variables are used in the closure.

This is because if a move is possible, then any type of borrow should also be possible. Note that the reverse is not true. If the parameter is annotated as `Fn`, then capturing variables by `&mut T` or `T` are not allowed. However, `&T` is allowed.

In the following example, try swapping the usage of `Fn`, `FnMut`, and `FnOnce` to see what happens:

```
1 // A function which takes a closure as an argument and calls it.
2 // <F> denotes that F is a "Generic type parameter"
3 fn apply<F>(f: F) where
4     // The closure takes no input and returns nothing.
5     F: FnOnce() {
6     // ^ TODO: Try changing this to `Fn` or `FnMut`.
7
8     f();
9 }
10
11 // A function which takes a closure and returns an `i32`.
12 fn apply_to_3<F>(f: F) -> i32 where
13     // The closure takes an `i32` and returns an `i32`.
14     F: Fn(i32) -> i32 {
15
16     f(3)
17 }
18
19 fn main() {
20     use std::mem;
21
22     let greeting = "hello";
23     // A non-copy type.
24     // `to_owned` creates owned data from borrowed one
25     let mut farewell = "goodbye".to_owned();
26
27     // Capture 2 variables: `greeting` by reference and
28     // `farewell` by value.
29     let diary = || {
30         // `greeting` is by reference: requires `Fn`.
31         println!("I said {}", greeting);
32
33         // Mutation forces `farewell` to be captured by
34         // mutable reference. Now requires `FnMut`.
35         farewell.push_str("!!!");
36         println!("Then I screamed {}", farewell);
37         println!("Now I can sleep. zzzzz");
38
39         // Manually calling drop forces `farewell` to
40         // be captured by value. Now requires `FnOnce`.
41         mem::drop(farewell);
42     };
43
44     // Call the function which applies the closure.
45     apply(diary);
46
47     // `double` satisfies `apply_to_3`'s trait bound
48     let double = |x| 2 * x;
49
50     println!("3 doubled: {}", apply_to_3(double));
51 }
```

See also:

`std::mem::drop` , `Fn` , `FnMut` , `Generics` , `where` and `FnOnce`

Type anonymity

Closures succinctly capture variables from enclosing scopes. Does this have any consequences? It surely does. Observe how using a closure as a function parameter requires [generics](#), which is necessary because of how they are defined:

```
// `F` must be generic.
fn apply<F>(f: F) where
    F: FnOnce() {
    f();
}
```

When a closure is defined, the compiler implicitly creates a new anonymous structure to store the captured variables inside, meanwhile implementing the functionality via one of the `traits: Fn, FnMut, or FnOnce` for this unknown type. This type is assigned to the variable which is stored until calling.

Since this new type is of unknown type, any usage in a function will require generics. However, an unbounded type parameter `<T>` would still be ambiguous and not be allowed. Thus, bounding by one of the `traits: Fn, FnMut, or FnOnce` (which it implements) is sufficient to specify its type.

```
1 // `F` must implement `Fn` for a closure which takes no
2 // inputs and returns nothing - exactly what is required
3 // for `print`.
4 fn apply<F>(f: F) where
5     F: Fn() {
6     f();
7 }
8
9 fn main() {
10     let x = 7;
11
12     // Capture `x` into an anonymous type and implement
13     // `Fn` for it. Store it in `print`.
14     let print = || println!("{}", x);
15
16     apply(print);
17 }
```

See also:

[A thorough analysis](#), [Fn](#), [FnMut](#), and [FnOnce](#)

Input functions

Since closures may be used as arguments, you might wonder if the same can be said about functions. And indeed they can! If you declare a function that takes a closure as parameter, then any function that satisfies the trait bound of that closure can be passed as a parameter.

```
1 // Define a function which takes a generic `F` argument
2 // bounded by `Fn`, and calls it
3 fn call_me<F: Fn()>(f: F) {
4     f();
5 }
6
7 // Define a wrapper function satisfying the `Fn` bound
8 fn function() {
9     println!("I'm a function!");
10 }
11
12 fn main() {
13     // Define a closure satisfying the `Fn` bound
14     let closure = || println!("I'm a closure!");
15
16     call_me(closure);
17     call_me(function);
18 }
```

As an additional note, the `Fn`, `FnMut`, and `FnOnce` traits dictate how a closure captures variables from the enclosing scope.

See also:

[Fn](#), [FnMut](#), and [FnOnce](#)

As output parameters

Closures as input parameters are possible, so returning closures as output parameters should also be possible. However, anonymous closure types are, by definition, unknown, so we have to use `impl Trait` to return them.

The valid traits for returning a closure are:

- `Fn`
- `FnMut`
- `FnOnce`

Beyond this, the `move` keyword must be used, which signals that all captures occur by value. This is required because any captures by reference would be dropped as soon as the function exited, leaving invalid references in the closure.

```
1 fn create_fn() -> impl Fn() {
2     let text = "Fn".to_owned();
3
4     move || println!("This is a: {}", text)
5 }
6
7 fn create_fnmut() -> impl FnMut() {
8     let text = "FnMut".to_owned();
9
10    move || println!("This is a: {}", text)
11 }
12
13 fn create_fnonce() -> impl FnOnce() {
14     let text = "FnOnce".to_owned();
15
16    move || println!("This is a: {}", text)
17 }
18
19 fn main() {
20     let fn_plain = create_fn();
21     let mut fn_mut = create_fnmut();
22     let fn_once = create_fnonce();
23
24     fn_plain();
25     fn_mut();
26     fn_once();
27 }
```

See also:

[Fn](#), [FnMut](#), [Generics](#) and [impl Trait](#).

Examples in `std`

This section contains a few examples of using closures from the `std` library.

Iterator::any

`Iterator::any` is a function which when passed an iterator, will return `true` if any element satisfies the predicate. Otherwise `false`. Its signature:

```

pub trait Iterator {
    // The type being iterated over.
    type Item;

    // `any` takes `&mut self` meaning the caller may be borrowed
    // and modified, but not consumed.
    fn any<F>(&mut self, f: F) -> bool where
        // `FnMut` meaning any captured variable may at most be
        // modified, not consumed. `Self::Item` states it takes
        // arguments to the closure by value.
        F: FnMut(Self::Item) -> bool;
}

1 fn main() {
2     let vec1 = vec![1, 2, 3];
3     let vec2 = vec![4, 5, 6];
4
5     // `iter()` for vecs yields `&i32`. Destructure to `i32`.
6     println!("2 in vec1: {}", vec1.iter().any(|&x| x == 2));
7     // `into_iter()` for vecs yields `i32`. No destructuring required.
8     println!("2 in vec2: {}", vec2.into_iter().any(|x| x == 2));
9
10    // `iter()` only borrows `vec1` and its elements, so they can be used
11    println!("vec1 len: {}", vec1.len());
12    println!("First element of vec1 is: {}", vec1[0]);
13    // `into_iter()` does move `vec2` and its elements, so they cannot be
14    // println!("First element of vec2 is: {}", vec2[0]);
15    // println!("vec2 len: {}", vec2.len());
16    // TODO: uncomment two lines above and see compiler errors.
17
18    let array1 = [1, 2, 3];
19    let array2 = [4, 5, 6];
20
21    // `iter()` for arrays yields `&i32`.
22    println!("2 in array1: {}", array1.iter().any(|&x| x == 2));
23    // `into_iter()` for arrays yields `i32`.
24    println!("2 in array2: {}", array2.into_iter().any(|x| x == 2));
25 }

```

See also:

[std::iter::Iterator::any](#)

Searching through iterators

`Iterator::find` is a function which iterates over an iterator and searches for the first value which satisfies some condition. If none of the values satisfy the condition, it returns `None`. Its signature:

```
pub trait Iterator {
    // The type being iterated over.
    type Item;

    // `find` takes `&mut self` meaning the caller may be borrowed
    // and modified, but not consumed.
    fn find<P>(&mut self, predicate: P) -> Option<Self::Item> where
        // `FnMut` meaning any captured variable may at most be
        // modified, not consumed. `&Self::Item` states it takes
        // arguments to the closure by reference.
        P: FnMut(&Self::Item) -> bool;
}
```

```
1 fn main() {
2     let vec1 = vec![1, 2, 3];
3     let vec2 = vec![4, 5, 6];
4
5     // `iter()` for vecs yields `&i32`.
6     let mut iter = vec1.iter();
7     // `into_iter()` for vecs yields `i32`.
8     let mut into_iter = vec2.into_iter();
9
10    // `iter()` for vecs yields `&i32`, and we want to reference one of i
11    // items, so we have to destructure `&&i32` to `i32`
12    println!("Find 2 in vec1: {:?}", iter.find(|&x| x == 2));
13    // `into_iter()` for vecs yields `i32`, and we want to reference one
14    // its items, so we have to destructure `&i32` to `i32`
15    println!("Find 2 in vec2: {:?}", into_iter.find(|&x| x == 2));
16
17    let array1 = [1, 2, 3];
18    let array2 = [4, 5, 6];
19
20    // `iter()` for arrays yields `&&i32`
21    println!("Find 2 in array1: {:?}", array1.iter().find(|&x| x ==
22    // `into_iter()` for arrays yields `i32`
23    println!("Find 2 in array2: {:?}", array2.into_iter().find(|&x| x ==
24 }
```

`Iterator::find` gives you a reference to the item. But if you want the *index* of the item, use `Iterator::position`.

```
1 fn main() {
2     let vec = vec![1, 9, 3, 3, 13, 2];
3
4     // `iter()` for vecs yields `&i32` and `position()` does not take a r
5     // we have to destructure `&i32` to `i32`
6     let index_of_first_even_number = vec.iter().position(|&x| x % 2 == 0)
7     assert_eq!(index_of_first_even_number, Some(5));
8
9     // `into_iter()` for vecs yields `i32` and `position()` does not take
10    // we do not have to destructure
11    let index_of_first_negative_number = vec.into_iter().position(|x| x <
12    assert_eq!(index_of_first_negative_number, None);
13 }
```

See also:

[std::iter::Iterator::find](#)

[std::iter::Iterator::find_map](#)

[std::iter::Iterator::position](#)

[std::iter::Iterator::rposition](#)

Higher Order Functions

Rust provides Higher Order Functions (HOF). These are functions that take one or more functions and/or produce a more useful function. HOFs and lazy iterators give Rust its functional flavor.

```
1 fn is_odd(n: u32) -> bool {
2     n % 2 == 1
3 }
4
5 fn main() {
6     println!("Find the sum of all the squared odd numbers under 1000");
7     let upper = 1000;
8
9     // Imperative approach
10    // Declare accumulator variable
11    let mut acc = 0;
12    // Iterate: 0, 1, 2, ... to infinity
13    for n in 0.. {
14        // Square the number
15        let n_squared = n * n;
16
17        if n_squared >= upper {
18            // Break loop if exceeded the upper limit
19            break;
20        } else if is_odd(n_squared) {
21            // Accumulate value, if it's odd
22            acc += n_squared;
23        }
24    }
25    println!("imperative style: {}", acc);
26
27    // Functional approach
28    let sum_of_squared_odd_numbers: u32 =
29        (0..).map(|n| n * n) // All natural n
30        .take_while(|&n_squared| n_squared < upper) // Below upper l
31        .filter(|&n_squared| is_odd(n_squared)) // That are odd
32        .sum(); // Sum them
33    println!("functional style: {}", sum_of_squared_odd_numbers);
34 }
```

[Option](#) and [Iterator](#) implement their fair share of HOFs.

Diverging functions

Diverging functions never return. They are marked using `!`, which is an empty type.

```
fn foo() -> ! {  
    panic!("This call never returns.");  
}
```

As opposed to all the other types, this one cannot be instantiated, because the set of all possible values this type can have is empty. Note that, it is different from the `()` type, which has exactly one possible value.

For example, this function returns as usual, although there is no information in the return value.

```
fn some_fn() {  
    ()  
}  
  
fn main() {  
    let _a: () = some_fn();  
    println!("This function returns and you can see this line.");  
}
```

As opposed to this function, which will never return the control back to the caller.

```
#![feature(never_type)]  
  
fn main() {  
    let x: ! = panic!("This call never returns.");  
    println!("You will never see this line!");  
}
```

Although this might seem like an abstract concept, it is in fact very useful and often handy. The main advantage of this type is that it can be cast to any other one and therefore used at places where an exact type is required, for instance in `match` branches. This allows us to write code like this:

```

fn main() {
    fn sum_odd_numbers(up_to: u32) -> u32 {
        let mut acc = 0;
        for i in 0..up_to {
            // Notice that the return type of this match expression must be
u32
            // because of the type of the "addition" variable.
            let addition: u32 = match i%2 == 1 {
                // The "i" variable is of type u32, which is perfectly fine.
                true => i,
                // On the other hand, the "continue" expression does not
return
                // u32, but it is still fine, because it never returns and
therefore
                // does not violate the type requirements of the match
expression.
                false => continue,
            };
            acc += addition;
        }
        acc
    }
    println!("Sum of odd numbers up to 9 (excluding): {}",
sum_odd_numbers(9));
}

```

It is also the return type of functions that loop forever (e.g. `loop {}`) like network servers or functions that terminate the process (e.g. `exit()`).

Modules

Rust provides a powerful module system that can be used to hierarchically split code in logical units (modules), and manage visibility (public/private) between them.

A module is a collection of items: functions, structs, traits, `impl` blocks, and even other modules.

Visibility

By default, the items in a module have private visibility, but this can be overridden with the `pub` modifier. Only the public items of a module can be accessed from outside the module scope.


```
1  // A module named `my_mod`
2  mod my_mod {
3      // Items in modules default to private visibility.
4      fn private_function() {
5          println!("called `my_mod::private_function()`");
6      }
7
8      // Use the `pub` modifier to override default visibility.
9      pub fn function() {
10         println!("called `my_mod::function()`");
11     }
12
13     // Items can access other items in the same module,
14     // even when private.
15     pub fn indirect_access() {
16         print!("called `my_mod::indirect_access()`, that\n> ");
17         private_function();
18     }
19
20     // Modules can also be nested
21     pub mod nested {
22         pub fn function() {
23             println!("called `my_mod::nested::function()`");
24         }
25
26         #[allow(dead_code)]
27         fn private_function() {
28             println!("called `my_mod::nested::private_function()`");
29         }
30
31         // Functions declared using `pub(in path)` syntax are only visible
32         // within the given path. `path` must be a parent or ancestor module
33         pub(in crate::my_mod) fn public_function_in_my_mod() {
34             print!("called `my_mod::nested::public_function_in_my_mod()`, that\n> ");
35             public_function_in_nested();
36         }
37
38         // Functions declared using `pub(self)` syntax are only visible within
39         // the current module, which is the same as leaving them private
40         pub(self) fn public_function_in_nested() {
41             println!("called `my_mod::nested::public_function_in_nested()`, that\n> ");
42         }
43
44         // Functions declared using `pub(super)` syntax are only visible within
45         // the parent module
46         pub(super) fn public_function_in_super_mod() {
47             println!("called `my_mod::nested::public_function_in_super_mod()`, that\n> ");
48         }
49     }
50
51     pub fn call_public_function_in_my_mod() {
52         print!("called `my_mod::call_public_function_in_my_mod()`, that\n> ");
53         nested::public_function_in_my_mod();
54         print!("\n> ");
55         nested::public_function_in_super_mod();
56     }
57 }
```


Struct visibility

Structs have an extra level of visibility with their fields. The visibility defaults to private, and can be overridden with the `pub` modifier. This visibility only matters when a struct is accessed from outside the module where it is defined, and has the goal of hiding information (encapsulation).

```
1 mod my {
2     // A public struct with a public field of generic type `T`
3     pub struct OpenBox<T> {
4         pub contents: T,
5     }
6
7     // A public struct with a private field of generic type `T`
8     pub struct ClosedBox<T> {
9         contents: T,
10    }
11
12    impl<T> ClosedBox<T> {
13        // A public constructor method
14        pub fn new(contents: T) -> ClosedBox<T> {
15            ClosedBox {
16                contents: contents,
17            }
18        }
19    }
20 }
21
22 fn main() {
23     // Public structs with public fields can be constructed as usual
24     let open_box = my::OpenBox { contents: "public information" };
25
26     // and their fields can be normally accessed.
27     println!("The open box contains: {}", open_box.contents);
28
29     // Public structs with private fields cannot be constructed using file
30     // Error! `ClosedBox` has private fields
31     //let closed_box = my::ClosedBox { contents: "classified information"
32     // TODO ^ Try uncommenting this line
33
34     // However, structs with private fields can be created using
35     // public constructors
36     let _closed_box = my::ClosedBox::new("classified information");
37
38     // and the private fields of a public struct cannot be accessed.
39     // Error! The `contents` field is private
40     //println!("The closed box contains: {}", _closed_box.contents);
41     // TODO ^ Try uncommenting this line
42 }
```

See also:

[generics](#) and [methods](#)

The use declaration

The `use` declaration can be used to bind a full path to a new name, for easier access. It is often used like this:

```
1 use crate::deeply::nested::{
2     my_first_function,
3     my_second_function,
4     AndATraitType
5 };
6
7 fn main() {
8     my_first_function();
9 }
```

You can use the `as` keyword to bind imports to a different name:

```
1 // Bind the `deeply::nested::function` path to `other_function`.
2 use deeply::nested::function as other_function;
3
4 fn function() {
5     println!("called `function()`");
6 }
7
8 mod deeply {
9     pub mod nested {
10         pub fn function() {
11             println!("called `deeply::nested::function()`");
12         }
13     }
14 }
15
16 fn main() {
17     // Easier access to `deeply::nested::function`
18     other_function();
19
20     println!("Entering block");
21     {
22         // This is equivalent to `use deeply::nested::function as function`
23         // This `function()` will shadow the outer one.
24         use crate::deeply::nested::function;
25
26         // `use` bindings have a local scope. In this case, the
27         // shadowing of `function()` is only in this block.
28         function();
29
30         println!("Leaving block");
31     }
32
33     function();
34 }
```

super and self

The `super` and `self` keywords can be used in the path to remove ambiguity when accessing items and to prevent unnecessary hardcoding of paths.

```
1 fn function() {
2     println!("called `function()`");
3 }
4
5 mod cool {
6     pub fn function() {
7         println!("called `cool::function()`");
8     }
9 }
10
11 mod my {
12     fn function() {
13         println!("called `my::function()`");
14     }
15
16     mod cool {
17         pub fn function() {
18             println!("called `my::cool::function()`");
19         }
20     }
21
22     pub fn indirect_call() {
23         // Let's access all the functions named `function` from this scope
24         print!("called `my::indirect_call()`, that\n> ");
25
26         // The `self` keyword refers to the current module scope - in this case, the `my` scope
27         // Calling `self::function()` and calling `function()` directly both result in the same
28         // the same result, because they refer to the same function.
29         self::function();
30         function();
31
32         // We can also use `self` to access another module inside `my`:
33         self::cool::function();
34
35         // The `super` keyword refers to the parent scope (outside the `my` module)
36         super::function();
37
38         // This will bind to the `cool::function` in the *crate* scope.
39         // In this case the crate scope is the outermost scope.
40         {
41             use crate::cool::function as root_function;
42             root_function();
43         }
44     }
45 }
46
47 fn main() {
48     my::indirect_call();
49 }
```

File hierarchy

Modules can be mapped to a file/directory hierarchy. Let's break down the [visibility example](#) in files:

```
$ tree .
```

```
.
├── my
│   ├── inaccessible.rs
│   └── nested.rs
├── my.rs
└── split.rs
```

In `split.rs`:

```
// This declaration will look for a file named `my.rs` and will
// insert its contents inside a module named `my` under this scope
mod my;

fn function() {
    println!("called `function()`");
}

fn main() {
    my::function();

    function();

    my::indirect_access();

    my::nested::function();
}
```

In `my.rs`:


```
// Similarly `mod inaccessible` and `mod nested` will locate the `nested.rs`  
// and `inaccessible.rs` files and insert them here under their respective  
// modules  
mod inaccessible;  
pub mod nested;  
  
pub fn function() {  
    println!("called `my::function()`");  
}  
  
fn private_function() {  
    println!("called `my::private_function()`");  
}  
  
pub fn indirect_access() {  
    print!("called `my::indirect_access()`, that\n> ");  
  
    private_function();  
}
```

In my/nested.rs:

```
pub fn function() {  
    println!("called `my::nested::function()`");  
}  
  
#[allow(dead_code)]  
fn private_function() {  
    println!("called `my::nested::private_function()`");  
}
```

In my/inaccessible.rs:

```
#[allow(dead_code)]  
pub fn public_function() {  
    println!("called `my::inaccessible::public_function()`");  
}
```

Let's check that things still work as before:

```
$ rustc split.rs && ./split  
called `my::function()`  
called `function()`  
called `my::indirect_access()`, that  
> called `my::private_function()`  
called `my::nested::function()`
```

Crates

A crate is a compilation unit in Rust. Whenever `rustc some_file.rs` is called, `some_file.rs` is treated as the *crate file*. If `some_file.rs` has `mod` declarations in it, then the contents of the module files would be inserted in places where `mod` declarations in the crate file are found, *before* running the compiler over it. In other words, modules do *not* get compiled individually, only crates get compiled.

A crate can be compiled into a binary or into a library. By default, `rustc` will produce a binary from a crate. This behavior can be overridden by passing the `--crate-type` flag to `lib`.

Creating a Library

Let's create a library, and then see how to link it to another crate.

In `rary.rs`:

```
pub fn public_function() {
    println!("called rary's `public_function()`");
}

fn private_function() {
    println!("called rary's `private_function()`");
}

pub fn indirect_access() {
    print!("called rary's `indirect_access()`, that\n> ");

    private_function();
}

$ rustc --crate-type=lib rary.rs
$ ls lib*
library.rlib
```

Libraries get prefixed with "lib", and by default they get named after their crate file, but this default name can be overridden by passing the `--crate-name` option to `rustc` or by using the [crate_name attribute](#).

Using a Library

To link a crate to this new library you may use `rustc`'s `--extern` flag. All of its items will then be imported under a module named the same as the library. This module generally behaves the same way as any other module.

```
// extern crate rary; // May be required for Rust 2015 edition or earlier
```

```
fn main() {  
    rary::public_function();  
  
    // Error! `private_function` is private  
    //rary::private_function();  
  
    rary::indirect_access();  
}
```

```
# Where library.rlib is the path to the compiled library, assumed that it's  
# in the same directory here:  
$ rustc executable.rs --extern rary=library.rlib && ./executable  
called rary's `public_function()`  
called rary's `indirect_access()`, that  
> called rary's `private_function()`
```

Cargo

`cargo` is the official Rust package management tool. It has lots of really useful features to improve code quality and developer velocity! These include

- Dependency management and integration with crates.io (the official Rust package registry)
- Awareness of unit tests
- Awareness of benchmarks

This chapter will go through some quick basics, but you can find the comprehensive docs in [The Cargo Book](#).

Dependencies

Most programs have dependencies on some libraries. If you have ever managed dependencies by hand, you know how much of a pain this can be. Luckily, the Rust ecosystem comes standard with `cargo`! `cargo` can manage dependencies for a project.

To create a new Rust project,

```
# A binary
cargo new foo

# A library
cargo new --lib bar
```

For the rest of this chapter, let's assume we are making a binary, rather than a library, but all of the concepts are the same.

After the above commands, you should see a file hierarchy like this:

```
.
├── bar
│   ├── Cargo.toml
│   └── src
│       └── lib.rs
└── foo
    ├── Cargo.toml
    └── src
        └── main.rs
```

The `main.rs` is the root source file for your new `foo` project -- nothing new there. The `Cargo.toml` is the config file for `cargo` for this project. If you look inside it, you should see something like this:

```
[package]
name = "foo"
version = "0.1.0"
authors = ["mark"]

[dependencies]
```

The `name` field under `[package]` determines the name of the project. This is used by `crates.io` if you publish the crate (more later). It is also the name of the output binary when you compile.

The `version` field is a crate version number using [Semantic Versioning](#).

The `authors` field is a list of authors used when publishing the crate.

The `[dependencies]` section lets you add dependencies for your project.

For example, suppose that we want our program to have a great CLI. You can find lots of great packages on crates.io (the official Rust package registry). One popular choice is [clap](#). As of this writing, the most recent published version of `clap` is `2.27.1`. To add a dependency to our program, we can simply add the following to our `Cargo.toml` under `[dependencies]`: `clap = "2.27.1"`. And that's it! You can start using `clap` in your program.

`cargo` also supports [other types of dependencies](#). Here is just a small sampling:

```
[package]
name = "foo"
version = "0.1.0"
authors = ["mark"]

[dependencies]
clap = "2.27.1" # from crates.io
rand = { git = "https://github.com/rust-lang-nursery/rand" } # from online
repo
bar = { path = "../bar" } # from a path in the local filesystem
```

`cargo` is more than a dependency manager. All of the available configuration options are listed in the [format specification](#) of `Cargo.toml`.

To build our project we can execute `cargo build` anywhere in the project directory (including subdirectories!). We can also do `cargo run` to build and run. Notice that these commands will resolve all dependencies, download crates if needed, and build everything, including your crate. (Note that it only rebuilds what it has not already built, similar to `make`).

Voila! That's all there is to it!

Conventions

In the previous chapter, we saw the following directory hierarchy:

```
foo
├── Cargo.toml
└── src
    └── main.rs
```

Suppose that we wanted to have two binaries in the same project, though. What then?

It turns out that `cargo` supports this. The default binary name is `main`, as we saw before, but you can add additional binaries by placing them in a `bin/` directory:

```
foo
├── Cargo.toml
└── src
    ├── main.rs
    └── bin
        └── my_other_bin.rs
```

To tell `cargo` to only compile or run this binary, we just pass `cargo` the `--bin my_other_bin` flag, where `my_other_bin` is the name of the binary we want to work with.

In addition to extra binaries, `cargo` supports [more features](#) such as benchmarks, tests, and examples.

In the next chapter, we will look more closely at tests.

Testing

As we know testing is integral to any piece of software! Rust has first-class support for unit and integration testing ([see this chapter](#) in TRPL).

From the testing chapters linked above, we see how to write unit tests and integration tests. Organizationally, we can place unit tests in the modules they test and integration tests in their own `tests/` directory:

```
foo
├── Cargo.toml
├── src
│   ├── main.rs
│   └── lib.rs
└── tests
    ├── my_test.rs
    └── my_other_test.rs
```

Each file in `tests` is a separate [integration test](#), i.e. a test that is meant to test your library as if it were being called from a dependent crate.

The [Testing](#) chapter elaborates on the three different testing styles: [Unit](#), [Doc](#), and [Integration](#).

`cargo` naturally provides an easy way to run all of your tests!

```
$ cargo test
```

You should see output like this:

```
$ cargo test
Compiling blah v0.1.0 (file:///nobackup/blah)
Finished dev [unoptimized + debuginfo] target(s) in 0.89 secs
Running target/debug/deps/blah-d3b32b97275ec472

running 3 tests
test test_bar ... ok
test test_baz ... ok
test test_foo_bar ... ok
test test_foo ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

You can also run tests whose name matches a pattern:

```
$ cargo test test_foo
```

```
$ cargo test test_foo
  Compiling blah v0.1.0 (file:///nobackup/blah)
  Finished dev [unoptimized + debuginfo] target(s) in 0.35 secs
  Running target/debug/deps/blah-d3b32b97275ec472

running 2 tests
test test_foo ... ok
test test_foo_bar ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 2 filtered out
```

One word of caution: Cargo may run multiple tests concurrently, so make sure that they don't race with each other.

One example of this concurrency causing issues is if two tests output to a file, such as below:

```
#[cfg(test)]
mod tests {
    // Import the necessary modules
    use std::fs::OpenOptions;
    use std::io::Write;

    // This test writes to a file
    #[test]
    fn test_file() {
        // Opens the file ferris.txt or creates one if it doesn't exist.
        let mut file = OpenOptions::new()
            .append(true)
            .create(true)
            .open("ferris.txt")
            .expect("Failed to open ferris.txt");

        // Print "Ferris" 5 times.
        for _ in 0..5 {
            file.write_all("Ferris\n".as_bytes())
                .expect("Could not write to ferris.txt");
        }
    }

    // This test tries to write to the same file
    #[test]
    fn test_file_also() {
        // Opens the file ferris.txt or creates one if it doesn't exist.
        let mut file = OpenOptions::new()
            .append(true)
            .create(true)
            .open("ferris.txt")
            .expect("Failed to open ferris.txt");

        // Print "Corro" 5 times.
        for _ in 0..5 {
            file.write_all("Corro\n".as_bytes())
                .expect("Could not write to ferris.txt");
        }
    }
}
```

Although the intent is to get the following:

```
$ cat ferris.txt
Ferris
Ferris
Ferris
Ferris
Ferris
Corro
Corro
Corro
Corro
Corro
```

What actually gets put into `ferris.txt` is this:

```
$ cargo test test_foo
Corro
Ferris
Corro
Ferris
Corro
Ferris
Corro
Ferris
Corro
Ferris
```

Build Scripts

Sometimes a normal build from `cargo` is not enough. Perhaps your crate needs some pre-requisites before `cargo` will successfully compile, things like code generation, or some native code that needs to be compiled. To solve this problem we have build scripts that Cargo can run.

To add a build script to your package it can either be specified in the `Cargo.toml` as follows:

```
[package]
...
build = "build.rs"
```

Otherwise Cargo will look for a `build.rs` file in the project directory by default.

How to use a build script

The build script is simply another Rust file that will be compiled and invoked prior to compiling anything else in the package. Hence it can be used to fulfill pre-requisites of your crate.

Cargo provides the script with inputs via environment variables [specified here](#) that can be used.

The script provides output via stdout. All lines printed are written to `target/debug/build/<pkg>/output`. Further, lines prefixed with `cargo:` will be interpreted by Cargo directly and hence can be used to define parameters for the package's compilation.

For further specification and examples have a read of the [Cargo specification](#).

Attributes

An attribute is metadata applied to some module, crate or item. This metadata can be used to/for:

- [conditional compilation of code](#)
- [set crate name, version and type \(binary or library\)](#)
- disable [lints](#) (warnings)
- enable compiler features (macros, glob imports, etc.)
- link to a foreign library
- mark functions as unit tests
- mark functions that will be part of a benchmark
- [attribute like macros](#)

When attributes apply to a whole crate, their syntax is `#![crate_attribute]`, and when they apply to a module or item, the syntax is `#[item_attribute]` (notice the missing bang `!`).

Attributes can take arguments with different syntaxes:

- `#[attribute = "value"]`
- `#[attribute(key = "value")]`
- `#[attribute(value)]`

Attributes can have multiple values and can be separated over multiple lines, too:

```
#[attribute(value, value2)]
```

```
#[attribute(value, value2, value3,  
            value4, value5)]
```

dead_code

The compiler provides a `dead_code` [lint](#) that will warn about unused functions. An *attribute* can be used to disable the lint.

```
1 fn used_function() {}
2
3 // `#[allow(dead_code)]` is an attribute that disables the `dead_code` li
4 #[allow(dead_code)]
5 fn unused_function() {}
6
7 fn noisy_unused_function() {}
8 // FIXME ^ Add an attribute to suppress the warning
9
10 fn main() {
11     used_function();
12 }
```

Note that in real programs, you should eliminate dead code. In these examples we'll allow dead code in some places because of the interactive nature of the examples.

Crates

The `crate_type` attribute can be used to tell the compiler whether a crate is a binary or a library (and even which type of library), and the `crate_name` attribute can be used to set the name of the crate.

However, it is important to note that both the `crate_type` and `crate_name` attributes have **no** effect whatsoever when using Cargo, the Rust package manager. Since Cargo is used for the majority of Rust projects, this means real-world uses of `crate_type` and `crate_name` are relatively limited.

```
1 // This crate is a library
2 #![crate_type = "lib"]
3 // The library is named "rary"
4 #![crate_name = "rary"]
5
6 pub fn public_function() {
7     println!("called rary's `public_function()`");
8 }
9
10 fn private_function() {
11     println!("called rary's `private_function()`");
12 }
13
14 pub fn indirect_access() {
15     print!("called rary's `indirect_access()`, that\n> ");
16
17     private_function();
18 }
```

When the `crate_type` attribute is used, we no longer need to pass the `--crate-type` flag to `rustc`.

```
$ rustc lib.rs
$ ls lib*
library.rlib
```


cfg

Configuration conditional checks are possible through two different operators:

- the `cfg` attribute: `#[cfg(...)]` in attribute position
- the `cfg!` macro: `cfg!(...)` in boolean expressions

While the former enables conditional compilation, the latter conditionally evaluates to `true` or `false` literals allowing for checks at run-time. Both utilize identical argument syntax.

`cfg!`, unlike `#[cfg]`, does not remove any code and only evaluates to `true` or `false`. For example, all blocks in an `if/else` expression need to be valid when `cfg!` is used for the condition, regardless of what `cfg!` is evaluating.

```
1 // This function only gets compiled if the target OS is linux
2 #[cfg(target_os = "linux")]
3 fn are_you_on_linux() {
4     println!("You are running linux!");
5 }
6
7 // And this function only gets compiled if the target OS is *not* linux
8 #[cfg(not(target_os = "linux"))]
9 fn are_you_on_linux() {
10     println!("You are *not* running linux!");
11 }
12
13 fn main() {
14     are_you_on_linux();
15
16     println!("Are you sure?");
17     if cfg!(target_os = "linux") {
18         println!("Yes. It's definitely linux!");
19     } else {
20         println!("Yes. It's definitely *not* linux!");
21     }
22 }
```

See also:

[the reference](#), `cfg!`, and [macros](#).

Custom

Some conditionals like `target_os` are implicitly provided by `rustc`, but custom conditionals must be passed to `rustc` using the `--cfg` flag.

```
1 #[cfg(some_condition)]
2 fn conditional_function() {
3     println!("condition met!");
4 }
5
6 fn main() {
7     conditional_function();
8 }
```

Try to run this to see what happens without the custom `cfg` flag.

With the custom `cfg` flag:

```
$ rustc --cfg some_condition custom.rs && ./custom
condition met!
```

Generics

Generics is the topic of generalizing types and functionalities to broader cases. This is extremely useful for reducing code duplication in many ways, but can call for rather involved syntax. Namely, being generic requires taking great care to specify over which types a generic type is actually considered valid. The simplest and most common use of generics is for type parameters.

A type parameter is specified as generic by the use of angle brackets and upper [camel case](#): `<Aaa, Bbb, ...>`. "Generic type parameters" are typically represented as `<T>`. In Rust, "generic" also describes anything that accepts one or more generic type parameters `<T>`. Any type specified as a generic type parameter is generic, and everything else is concrete (non-generic).

For example, defining a *generic function* named `foo` that takes an argument `T` of any type:

```
fn foo<T>(arg: T) { ... }
```

Because `T` has been specified as a generic type parameter using `<T>`, it is considered generic when used here as `(arg: T)`. This is the case even if `T` has previously been defined as a `struct`.

This example shows some of the syntax in action:

```
1 // A concrete type `A`.
2 struct A;
3
4 // In defining the type `Single`, the first use of `A` is not preceded by
5 // Therefore, `Single` is a concrete type, and `A` is defined as above.
6 struct Single(A);
7 //           ^ Here is `Single`'s first use of the type `A`.
8
9 // Here, `` precedes the first use of `T`, so `SingleGen` is a generic
10 // Because the type parameter `T` is generic, it could be anything, inclu
11 // the concrete type `A` defined at the top.
12 struct SingleGen<T>(T);
13
14 fn main() {
15     // `Single` is concrete and explicitly takes `A`.
16     let _s = Single(A);
17
18     // Create a variable `_char` of type `SingleGen<char>`
19     // and give it the value `SingleGen('a')`.
20     // Here, `SingleGen` has a type parameter explicitly specified.
21     let _char: SingleGen<char> = SingleGen('a');
22
23     // `SingleGen` can also have a type parameter implicitly specified:
24     let _t    = SingleGen(A); // Uses `A` defined at the top.
25     let _i32  = SingleGen(6); // Uses `i32`.
26     let _char = SingleGen('a'); // Uses `char`.
27 }
```

See also:

[structs](#)

Functions

The same set of rules can be applied to functions: a type `T` becomes generic when preceded by `<T>`.

Using generic functions sometimes requires explicitly specifying type parameters. This may be the case if the function is called where the return type is generic, or if the compiler doesn't have enough information to infer the necessary type parameters.

A function call with explicitly specified type parameters looks like: `fun::<A, B, ...>()`.

```

1  struct A;           // Concrete type `A`.
2  struct S(A);        // Concrete type `S`.
3  struct SGen<T>(T);  // Generic type `SGen`.
4
5  // The following functions all take ownership of the variable passed into
6  // them and immediately go out of scope, freeing the variable.
7
8  // Define a function `reg_fn` that takes an argument `_s` of type `S`.
9  // This has no `` so this is not a generic function.
10 fn reg_fn(_s: S) {}
11
12 // Define a function `gen_spec_t` that takes an argument `_s` of type `SG
13 // It has been explicitly given the type parameter `A`, but because `A` h
14 // been specified as a generic type parameter for `gen_spec_t`, it is not
15 fn gen_spec_t(_s: SGen<A>) {}
16
17 // Define a function `gen_spec_i32` that takes an argument `_s` of type `
18 // It has been explicitly given the type parameter `i32`, which is a spec
19 // Because `i32` is not a generic type, this function is also not generic
20 fn gen_spec_i32(_s: SGen<i32>) {}
21
22 // Define a function `generic` that takes an argument `_s` of type `SGen<
23 // Because `SGen<T>` is preceded by `

```

See also:

functions and structs

Implementation

Similar to functions, implementations require care to remain generic.

```
struct S; // Concrete type `S`
struct GenericVal<T>(T); // Generic type `GenericVal`

// impl of GenericVal where we explicitly specify type parameters:
impl GenericVal<f32> {} // Specify `f32`
impl GenericVal<S> {} // Specify `S` as defined above

// `` Must precede the type to remain generic
impl<T> GenericVal<T> {}

1 struct Val {
2     val: f64,
3 }
4
5 struct GenVal<T> {
6     gen_val: T,
7 }
8
9 // impl of Val
10 impl Val {
11     fn value(&self) -> &f64 {
12         &self.val
13     }
14 }
15
16 // impl of GenVal for a generic type `T`
17 impl<T> GenVal<T> {
18     fn value(&self) -> &T {
19         &self.gen_val
20     }
21 }
22
23 fn main() {
24     let x = Val { val: 3.0 };
25     let y = GenVal { gen_val: 3i32 };
26
27     println!("{}", x.value(), y.value());
28 }
```

See also:

[functions returning references](#), [impl](#), and [struct](#)

Traits

Of course `traits` can also be generic. Here we define one which reimplements the `Drop` trait as a generic method to `drop` itself and an input.

```
1 // Non-copyable types.
2 struct Empty;
3 struct Null;
4
5 // A trait generic over `T`.
6 trait DoubleDrop<T> {
7     // Define a method on the caller type which takes an
8     // additional single parameter `T` and does nothing with it.
9     fn double_drop(self, _: T);
10 }
11
12 // Implement `DoubleDrop<T>` for any generic parameter `T` and
13 // caller `U`.
14 impl<T, U> DoubleDrop<T> for U {
15     // This method takes ownership of both passed arguments,
16     // deallocating both.
17     fn double_drop(self, _: T) {}
18 }
19
20 fn main() {
21     let empty = Empty;
22     let null = Null;
23
24     // Deallocate `empty` and `null`.
25     empty.double_drop(null);
26
27     //empty;
28     //null;
29     // ^ TODO: Try uncommenting these lines.
30 }
```

See also:

[Drop](#), [struct](#), and [trait](#)

Bounds

When working with generics, the type parameters often must use traits as *bounds* to stipulate what functionality a type implements. For example, the following example uses the trait `Display` to print and so it requires `T` to be bound by `Display`; that is, `T` *must* implement `Display`.

```
// Define a function `printer` that takes a generic type `T` which
// must implement trait `Display`.
fn printer<T: Display>(t: T) {
    println!("{}", t);
}
```

Bounding restricts the generic to types that conform to the bounds. That is:

```
struct S<T: Display>(T);

// Error! `Vec<T>` does not implement `Display`. This
// specialization will fail.
let s = S(vec![1]);
```

Another effect of bounding is that generic instances are allowed to access the [methods](#) of traits specified in the bounds. For example:

```
1 // A trait which implements the print marker: `{:?}`.
2 use std::fmt::Debug;
3
4 trait HasArea {
5     fn area(&self) -> f64;
6 }
7
8 impl HasArea for Rectangle {
9     fn area(&self) -> f64 { self.length * self.height }
10 }
11
12 #[derive(Debug)]
13 struct Rectangle { length: f64, height: f64 }
14 #[allow(dead_code)]
15 struct Triangle { length: f64, height: f64 }
16
17 // The generic `T` must implement `Debug`. Regardless
18 // of the type, this will work properly.
19 fn print_debug<T: Debug>(t: &T) {
20     println!("{:?}", t);
21 }
22
23 // `T` must implement `HasArea`. Any type which meets
24 // the bound can access `HasArea`'s function `area`.
25 fn area<T: HasArea>(t: &T) -> f64 { t.area() }
26
27 fn main() {
28     let rectangle = Rectangle { length: 3.0, height: 4.0 };
29     let _triangle = Triangle { length: 3.0, height: 4.0 };
30
31     print_debug(&rectangle);
32     println!("Area: {}", area(&rectangle));
33
34     //print_debug(&_triangle);
35     //println!("Area: {}", area(&_triangle));
36     // ^ TODO: Try uncommenting these.
37     // | Error: Does not implement either `Debug` or `HasArea`.
38 }
```

As an additional note, [where](#) clauses can also be used to apply bounds in some cases to be more expressive.

See also:

[std::fmt](#), [struct S](#), and [trait S](#)

Testcase: empty bounds

A consequence of how bounds work is that even if a `trait` doesn't include any functionality, you can still use it as a bound. `Eq` and `Copy` are examples of such `traits` from the `std` library.

```
1 struct Cardinal;
2 struct BlueJay;
3 struct Turkey;
4
5 trait Red {}
6 trait Blue {}
7
8 impl Red for Cardinal {}
9 impl Blue for BlueJay {}
10
11 // These functions are only valid for types which implement these
12 // traits. The fact that the traits are empty is irrelevant.
13 fn red<T: Red>(_: &T) -> &'static str { "red" }
14 fn blue<T: Blue>(_: &T) -> &'static str { "blue" }
15
16 fn main() {
17     let cardinal = Cardinal;
18     let blue_jay = BlueJay;
19     let _turkey = Turkey;
20
21     // `red()` won't work on a blue jay nor vice versa
22     // because of the bounds.
23     println!("A cardinal is {}", red(&cardinal));
24     println!("A blue jay is {}", blue(&blue_jay));
25     //println!("A turkey is {}", red(&_turkey));
26     // ^ TODO: Try uncommenting this line.
27 }
```

See also:

[std::cmp::Eq](#), [std::marker::Copy](#), and [traits](#)

Multiple bounds

Multiple bounds for a single type can be applied with a `+`. Like normal, different types are separated with `,`.

```
1 use std::fmt::{Debug, Display};
2
3 fn compare_prints<T: Debug + Display>(t: &T) {
4     println!("Debug: `{:?}`", t);
5     println!("Display: `{}`", t);
6 }
7
8 fn compare_types<T: Debug, U: Display>(t: &T, u: &U) {
9     println!("t: `{:?}`", t);
10    println!("u: `{}`", u);
11 }
12
13 fn main() {
14     let string = "words";
15     let array = [1, 2, 3];
16     let vec = vec![1, 2, 3];
17
18     compare_prints(&string);
19     //compare_prints(&array);
20     // TODO ^ Try uncommenting this.
21
22     compare_types(&array, &vec);
23 }
```

See also:

[std::fmt](#) and [traits](#)

Where clauses

A bound can also be expressed using a `where` clause immediately before the opening `{`, rather than at the type's first mention. Additionally, `where` clauses can apply bounds to arbitrary types, rather than just to type parameters.

Some cases that a `where` clause is useful:

- When specifying generic types and bounds separately is clearer:

```
impl <A: TraitB + TraitC, D: TraitE + TraitF> MyTrait<A, D> for YourType {}
```

```
// Expressing bounds with a `where` clause
impl <A, D> MyTrait<A, D> for YourType where
    A: TraitB + TraitC,
    D: TraitE + TraitF {}
```

- When using a `where` clause is more expressive than using normal syntax. The `impl` in this example cannot be directly expressed without a `where` clause:

```
1 use std::fmt::Debug;
2
3 trait PrintInOption {
4     fn print_in_option(self);
5 }
6
7 // Because we would otherwise have to express this as `T: Debug` or
8 // use another method of indirect approach, this requires a `where` clause
9 impl<T> PrintInOption for T where
10     Option<T>: Debug {
11     // We want `Option<T>: Debug` as our bound because that is what's
12     // being printed. Doing otherwise would be using the wrong bound.
13     fn print_in_option(self) {
14         println!("{:?}", Some(self));
15     }
16 }
17
18 fn main() {
19     let vec = vec![1, 2, 3];
20
21     vec.print_in_option();
22 }
```

See also:

[RFC](#), [struct](#), and [trait](#)

New Type Idiom

The `newtype` idiom gives compile time guarantees that the right type of value is supplied to a program.

For example, an age verification function that checks age in years, *must* be given a value of type `Years`.

```
1 struct Years(i64);
2
3 struct Days(i64);
4
5 impl Years {
6     pub fn to_days(&self) -> Days {
7         Days(self.0 * 365)
8     }
9 }
10
11
12 impl Days {
13     /// truncates partial years
14     pub fn to_years(&self) -> Years {
15         Years(self.0 / 365)
16     }
17 }
18
19 fn old_enough(age: &Years) -> bool {
20     age.0 >= 18
21 }
22
23 fn main() {
24     let age = Years(5);
25     let age_days = age.to_days();
26     println!("Old enough {}", old_enough(&age));
27     println!("Old enough {}", old_enough(&age_days.to_years()));
28     // println!("Old enough {}", old_enough(&age_days));
29 }
```

Uncomment the last print statement to observe that the type supplied must be `Years`.

To obtain the `newtype`'s value as the base type, you may use the tuple or destructuring syntax like so:

```
1 struct Years(i64);
2
3 fn main() {
4     let years = Years(42);
5     let years_as_primitive_1: i64 = years.0; // Tuple
6     let Years(years_as_primitive_2) = years; // Destructuring
7 }
```

See also:

[structs](#)

Associated items

"Associated Items" refers to a set of rules pertaining to [items](#) of various types. It is an extension to `trait` generics, and allows `trait`s to internally define new items.

One such item is called an *associated type*, providing simpler usage patterns when the `trait` is generic over its container type.

See also:

[RFC](#)

The Problem

A `trait` that is generic over its container type has type specification requirements - users of the `trait` *must* specify all of its generic types.

In the example below, the `Contains` `trait` allows the use of the generic types `A` and `B`. The trait is then implemented for the `Container` type, specifying `i32` for `A` and `B` so that it can be used with `fn difference()`.

Because `Contains` is generic, we are forced to explicitly state *all* of the generic types for `fn difference()`. In practice, we want a way to express that `A` and `B` are determined by the *input* `c`. As you will see in the next section, associated types provide exactly that capability.

```
1 struct Container(i32, i32);
2
3 // A trait which checks if 2 items are stored inside of container.
4 // Also retrieves first or last value.
5 trait Contains<A, B> {
6     fn contains(&self, _: &A, _: &B) -> bool; // Explicitly requires `A`
7     fn first(&self) -> i32; // Doesn't explicitly require `A` or `B`.
8     fn last(&self) -> i32; // Doesn't explicitly require `A` or `B`.
9 }
10
11 impl Contains<i32, i32> for Container {
12     // True if the numbers stored are equal.
13     fn contains(&self, number_1: &i32, number_2: &i32) -> bool {
14         (&self.0 == number_1) && (&self.1 == number_2)
15     }
16
17     // Grab the first number.
18     fn first(&self) -> i32 { self.0 }
19
20     // Grab the last number.
21     fn last(&self) -> i32 { self.1 }
22 }
23
24 // `C` contains `A` and `B`. In light of that, having to express `A` and
25 // `B` again is a nuisance.
26 fn difference<A, B, C>(container: &C) -> i32 where
27     C: Contains<A, B> {
28     container.last() - container.first()
29 }
30
31 fn main() {
32     let number_1 = 3;
33     let number_2 = 10;
34
35     let container = Container(number_1, number_2);
36
37     println!("Does container contain {} and {}: {}",
38         &number_1, &number_2,
39         container.contains(&number_1, &number_2));
40     println!("First number: {}", container.first());
41     println!("Last number: {}", container.last());
42
43     println!("The difference is: {}", difference(&container));
44 }
```

See also:

[struct S](#), and [trait S](#)

Associated types

The use of "Associated types" improves the overall readability of code by moving inner types locally into a trait as *output* types. Syntax for the `trait` definition is as follows:

```
// `A` and `B` are defined in the trait via the `type` keyword.  
// (Note: `type` in this context is different from `type` when used for  
// aliases).  
trait Contains {  
    type A;  
    type B;  
  
    // Updated syntax to refer to these new types generically.  
    fn contains(&self, _: &Self::A, _: &Self::B) -> bool;  
}
```

Note that functions that use the `trait Contains` are no longer required to express `A` or `B` at all:

```
// Without using associated types  
fn difference<A, B, C>(container: &C) -> i32 where  
    C: Contains<A, B> { ... }  
  
// Using associated types  
fn difference<C: Contains>(container: &C) -> i32 { ... }
```

Let's rewrite the example from the previous section using associated types:

```
1 struct Container(i32, i32);
2
3 // A trait which checks if 2 items are stored inside of container.
4 // Also retrieves first or last value.
5 trait Contains {
6     // Define generic types here which methods will be able to utilize.
7     type A;
8     type B;
9
10    fn contains(&self, _: &Self::A, _: &Self::B) -> bool;
11    fn first(&self) -> i32;
12    fn last(&self) -> i32;
13 }
14
15 impl Contains for Container {
16     // Specify what types `A` and `B` are. If the `input` type
17     // is `Container(i32, i32)`, the `output` types are determined
18     // as `i32` and `i32`.
19     type A = i32;
20     type B = i32;
21
22     // `&Self::A` and `&Self::B` are also valid here.
23     fn contains(&self, number_1: &i32, number_2: &i32) -> bool {
24         (&self.0 == number_1) && (&self.1 == number_2)
25     }
26     // Grab the first number.
27     fn first(&self) -> i32 { self.0 }
28
29     // Grab the last number.
30     fn last(&self) -> i32 { self.1 }
31 }
32
33 fn difference<C: Contains>(container: &C) -> i32 {
34     container.last() - container.first()
35 }
36
37 fn main() {
38     let number_1 = 3;
39     let number_2 = 10;
40
41     let container = Container(number_1, number_2);
42
43     println!("Does container contain {} and {}: {}",
44         &number_1, &number_2,
45         container.contains(&number_1, &number_2));
46     println!("First number: {}", container.first());
47     println!("Last number: {}", container.last());
48
49     println!("The difference is: {}", difference(&container));
50 }
```

Phantom type parameters

A phantom type parameter is one that doesn't show up at runtime, but is checked statically (and only) at compile time.

Data types can use extra generic type parameters to act as markers or to perform type checking at compile time. These extra parameters hold no storage values, and have no runtime behavior.

In the following example, we combine `std::marker::PhantomData` with the phantom type parameter concept to create tuples containing different data types.

```
1 use std::marker::PhantomData;
2
3 // A phantom tuple struct which is generic over `A` with hidden parameter
4 #[derive(PartialEq)] // Allow equality test for this type.
5 struct PhantomTuple<A, B>(A, PhantomData<B>);
6
7 // A phantom type struct which is generic over `A` with hidden parameter
8 #[derive(PartialEq)] // Allow equality test for this type.
9 struct PhantomStruct<A, B> { first: A, phantom: PhantomData<B> }
10
11 // Note: Storage is allocated for generic type `A`, but not for `B`.
12 //       Therefore, `B` cannot be used in computations.
13
14 fn main() {
15     // Here, `f32` and `f64` are the hidden parameters.
16     // PhantomTuple type specified as ``.
17     let _tuple1: PhantomTuple<char, f32> = PhantomTuple('Q', PhantomData)
18     // PhantomTuple type specified as ``.
19     let _tuple2: PhantomTuple<char, f64> = PhantomTuple('Q', PhantomData)
20
21     // Type specified as ``.
22     let _struct1: PhantomStruct<char, f32> = PhantomStruct {
23         first: 'Q',
24         phantom: PhantomData,
25     };
26     // Type specified as ``.
27     let _struct2: PhantomStruct<char, f64> = PhantomStruct {
28         first: 'Q',
29         phantom: PhantomData,
30     };
31
32     // Compile-time Error! Type mismatch so these cannot be compared:
33     // println!("_tuple1 == _tuple2 yields: {}",
34     //         _tuple1 == _tuple2);
35
36     // Compile-time Error! Type mismatch so these cannot be compared:
37     // println!("_struct1 == _struct2 yields: {}",
38     //         _struct1 == _struct2);
39 }
```

See also:

[Derive](#), [struct](#), and [TupleStructs](#)

Testcase: unit clarification

A useful method of unit conversions can be examined by implementing `Add` with a phantom type parameter. The `Add` trait is examined below:

```
// This construction would impose: `Self + RHS = Output`  
// where RHS defaults to Self if not specified in the implementation.  
pub trait Add<RHS = Self> {  
    type Output;  
  
    fn add(self, rhs: RHS) -> Self::Output;  
}  
  
// `Output` must be `T<U>` so that `T<U> + T<U> = T<U>`.  
impl<U> Add for T<U> {  
    type Output = T<U>;  
    ...  
}
```

The whole implementation:

```
1 use std::ops::Add;
2 use std::marker::PhantomData;
3
4 /// Create void enumerations to define unit types.
5 #[derive(Debug, Clone, Copy)]
6 enum Inch {}
7 #[derive(Debug, Clone, Copy)]
8 enum Mm {}
9
10 /// `Length` is a type with phantom type parameter `Unit`,
11 /// and is not generic over the length type (that is `f64`).
12 ///
13 /// `f64` already implements the `Clone` and `Copy` traits.
14 #[derive(Debug, Clone, Copy)]
15 struct Length<Unit>(f64, PhantomData<Unit>);
16
17 /// The `Add` trait defines the behavior of the `+` operator.
18 impl<Unit> Add for Length<Unit> {
19     type Output = Length<Unit>;
20
21     // add() returns a new `Length` struct containing the sum.
22     fn add(self, rhs: Length<Unit>) -> Length<Unit> {
23         // `+` calls the `Add` implementation for `f64`.
24         Length(self.0 + rhs.0, PhantomData)
25     }
26 }
27
28 fn main() {
29     // Specifies `one_foot` to have phantom type parameter `Inch`.
30     let one_foot: Length<Inch> = Length(12.0, PhantomData);
31     // `one_meter` has phantom type parameter `Mm`.
32     let one_meter: Length<Mm> = Length(1000.0, PhantomData);
33
34     // `+` calls the `add()` method we implemented for `Length<Unit>`.
35     //
36     // Since `Length` implements `Copy`, `add()` does not consume
37     // `one_foot` and `one_meter` but copies them into `self` and `rhs`.
38     let two_feet = one_foot + one_foot;
39     let two_meters = one_meter + one_meter;
40
41     // Addition works.
42     println!("one foot + one_foot = {:?} in", two_feet.0);
43     println!("one meter + one_meter = {:?} mm", two_meters.0);
44
45     // Nonsensical operations fail as they should:
46     // Compile-time Error: type mismatch.
47     //let one_feter = one_foot + one_meter;
48 }
```

See also:

[Borrowing \(&\)](#), [Bounds \(x: y\)](#), [enum](#), [impl & self](#), [Overloading](#), [ref](#), [Traits \(x for y\)](#), and [TupleStructs](#).

Scoping rules

Scopes play an important part in ownership, borrowing, and lifetimes. That is, they indicate to the compiler when borrows are valid, when resources can be freed, and when variables are created or destroyed.

RAII

Variables in Rust do more than just hold data in the stack: they also *own* resources, e.g. `Box<T>` owns memory in the heap. Rust enforces [RAII](#) (Resource Acquisition Is Initialization), so whenever an object goes out of scope, its destructor is called and its owned resources are freed.

This behavior shields against *resource leak* bugs, so you'll never have to manually free memory or worry about memory leaks again! Here's a quick showcase:

```
1 // raii.rs
2 fn create_box() {
3     // Allocate an integer on the heap
4     let _box1 = Box::new(3i32);
5
6     // `_box1` is destroyed here, and memory gets freed
7 }
8
9 fn main() {
10    // Allocate an integer on the heap
11    let _box2 = Box::new(5i32);
12
13    // A nested scope:
14    {
15        // Allocate an integer on the heap
16        let _box3 = Box::new(4i32);
17
18        // `_box3` is destroyed here, and memory gets freed
19    }
20
21    // Creating lots of boxes just for fun
22    // There's no need to manually free memory!
23    for _ in 0u32..1_000 {
24        create_box();
25    }
26
27    // `_box2` is destroyed here, and memory gets freed
28 }
```

Of course, we can double check for memory errors using [valgrind](#):

```
$ rustc raii.rs && valgrind ./raii
==26873== Memcheck, a memory error detector
==26873== Copyright (C) 2002-2013, and GNU GPL'd, by Julian Seward et al.
==26873== Using Valgrind-3.9.0 and LibVEX; rerun with -h for copyright info
==26873== Command: ./raii
==26873==
==26873==
==26873== HEAP SUMMARY:
==26873==       in use at exit: 0 bytes in 0 blocks
==26873==   total heap usage: 1,013 allocs, 1,013 frees, 8,696 bytes
allocated
==26873==
==26873== All heap blocks were freed -- no leaks are possible
==26873==
==26873== For counts of detected and suppressed errors, rerun with: -v
==26873== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 2 from 2)
```

No leaks here!

Destructor

The notion of a destructor in Rust is provided through the [Drop](#) trait. The destructor is called when the resource goes out of scope. This trait is not required to be implemented for every type, only implement it for your type if you require its own destructor logic.

Run the below example to see how the [Drop](#) trait works. When the variable in the `main` function goes out of scope the custom destructor will be invoked.

```
1 struct ToDrop;
2
3 impl Drop for ToDrop {
4     fn drop(&mut self) {
5         println!("ToDrop is being dropped");
6     }
7 }
8
9 fn main() {
10     let x = ToDrop;
11     println!("Made a ToDrop!");
12 }
```

See also:

[Box](#)

Ownership and moves

Because variables are in charge of freeing their own resources, **resources can only have one owner**. This also prevents resources from being freed more than once. Note that not all variables own resources (e.g. [references](#)).

When doing assignments (`let x = y`) or passing function arguments by value (`foo(x)`), the *ownership* of the resources is transferred. In Rust-speak, this is known as a *move*.

After moving resources, the previous owner can no longer be used. This avoids creating dangling pointers.

```
1 // This function takes ownership of the heap allocated memory
2 fn destroy_box(c: Box<i32>) {
3     println!("Destroying a box that contains {}", c);
4
5     // `c` is destroyed and the memory freed
6 }
7
8 fn main() {
9     // _Stack_ allocated integer
10    let x = 5u32;
11
12    // *Copy* `x` into `y` - no resources are moved
13    let y = x;
14
15    // Both values can be independently used
16    println!("x is {}, and y is {}", x, y);
17
18    // `a` is a pointer to a _heap_ allocated integer
19    let a = Box::new(5i32);
20
21    println!("a contains: {}", a);
22
23    // *Move* `a` into `b`
24    let b = a;
25    // The pointer address of `a` is copied (not the data) into `b`.
26    // Both are now pointers to the same heap allocated data, but
27    // `b` now owns it.
28
29    // Error! `a` can no longer access the data, because it no longer own
30    // heap memory
31    //println!("a contains: {}", a);
32    // TODO ^ Try uncommenting this line
33
34    // This function takes ownership of the heap allocated memory from `b`
35    destroy_box(b);
36
37    // Since the heap memory has been freed at this point, this action wo
38    // result in dereferencing freed memory, but it's forbidden by the co
39    // Error! Same reason as the previous Error
40    //println!("b contains: {}", b);
41    // TODO ^ Try uncommenting this line
42 }
```

Mutability

Mutability of data can be changed when ownership is transferred.

```
1 fn main() {
2     let immutable_box = Box::new(5u32);
3
4     println!("immutable_box contains {}", immutable_box);
5
6     // Mutability error
7     /*immutable_box = 4;
8
9     // *Move* the box, changing the ownership (and mutability)
10    let mut mutable_box = immutable_box;
11
12    println!("mutable_box contains {}", mutable_box);
13
14    // Modify the contents of the box
15    *mutable_box = 4;
16
17    println!("mutable_box now contains {}", mutable_box);
18 }
```

Partial moves

Within the [destructuring](#) of a single variable, both `by-move` and `by-reference` pattern bindings can be used at the same time. Doing this will result in a *partial move* of the variable, which means that parts of the variable will be moved while other parts stay. In such a case, the parent variable cannot be used afterwards as a whole, however the parts that are only referenced (and not moved) can still be used.

```

1 fn main() {
2     #[derive(Debug)]
3     struct Person {
4         name: String,
5         age: Box<u8>,
6     }
7
8     let person = Person {
9         name: String::from("Alice"),
10        age: Box::new(20),
11    };
12
13    // `name` is moved out of person, but `age` is referenced
14    let Person { name, ref age } = person;
15
16    println!("The person's age is {}", age);
17
18    println!("The person's name is {}", name);
19
20    // Error! borrow of partially moved value: `person` partial move occurs here
21    //println!("The person struct is {:?}", person);
22
23    // `person` cannot be used but `person.age` can be used as it is not moved
24    println!("The person's age from person struct is {}", person.age);
25 }
```

(In this example, we store the `age` variable on the heap to illustrate the partial move: deleting `ref` in the above code would give an error as the ownership of `person.age` would be moved to the variable `age`. If `Person.age` were stored on the stack, `ref` would not be required as the definition of `age` would copy the data from `person.age` without moving it.)

See also:

[destructuring](#)

Borrowing

Most of the time, we'd like to access data without taking ownership over it. To accomplish this, Rust uses a *borrowing* mechanism. Instead of passing objects by value (`T`), objects can be passed by reference (`&T`).

The compiler statically guarantees (via its borrow checker) that references *always* point to valid objects. That is, while references to an object exist, the object cannot be destroyed.

```
1 // This function takes ownership of a box and destroys it
2 fn eat_box_i32(boxed_i32: Box<i32>) {
3     println!("Destroying box that contains {}", boxed_i32);
4 }
5
6 // This function borrows an i32
7 fn borrow_i32(borrowed_i32: &i32) {
8     println!("This int is: {}", borrowed_i32);
9 }
10
11 fn main() {
12     // Create a boxed i32, and a stacked i32
13     let boxed_i32 = Box::new(5_i32);
14     let stacked_i32 = 6_i32;
15
16     // Borrow the contents of the box. Ownership is not taken,
17     // so the contents can be borrowed again.
18     borrow_i32(&boxed_i32);
19     borrow_i32(&stacked_i32);
20
21     {
22         // Take a reference to the data contained inside the box
23         let _ref_to_i32: &i32 = &boxed_i32;
24
25         // Error!
26         // Can't destroy `boxed_i32` while the inner value is borrowed la
27         eat_box_i32(boxed_i32);
28         // FIXME ^ Comment out this line
29
30         // Attempt to borrow `_ref_to_i32` after inner value is destroyed
31         borrow_i32(_ref_to_i32);
32         // `_ref_to_i32` goes out of scope and is no longer borrowed.
33     }
34
35     // `boxed_i32` can now give up ownership to `eat_box` and be destroye
36     eat_box_i32(boxed_i32);
37 }
```


Mutability

Mutable data can be mutably borrowed using `&mut T`. This is called a *mutable reference* and gives read/write access to the borrower. In contrast, `&T` borrows the data via an immutable reference, and the borrower can read the data but not modify it:

```
1  #[allow(dead_code)]
2  #[derive(Clone, Copy)]
3  struct Book {
4      // `&'static str` is a reference to a string allocated in read only m
5      author: &'static str,
6      title: &'static str,
7      year: u32,
8  }
9
10 // This function takes a reference to a book
11 fn borrow_book(book: &Book) {
12     println!("I immutably borrowed {} - {} edition", book.title, book.yea
13 }
14
15 // This function takes a reference to a mutable book and changes `year` t
16 fn new_edition(book: &mut Book) {
17     book.year = 2014;
18     println!("I mutably borrowed {} - {} edition", book.title, book.year)
19 }
20
21 fn main() {
22     // Create an immutable Book named `immutabook`
23     let immutabook = Book {
24         // string literals have type `&'static str`
25         author: "Douglas Hofstadter",
26         title: "Gödel, Escher, Bach",
27         year: 1979,
28     };
29
30     // Create a mutable copy of `immutabook` and call it `mutabook`
31     let mut mutabook = immutabook;
32
33     // Immutably borrow an immutable object
34     borrow_book(&immutabook);
35
36     // Immutably borrow a mutable object
37     borrow_book(&mutabook);
38
39     // Borrow a mutable object as mutable
40     new_edition(&mut mutabook);
41
42     // Error! Cannot borrow an immutable object as mutable
43     new_edition(&mut immutabook);
44     // FIXME ^ Comment out this line
45 }
```

See also:[static](#)

Aliasing

Data can be immutably borrowed any number of times, but while immutably borrowed, the original data can't be mutably borrowed. On the other hand, only *one* mutable borrow is allowed at a time. The original data can be borrowed again only *after* the mutable reference has been used for the last time.

```
1 struct Point { x: i32, y: i32, z: i32 }
2
3 fn main() {
4     let mut point = Point { x: 0, y: 0, z: 0 };
5
6     let borrowed_point = &point;
7     let another_borrow = &point;
8
9     // Data can be accessed via the references and the original owner
10    println!("Point has coordinates: ({}, {}, {})",
11            borrowed_point.x, another_borrow.y, point.z);
12
13    // Error! Can't borrow `point` as mutable because it's currently
14    // borrowed as immutable.
15    // let mutable_borrow = &mut point;
16    // TODO ^ Try uncommenting this line
17
18    // The borrowed values are used again here
19    println!("Point has coordinates: ({}, {}, {})",
20            borrowed_point.x, another_borrow.y, point.z);
21
22    // The immutable references are no longer used for the rest of the co
23    // it is possible to reborrow with a mutable reference.
24    let mutable_borrow = &mut point;
25
26    // Change data via mutable reference
27    mutable_borrow.x = 5;
28    mutable_borrow.y = 2;
29    mutable_borrow.z = 1;
30
31    // Error! Can't borrow `point` as immutable because it's currently
32    // borrowed as mutable.
33    // let y = &point.y;
34    // TODO ^ Try uncommenting this line
35
36    // Error! Can't print because `println!` takes an immutable reference
37    // println!("Point Z coordinate is {}", point.z);
38    // TODO ^ Try uncommenting this line
39
40    // Ok! Mutable references can be passed as immutable to `println!`
41    println!("Point has coordinates: ({}, {}, {})",
42            mutable_borrow.x, mutable_borrow.y, mutable_borrow.z);
43
44    // The mutable reference is no longer used for the rest of the code s
45    // is possible to reborrow
46    let new_borrowed_point = &point;
47    println!("Point now has coordinates: ({}, {}, {})",
48            new_borrowed_point.x, new_borrowed_point.y, new_borrowed_poi
49 }
```

The `ref` pattern

When doing pattern matching or destructuring via the `let` binding, the `ref` keyword can be used to take references to the fields of a struct/tuple. The example below shows a few instances where this can be useful:

```
1  #[derive(Clone, Copy)]
2  struct Point { x: i32, y: i32 }
3
4  fn main() {
5      let c = 'Q';
6
7      // A `ref` borrow on the left side of an assignment is equivalent to
8      // an `&` borrow on the right side.
9      let ref ref_c1 = c;
10     let ref_c2 = &c;
11
12     println!("ref_c1 equals ref_c2: {}", *ref_c1 == *ref_c2);
13
14     let point = Point { x: 0, y: 0 };
15
16     // `ref` is also valid when destructuring a struct.
17     let _copy_of_x = {
18         // `ref_to_x` is a reference to the `x` field of `point`.
19         let Point { x: ref ref_to_x, y: _ } = point;
20
21         // Return a copy of the `x` field of `point`.
22         *ref_to_x
23     };
24
25     // A mutable copy of `point`
26     let mut mutable_point = point;
27
28     {
29         // `ref` can be paired with `mut` to take mutable references.
30         let Point { x: _, y: ref mut mut_ref_to_y } = mutable_point;
31
32         // Mutate the `y` field of `mutable_point` via a mutable reference
33         *mut_ref_to_y = 1;
34     }
35
36     println!("point is ({}, {})", point.x, point.y);
37     println!("mutable_point is ({}, {})", mutable_point.x, mutable_point.y);
38
39     // A mutable tuple that includes a pointer
40     let mut mutable_tuple = (Box::new(5u32), 3u32);
41
42     {
43         // Destructure `mutable_tuple` to change the value of `last`.
44         let (_, ref mut last) = mutable_tuple;
45         *last = 2u32;
46     }
47
48     println!("tuple is {:?}", mutable_tuple);
49 }
```

Lifetimes

A *lifetime* is a construct of the compiler (or more specifically, its *borrow checker*) uses to ensure all borrows are valid. Specifically, a variable's lifetime begins when it is created and ends when it is destroyed. While lifetimes and scopes are often referred to together, they are not the same.

Take, for example, the case where we borrow a variable via `&`. The borrow has a lifetime that is determined by where it is declared. As a result, the borrow is valid as long as it ends before the lender is destroyed. However, the scope of the borrow is determined by where the reference is used.

In the following example and in the rest of this section, we will see how lifetimes relate to scopes, as well as how the two differ.

```
1 // Lifetimes are annotated below with lines denoting the creation
2 // and destruction of each variable.
3 // `i` has the longest lifetime because its scope entirely encloses
4 // both `borrow1` and `borrow2`. The duration of `borrow1` compared
5 // to `borrow2` is irrelevant since they are disjoint.
6 fn main() {
7     let i = 3; // Lifetime for `i` starts.
8     //
9     { //
10         let borrow1 = &i; // `borrow1` lifetime starts.
11         //
12         println!("borrow1: {}", borrow1); //
13     } // `borrow1` ends.
14     //
15     //
16     { //
17         let borrow2 = &i; // `borrow2` lifetime starts.
18         //
19         println!("borrow2: {}", borrow2); //
20     } // `borrow2` ends.
21     //
22 } // Lifetime ends.
```

Note that no names or types are assigned to label lifetimes. This restricts how lifetimes will be able to be used as we will see.

Explicit annotation

The borrow checker uses explicit lifetime annotations to determine how long references should be valid. In cases where lifetimes are not elided¹, Rust requires explicit annotations to determine what the lifetime of a reference should be. The syntax for explicitly annotating a lifetime uses an apostrophe character as follows:

```
foo<'a>
// `foo` has a lifetime parameter `'a`
```

Similar to [closures](#), using lifetimes requires generics. Additionally, this lifetime syntax indicates that the lifetime of `foo` may not exceed that of `'a`. Explicit annotation of a type has the form `&'a T` where `'a` has already been introduced.

In cases with multiple lifetimes, the syntax is similar:

```
foo<'a, 'b>
// `foo` has lifetime parameters `'a` and `'b`
```

In this case, the lifetime of `foo` cannot exceed that of either `'a` or `'b`.

See the following example for explicit lifetime annotation in use:


```
1 // `print_refs` takes two references to `i32` which have different
2 // lifetimes `'a` and `'b`. These two lifetimes must both be at
3 // least as long as the function `print_refs`.
4 fn print_refs<'a, 'b>(x: &'a i32, y: &'b i32) {
5     println!("x is {} and y is {}", x, y);
6 }
7
8 // A function which takes no arguments, but has a lifetime parameter `'a`
9 fn failed_borrow<'a>() {
10     let _x = 12;
11
12     // ERROR: `_x` does not live long enough
13     let y: &'a i32 = &_x;
14     // Attempting to use the lifetime `'a` as an explicit type annotation
15     // inside the function will fail because the lifetime of `&_x` is sho
16     // than that of `y`. A short lifetime cannot be coerced into a longer
17 }
18
19 fn main() {
20     // Create variables to be borrowed below.
21     let (four, nine) = (4, 9);
22
23     // Borrows (`&`) of both variables are passed into the function.
24     print_refs(&four, &nine);
25     // Any input which is borrowed must outlive the borrower.
26     // In other words, the lifetime of `four` and `nine` must
27     // be longer than that of `print_refs`.
28
29     failed_borrow();
30     // `failed_borrow` contains no references to force `'a` to be
31     // longer than the lifetime of the function, but `'a` is longer.
32     // Because the lifetime is never constrained, it defaults to `'static
33 }
```

¹ [elision](#) implicitly annotates lifetimes and so is different.

See also:

[generics](#) and [closures](#)

Functions

Ignoring [elision](#), function signatures with lifetimes have a few constraints:

- any reference *must* have an annotated lifetime.
- any reference being returned *must* have the same lifetime as an input or be `static`.

Additionally, note that returning references without input is banned if it would result in returning references to invalid data. The following example shows off some valid forms of functions with lifetimes:

```
1 // One input reference with lifetime `a` which must live
2 // at least as long as the function.
3 fn print_one<'a>(x: &'a i32) {
4     println!("`print_one`: x is {}", x);
5 }
6
7 // Mutable references are possible with lifetimes as well.
8 fn add_one<'a>(x: &'a mut i32) {
9     *x += 1;
10 }
11
12 // Multiple elements with different lifetimes. In this case, it
13 // would be fine for both to have the same lifetime `a`, but
14 // in more complex cases, different lifetimes may be required.
15 fn print_multi<'a, 'b>(x: &'a i32, y: &'b i32) {
16     println!("`print_multi`: x is {}, y is {}", x, y);
17 }
18
19 // Returning references that have been passed in is acceptable.
20 // However, the correct lifetime must be returned.
21 fn pass_x<'a, 'b>(x: &'a i32, _: &'b i32) -> &'a i32 { x }
22
23 //fn invalid_output<'a>() -> &'a String { &String::from("foo") }
24 // The above is invalid: `a` must live longer than the function.
25 // Here, `&String::from("foo")` would create a `String`, followed by a
26 // reference. Then the data is dropped upon exiting the scope, leaving
27 // a reference to invalid data to be returned.
28
29 fn main() {
30     let x = 7;
31     let y = 9;
32
33     print_one(&x);
34     print_multi(&x, &y);
35
36     let z = pass_x(&x, &y);
37     print_one(z);
38
39     let mut t = 3;
40     add_one(&mut t);
41     print_one(&t);
42 }
```

See also:[functions](#)

Methods

Methods are annotated similarly to functions:

```
1 struct Owner(i32);
2
3 impl Owner {
4     // Annotate lifetimes as in a standalone function.
5     fn add_one<'a>(&'a mut self) { self.0 += 1; }
6     fn print<'a>(&'a self) {
7         println!("`print`: {}", self.0);
8     }
9 }
10
11 fn main() {
12     let mut owner = Owner(18);
13
14     owner.add_one();
15     owner.print();
16 }
```

See also:

[methods](#)

Structs

Annotation of lifetimes in structures are also similar to functions:

```
1 // A type `Borrowed` which houses a reference to an
2 // `i32`. The reference to `i32` must outlive `Borrowed`.
3 #[derive(Debug)]
4 struct Borrowed<'a>(&'a i32);
5
6 // Similarly, both references here must outlive this structure.
7 #[derive(Debug)]
8 struct NamedBorrowed<'a> {
9     x: &'a i32,
10    y: &'a i32,
11 }
12
13 // An enum which is either an `i32` or a reference to one.
14 #[derive(Debug)]
15 enum Either<'a> {
16     Num(i32),
17     Ref(&'a i32),
18 }
19
20 fn main() {
21     let x = 18;
22     let y = 15;
23
24     let single = Borrowed(&x);
25     let double = NamedBorrowed { x: &x, y: &y };
26     let reference = Either::Ref(&x);
27     let number    = Either::Num(y);
28
29     println!("x is borrowed in {:?}", single);
30     println!("x and y are borrowed in {:?}", double);
31     println!("x is borrowed in {:?}", reference);
32     println!("y is *not* borrowed in {:?}", number);
33 }
```

See also:

[struct S](#)

Traits

Annotation of lifetimes in trait methods basically are similar to functions. Note that `impl` may have annotation of lifetimes too.

```
1 // A struct with annotation of lifetimes.
2 #[derive(Debug)]
3 struct Borrowed<'a> {
4     x: &'a i32,
5 }
6
7 // Annotate lifetimes to impl.
8 impl<'a> Default for Borrowed<'a> {
9     fn default() -> Self {
10         Self {
11             x: &10,
12         }
13     }
14 }
15
16 fn main() {
17     let b: Borrowed = Default::default();
18     println!("b is {:?}", b);
19 }
```

See also:

[traits](#)

Bounds

Just like generic types can be bounded, lifetimes (themselves generic) use bounds as well. The `:` character has a slightly different meaning here, but `+` is the same. Note how the following read:

1. `T: 'a:` *All* references in `T` must outlive lifetime `'a`.
2. `T: Trait + 'a:` Type `T` must implement trait `Trait` and *all* references in `T` must outlive `'a`.

The example below shows the above syntax in action used after keyword `where`:

```

1 use std::fmt::Debug; // Trait to bound with.
2
3 #[derive(Debug)]
4 struct Ref<'a, T: 'a>(&'a T);
5 // `Ref` contains a reference to a generic type `T` that has
6 // an unknown lifetime `a`. `T` is bounded such that any
7 // *references* in `T` must outlive `a`. Additionally, the lifetime
8 // of `Ref` may not exceed `a`.
9
10 // A generic function which prints using the `Debug` trait.
11 fn print<T>(t: T) where
12     T: Debug {
13     println!("`print`: t is {:?}", t);
14 }
15
16 // Here a reference to `T` is taken where `T` implements
17 // `Debug` and all *references* in `T` outlive `a`. In
18 // addition, `a` must outlive the function.
19 fn print_ref<'a, T>(t: &'a T) where
20     T: Debug + 'a {
21     println!("`print_ref`: t is {:?}", t);
22 }
23
24 fn main() {
25     let x = 7;
26     let ref_x = Ref(&x);
27
28     print_ref(&ref_x);
29     print(ref_x);
30 }
```

See also:

[generics](#), [bounds in generics](#), and [multiple bounds in generics](#)

Coercion

A longer lifetime can be coerced into a shorter one so that it works inside a scope it normally wouldn't work in. This comes in the form of inferred coercion by the Rust compiler, and also in the form of declaring a lifetime difference:

```
1 // Here, Rust infers a lifetime that is as short as possible.
2 // The two references are then coerced to that lifetime.
3 fn multiply<'a>(first: &'a i32, second: &'a i32) -> i32 {
4     first * second
5 }
6
7 // `<'a: 'b, 'b>` reads as lifetime `a` is at least as long as `b`.
8 // Here, we take in an `&'a i32` and return a `&'b i32` as a result of co
9 fn choose_first<'a: 'b, 'b>(first: &'a i32, _: &'b i32) -> &'b i32 {
10     first
11 }
12
13 fn main() {
14     let first = 2; // Longer lifetime
15
16     {
17         let second = 3; // Shorter lifetime
18
19         println!("The product is {}", multiply(&first, &second));
20         println!("{}", choose_first(&first, &second));
21     };
22 }
```


Static

Rust has a few reserved lifetime names. One of those is `'static`. You might encounter it in two situations:

```
1 // A reference with 'static lifetime:
2 let s: &'static str = "hello world";
3
4 // 'static as part of a trait bound:
5 fn generic<T>(x: T) where T: 'static {}
```

Both are related but subtly different and this is a common source for confusion when learning Rust. Here are some examples for each situation:

Reference lifetime

As a reference lifetime `'static` indicates that the data pointed to by the reference lives for the entire lifetime of the running program. It can still be coerced to a shorter lifetime.

There are two ways to make a variable with `'static` lifetime, and both are stored in the read-only memory of the binary:

- Make a constant with the `static` declaration.
- Make a `string` literal which has type: `&'static str`.

See the following example for a display of each method:

```
1 // Make a constant with `static` lifetime.
2 static NUM: i32 = 18;
3
4 // Returns a reference to `NUM` where its `static`
5 // lifetime is coerced to that of the input argument.
6 fn coerce_static<'a>(_: &'a i32) -> &'a i32 {
7     &NUM
8 }
9
10 fn main() {
11     {
12         // Make a `string` literal and print it:
13         let static_string = "I'm in read-only memory";
14         println!("static_string: {}", static_string);
15
16         // When `static_string` goes out of scope, the reference
17         // can no longer be used, but the data remains in the binary.
18     }
19
20     {
21         // Make an integer to use for `coerce_static`:
22         let lifetime_num = 9;
23
24         // Coerce `NUM` to lifetime of `lifetime_num`:
25         let coerced_static = coerce_static(&lifetime_num);
26
27         println!("coerced_static: {}", coerced_static);
28     }
29
30     println!("NUM: {} stays accessible!", NUM);
31 }
```

Trait bound

As a trait bound, it means the type does not contain any non-static references. Eg. the receiver can hold on to the type for as long as they want and it will never become invalid until they drop it.

It's important to understand this means that any owned data always passes a `'static` lifetime bound, but a reference to that owned data generally does not:

```
1 use std::fmt::Debug;
2
3 fn print_it( input: impl Debug + 'static ) {
4     println!( "'static value passed in is: {:?}'", input );
5 }
6
7 fn main() {
8     // i is owned and contains no references, thus it's 'static:
9     let i = 5;
10    print_it(i);
11
12    // oops, &i only has the lifetime defined by the scope of
13    // main(), so it's not 'static:
14    print_it(&i);
15 }
```

The compiler will tell you:

```
error[E0597]: `i` does not live long enough
--> src/lib.rs:15:15
15 |         print_it(&i);
    |         ^^^^^^^^^^^
    |         |
    |         | borrowed value does not live long enough
    |         | argument requires that `i` is borrowed for `'static`
16 |     }
    |     - `i` dropped here while still borrowed
```

See also:

['static constants](#)

Elision

Some lifetime patterns are overwhelmingly common and so the borrow checker will allow you to omit them to save typing and to improve readability. This is known as elision. Elision exists in Rust solely because these patterns are common.

The following code shows a few examples of elision. For a more comprehensive description of elision, see [lifetime elision](#) in the book.

```
1 // `elided_input` and `annotated_input` essentially have identical signature
2 // because the lifetime of `elided_input` is inferred by the compiler:
3 fn elided_input(x: &i32) {
4     println!("`elided_input`: {}", x);
5 }
6
7 fn annotated_input<'a>(x: &'a i32) {
8     println!("`annotated_input`: {}", x);
9 }
10
11 // Similarly, `elided_pass` and `annotated_pass` have identical signature
12 // because the lifetime is added implicitly to `elided_pass`:
13 fn elided_pass(x: &i32) -> &i32 { x }
14
15 fn annotated_pass<'a>(x: &'a i32) -> &'a i32 { x }
16
17 fn main() {
18     let x = 3;
19
20     elided_input(&x);
21     annotated_input(&x);
22
23     println!("`elided_pass`: {}", elided_pass(&x));
24     println!("`annotated_pass`: {}", annotated_pass(&x));
25 }
```

See also:

[elision](#)

Traits

A `trait` is a collection of methods defined for an unknown type: `self`. They can access other methods declared in the same trait.

Traits can be implemented for any data type. In the example below, we define `Animal`, a group of methods. The `Animal` `trait` is then implemented for the `Sheep` data type, allowing the use of methods from `Animal` with a `Sheep`.

```
1 struct Sheep { naked: bool, name: &'static str }
2
3 trait Animal {
4     // Associated function signature; `Self` refers to the implementor type
5     fn new(name: &'static str) -> Self;
6
7     // Method signatures; these will return a string.
8     fn name(&self) -> &'static str;
9     fn noise(&self) -> &'static str;
10
11     // Traits can provide default method definitions.
12     fn talk(&self) {
13         println!("{}", self.name(), self.noise());
14     }
15 }
16
17 impl Sheep {
18     fn is_naked(&self) -> bool {
19         self.naked
20     }
21
22     fn shear(&mut self) {
23         if self.is_naked() {
24             // Implementor methods can use the implementor's trait method
25             println!("{}", self.name());
26         } else {
27             println!("{}", self.name());
28
29             self.naked = true;
30         }
31     }
32 }
33
34 // Implement the `Animal` trait for `Sheep`.
35 impl Animal for Sheep {
36     // `Self` is the implementor type: `Sheep`.
37     fn new(name: &'static str) -> Sheep {
38         Sheep { name: name, naked: false }
39     }
40
41     fn name(&self) -> &'static str {
42         self.name
43     }
44
45     fn noise(&self) -> &'static str {
46         if self.is_naked() {
47             "baaaaah?"
48         } else {
49             "baaaaah!"
50         }
51     }
52
53     // Default trait methods can be overridden.
54     fn talk(&self) {
55         // For example, we can add some quiet contemplation.
56         println!("{}", self.name(), self.noise());
57     }
58 }
```


Derive

The compiler is capable of providing basic implementations for some traits via the `#[derive]` [attribute](#). These traits can still be manually implemented if a more complex behavior is required.

The following is a list of derivable traits:

- Comparison traits: [Eq](#), [PartialEq](#), [Ord](#), [PartialOrd](#).
- [Clone](#), to create `T` from `&T` via a copy.
- [Copy](#), to give a type 'copy semantics' instead of 'move semantics'.
- [Hash](#), to compute a hash from `&T`.
- [Default](#), to create an empty instance of a data type.
- [Debug](#), to format a value using the `{:?}` formatter.


```
1 // `Centimeters`, a tuple struct that can be compared
2 #[derive(PartialEq, PartialOrd)]
3 struct Centimeters(f64);
4
5 // `Inches`, a tuple struct that can be printed
6 #[derive(Debug)]
7 struct Inches(i32);
8
9 impl Inches {
10     fn to_centimeters(&self) -> Centimeters {
11         let &Inches(inches) = self;
12
13         Centimeters(inches as f64 * 2.54)
14     }
15 }
16
17 // `Seconds`, a tuple struct with no additional attributes
18 struct Seconds(i32);
19
20 fn main() {
21     let _one_second = Seconds(1);
22
23     // Error: `Seconds` can't be printed; it doesn't implement the `Debug`
24     //println!("One second looks like: {:?}", _one_second);
25     // TODO ^ Try uncommenting this line
26
27     // Error: `Seconds` can't be compared; it doesn't implement the `Part
28     //let _this_is_true = (_one_second == _one_second);
29     // TODO ^ Try uncommenting this line
30
31     let foot = Inches(12);
32
33     println!("One foot equals {:?}", foot);
34
35     let meter = Centimeters(100.0);
36
37     let cmp =
38         if foot.to_centimeters() < meter {
39             "smaller"
40         } else {
41             "bigger"
42         };
43
44     println!("One foot is {} than one meter.", cmp);
45 }
```

See also:

[derive](#)

Returning Traits with `dyn`

The Rust compiler needs to know how much space every function's return type requires. This means all your functions have to return a concrete type. Unlike other languages, if you have a trait like `Animal`, you can't write a function that returns `Animal`, because its different implementations will need different amounts of memory.

However, there's an easy workaround. Instead of returning a trait object directly, our functions return a `Box` which *contains* some `Animal`. A `box` is just a reference to some memory in the heap. Because a reference has a statically-known size, and the compiler can guarantee it points to a heap-allocated `Animal`, we can return a trait from our function!

Rust tries to be as explicit as possible whenever it allocates memory on the heap. So if your function returns a pointer-to-trait-on-heap in this way, you need to write the return type with the `dyn` keyword, e.g. `Box<dyn Animal>`.

```
1 struct Sheep {}
2 struct Cow {}
3
4 trait Animal {
5     // Instance method signature
6     fn noise(&self) -> &'static str;
7 }
8
9 // Implement the `Animal` trait for `Sheep`.
10 impl Animal for Sheep {
11     fn noise(&self) -> &'static str {
12         "baaaaaah!"
13     }
14 }
15
16 // Implement the `Animal` trait for `Cow`.
17 impl Animal for Cow {
18     fn noise(&self) -> &'static str {
19         "mooooooo!"
20     }
21 }
22
23 // Returns some struct that implements Animal, but we don't know which one
24 fn random_animal(random_number: f64) -> Box<dyn Animal> {
25     if random_number < 0.5 {
26         Box::new(Sheep {})
27     } else {
28         Box::new(Cow {})
29     }
30 }
31
32 fn main() {
33     let random_number = 0.234;
34     let animal = random_animal(random_number);
35     println!("You've randomly chosen an animal, and it says {}", animal.noise());
36 }
37
```

Operator Overloading

In Rust, many of the operators can be overloaded via traits. That is, some operators can be used to accomplish different tasks based on their input arguments. This is possible because operators are syntactic sugar for method calls. For example, the `+` operator in `a + b` calls the `add` method (as in `a.add(b)`). This `add` method is part of the `Add` trait. Hence, the `+` operator can be used by any implementor of the `Add` trait.

A list of the traits, such as `Add`, that overload operators can be found in [core::ops](#).

```
1 use std::ops;
2
3 struct Foo;
4 struct Bar;
5
6 #[derive(Debug)]
7 struct FooBar;
8
9 #[derive(Debug)]
10 struct BarFoo;
11
12 // The `std::ops::Add` trait is used to specify the functionality of `+`.
13 // Here, we make `Add<Bar>` - the trait for addition with a RHS of type `Bar`.
14 // The following block implements the operation: Foo + Bar = FooBar
15 impl ops::Add<Bar> for Foo {
16     type Output = FooBar;
17
18     fn add(self, _rhs: Bar) -> FooBar {
19         println!("> Foo.add(Bar) was called");
20
21         FooBar
22     }
23 }
24
25 // By reversing the types, we end up implementing non-commutative addition
26 // Here, we make `Add<Foo>` - the trait for addition with a RHS of type `Foo`.
27 // This block implements the operation: Bar + Foo = BarFoo
28 impl ops::Add<Foo> for Bar {
29     type Output = BarFoo;
30
31     fn add(self, _rhs: Foo) -> BarFoo {
32         println!("> Bar.add(Foo) was called");
33
34         BarFoo
35     }
36 }
37
38 fn main() {
39     println!("Foo + Bar = {:?}", Foo + Bar);
40     println!("Bar + Foo = {:?}", Bar + Foo);
41 }
```

See Also

[Add, Syntax Index](#)

Drop

The `Drop` trait only has one method: `drop`, which is called automatically when an object goes out of scope. The main use of the `Drop` trait is to free the resources that the implementor instance owns.

`Box`, `Vec`, `String`, `File`, and `Process` are some examples of types that implement the `Drop` trait to free resources. The `Drop` trait can also be manually implemented for any custom data type.

The following example adds a print to console to the `drop` function to announce when it is called.

```
1 struct Droppable {
2     name: &'static str,
3 }
4
5 // This trivial implementation of `drop` adds a print to console.
6 impl Drop for Droppable {
7     fn drop(&mut self) {
8         println!("> Dropping {}", self.name);
9     }
10 }
11
12 fn main() {
13     let _a = Droppable { name: "a" };
14
15     // block A
16     {
17         let _b = Droppable { name: "b" };
18
19         // block B
20         {
21             let _c = Droppable { name: "c" };
22             let _d = Droppable { name: "d" };
23
24             println!("Exiting block B");
25         }
26         println!("Just exited block B");
27
28         println!("Exiting block A");
29     }
30     println!("Just exited block A");
31
32     // Variable can be manually dropped using the `drop` function
33     drop(_a);
34     // TODO ^ Try commenting this line
35
36     println!("end of the main function");
37
38     // `_a` *won't* be `drop`ed again here, because it already has been
39     // (manually) `drop`ed
40 }
```

Iterators

The `Iterator` trait is used to implement iterators over collections such as arrays.

The trait requires only a method to be defined for the `next` element, which may be manually defined in an `impl` block or automatically defined (as in arrays and ranges).

As a point of convenience for common situations, the `for` construct turns some collections into iterators using the `.into_iter()` method.

```
1 struct Fibonacci {
2     curr: u32,
3     next: u32,
4 }
5
6 // Implement `Iterator` for `Fibonacci`.
7 // The `Iterator` trait only requires a method to be defined for the `next`
8 impl Iterator for Fibonacci {
9     // We can refer to this type using Self::Item
10    type Item = u32;
11
12    // Here, we define the sequence using `.curr` and `.next`.
13    // The return type is `Option<T>`:
14    //     * When the `Iterator` is finished, `None` is returned.
15    //     * Otherwise, the next value is wrapped in `Some` and returned.
16    // We use Self::Item in the return type, so we can change
17    // the type without having to update the function signatures.
18    fn next(&mut self) -> Option<Self::Item> {
19        let current = self.curr;
20
21        self.curr = self.next;
22        self.next = current + self.next;
23
24        // Since there's no endpoint to a Fibonacci sequence, the `Iterator`
25        // will never return `None`, and `Some` is always returned.
26        Some(current)
27    }
28 }
29
30 // Returns a Fibonacci sequence generator
31 fn fibonacci() -> Fibonacci {
32     Fibonacci { curr: 0, next: 1 }
33 }
34
35 fn main() {
36     // `0..3` is an `Iterator` that generates: 0, 1, and 2.
37     let mut sequence = 0..3;
38
39     println!("Four consecutive `next` calls on 0..3");
40     println!("> {:?}", sequence.next());
41     println!("> {:?}", sequence.next());
42     println!("> {:?}", sequence.next());
43     println!("> {:?}", sequence.next());
44
45     // `for` works through an `Iterator` until it returns `None`.
46     // Each `Some` value is unwrapped and bound to a variable (here, `i`)
47     println!("Iterate through 0..3 using `for`");
48     for i in 0..3 {
49         println!("> {}", i);
50     }
51
52     // The `take(n)` method reduces an `Iterator` to its first `n` terms.
53     println!("The first four terms of the Fibonacci sequence are: ");
54     for i in fibonacci().take(4) {
55         println!("> {}", i);
56     }
57 }
```


impl Trait

`impl Trait` can be used in two locations:

1. as an argument type
2. as a return type

As an argument type

If your function is generic over a trait but you don't mind the specific type, you can simplify the function declaration using `impl Trait` as the type of the argument.

For example, consider the following code:

```
1 fn parse_csv_document<R: std::io::BufRead>(src: R) -> std::io::Result<Vec<
2     src.lines()
3     .map(|line| {
4         // For each line in the source
5         line.map(|line| {
6             // If the line was read successfully, process it, if not,
7             line.split(',') // Split the line separated by commas
8             .map(|entry| String::from(entry.trim())) // Remove le
9             .collect() // Collect all strings in a row into a Vec
10        })
11    })
12    .collect() // Collect all lines into a Vec<Vec<String>>
13 }
```

`parse_csv_document` is generic, allowing it to take any type which implements `BufRead`, such as `BufReader<File>` or `[u8]`, but it's not important what type `R` is, and `R` is only used to declare the type of `src`, so the function can also be written as:

```
1 fn parse_csv_document(src: impl std::io::BufRead) -> std::io::Result<Vec<
2     src.lines()
3     .map(|line| {
4         // For each line in the source
5         line.map(|line| {
6             // If the line was read successfully, process it, if not,
7             line.split(',') // Split the line separated by commas
8             .map(|entry| String::from(entry.trim())) // Remove le
9             .collect() // Collect all strings in a row into a Vec
10        })
11    })
12    .collect() // Collect all lines into a Vec<Vec<String>>
13 }
```

Note that using `impl Trait` as an argument type means that you cannot explicitly state what form of the function you use, i.e. `parse_csv_document::<std::io::Empty>`

`(std::io::empty())` will not work with the second example.

As a return type

If your function returns a type that implements `MyTrait`, you can write its return type as `-> impl MyTrait`. This can help simplify your type signatures quite a lot!

```
1 use std::iter;
2 use std::vec::IntoIter;
3
4 // This function combines two `Vec<i32>` and returns an iterator over it.
5 // Look how complicated its return type is!
6 fn combine_vecs_explicit_return_type(
7     v: Vec<i32>,
8     u: Vec<i32>,
9 ) -> iter::Cycle<iter::Chain<IntoIter<i32>, IntoIter<i32>>> {
10     v.into_iter().chain(u.into_iter()).cycle()
11 }
12
13 // This is the exact same function, but its return type uses `impl Trait`
14 // Look how much simpler it is!
15 fn combine_vecs(
16     v: Vec<i32>,
17     u: Vec<i32>,
18 ) -> impl Iterator<Item=i32> {
19     v.into_iter().chain(u.into_iter()).cycle()
20 }
21
22 fn main() {
23     let v1 = vec![1, 2, 3];
24     let v2 = vec![4, 5];
25     let mut v3 = combine_vecs(v1, v2);
26     assert_eq!(Some(1), v3.next());
27     assert_eq!(Some(2), v3.next());
28     assert_eq!(Some(3), v3.next());
29     assert_eq!(Some(4), v3.next());
30     assert_eq!(Some(5), v3.next());
31     println!("all done");
32 }
```

More importantly, some Rust types can't be written out. For example, every closure has its own unnamed concrete type. Before `impl Trait` syntax, you had to allocate on the heap in order to return a closure. But now you can do it all statically, like this:

```
1 // Returns a function that adds `y` to its input
2 fn make_adder_function(y: i32) -> impl Fn(i32) -> i32 {
3     let closure = move |x: i32| { x + y };
4     closure
5 }
6
7 fn main() {
8     let plus_one = make_adder_function(1);
9     assert_eq!(plus_one(2), 3);
10 }
```

You can also use `impl Trait` to return an iterator that uses `map` or `filter` closures! This makes using `map` and `filter` easier. Because closure types don't have names, you can't write out an explicit return type if your function returns iterators with closures. But with `impl Trait` you can do this easily:

```
1 fn double_positives<'a>(numbers: &'a Vec<i32>) -> impl Iterator<Item = i32> {
2     numbers
3         .iter()
4         .filter(|x| x > &&0)
5         .map(|x| x * 2)
6 }
7
8 fn main() {
9     let singles = vec![-3, -2, 2, 3];
10    let doubles = double_positives(&singles);
11    assert_eq!(doubles.collect::<Vec<i32>>(), vec![4, 6]);
12 }
```

Clone

When dealing with resources, the default behavior is to transfer them during assignments or function calls. However, sometimes we need to make a copy of the resource as well.

The `Clone` trait helps us do exactly this. Most commonly, we can use the `.clone()` method defined by the `Clone` trait.

```
1 // A unit struct without resources
2 #[derive(Debug, Clone, Copy)]
3 struct Unit;
4
5 // A tuple struct with resources that implements the `Clone` trait
6 #[derive(Clone, Debug)]
7 struct Pair(Box<i32>, Box<i32>);
8
9 fn main() {
10     // Instantiate `Unit`
11     let unit = Unit;
12     // Copy `Unit`, there are no resources to move
13     let copied_unit = unit;
14
15     // Both `Unit`s can be used independently
16     println!("original: {:?}", unit);
17     println!("copy: {:?}", copied_unit);
18
19     // Instantiate `Pair`
20     let pair = Pair(Box::new(1), Box::new(2));
21     println!("original: {:?}", pair);
22
23     // Move `pair` into `moved_pair`, moves resources
24     let moved_pair = pair;
25     println!("moved: {:?}", moved_pair);
26
27     // Error! `pair` has lost its resources
28     //println!("original: {:?}", pair);
29     // TODO ^ Try uncommenting this line
30
31     // Clone `moved_pair` into `cloned_pair` (resources are included)
32     let cloned_pair = moved_pair.clone();
33     // Drop the original pair using std::mem::drop
34     drop(moved_pair);
35
36     // Error! `moved_pair` has been dropped
37     //println!("copy: {:?}", moved_pair);
38     // TODO ^ Try uncommenting this line
39
40     // The result from .clone() can still be used!
41     println!("clone: {:?}", cloned_pair);
42 }
```

Supertraits

Rust doesn't have "inheritance", but you can define a trait as being a superset of another trait. For example:

```
1 trait Person {
2     fn name(&self) -> String;
3 }
4
5 // Person is a supertrait of Student.
6 // Implementing Student requires you to also impl Person.
7 trait Student: Person {
8     fn university(&self) -> String;
9 }
10
11 trait Programmer {
12     fn fav_language(&self) -> String;
13 }
14
15 // CompSciStudent (computer science student) is a subtrait of both Program
16 // and Student. Implementing CompSciStudent requires you to impl both sup
17 trait CompSciStudent: Programmer + Student {
18     fn git_username(&self) -> String;
19 }
20
21 fn comp_sci_student_greeting(student: &dyn CompSciStudent) -> String {
22     format!(
23         "My name is {} and I attend {}. My favorite language is {}. My Gi
24         student.name(),
25         student.university(),
26         student.fav_language(),
27         student.git_username()
28     )
29 }
30
31 fn main() {}
```

See also:

[The Rust Programming Language chapter on supertraits](#)

Disambiguating overlapping traits

A type can implement many different traits. What if two traits both require the same name? For example, many traits might have a method named `get()`. They might even have different return types!

Good news: because each trait implementation gets its own `impl` block, it's clear which trait's `get` method you're implementing.

What about when it comes time to *call* those methods? To disambiguate between them, we have to use Fully Qualified Syntax.

```
1 trait UsernameWidget {
2     // Get the selected username out of this widget
3     fn get(&self) -> String;
4 }
5
6 trait AgeWidget {
7     // Get the selected age out of this widget
8     fn get(&self) -> u8;
9 }
10
11 // A form with both a UsernameWidget and an AgeWidget
12 struct Form {
13     username: String,
14     age: u8,
15 }
16
17 impl UsernameWidget for Form {
18     fn get(&self) -> String {
19         self.username.clone()
20     }
21 }
22
23 impl AgeWidget for Form {
24     fn get(&self) -> u8 {
25         self.age
26     }
27 }
28
29 fn main() {
30     let form = Form {
31         username: "rustacean".to_owned(),
32         age: 28,
33     };
34
35     // If you uncomment this line, you'll get an error saying
36     // "multiple `get` found". Because, after all, there are multiple met
37     // named `get`.
38     // println!("{}", form.get());
39
40     let username = <Form as UsernameWidget>::get(&form);
41     assert_eq!("rustacean".to_owned(), username);
42     let age = <Form as AgeWidget>::get(&form);
43     assert_eq!(28, age);
44 }
```

See also:

[The Rust Programming Language chapter on Fully Qualified syntax](#)

macro_rules!

Rust provides a powerful macro system that allows metaprogramming. As you've seen in previous chapters, macros look like functions, except that their name ends with a bang `!`, but instead of generating a function call, macros are expanded into source code that gets compiled with the rest of the program. However, unlike macros in C and other languages, Rust macros are expanded into abstract syntax trees, rather than string preprocessing, so you don't get unexpected precedence bugs.

Macros are created using the `macro_rules!` macro.

```
1 // This is a simple macro named `say_hello`.
2 macro_rules! say_hello {
3     // `()` indicates that the macro takes no argument.
4     () => {
5         // The macro will expand into the contents of this block.
6         println!("Hello!")
7     };
8 }
9
10 fn main() {
11     // This call will expand into `println!("Hello")`
12     say_hello!()
13 }
```

So why are macros useful?

1. Don't repeat yourself. There are many cases where you may need similar functionality in multiple places but with different types. Often, writing a macro is a useful way to avoid repeating code. (More on this later)
2. Domain-specific languages. Macros allow you to define special syntax for a specific purpose. (More on this later)
3. Variadic interfaces. Sometimes you want to define an interface that takes a variable number of arguments. An example is `println!` which could take any number of arguments, depending on the format string. (More on this later)

Syntax

In following subsections, we will show how to define macros in Rust. There are three basic ideas:

- [Patterns and Designators](#)
- [Overloading](#)
- [Repetition](#)

Designators

The arguments of a macro are prefixed by a dollar sign `$` and type annotated with a *designator*:

```

1 macro_rules! create_function {
2     // This macro takes an argument of designator `ident` and
3     // creates a function named `$func_name`.
4     // The `ident` designator is used for variable/function names.
5     ($func_name:ident) => {
6         fn $func_name() {
7             // The `stringify!` macro converts an `ident` into a string.
8             println!("You called {:?}()",
9                     stringify!($func_name));
10        }
11    };
12 }
13
14 // Create functions named `foo` and `bar` with the above macro.
15 create_function!(foo);
16 create_function!(bar);
17
18 macro_rules! print_result {
19     // This macro takes an expression of type `expr` and prints
20     // it as a string along with its result.
21     // The `expr` designator is used for expressions.
22     ($expression:expr) => {
23         // `stringify!` will convert the expression as it is into a str
24         println!("{:?} = {:?}",
25                 stringify!($expression),
26                 $expression);
27     };
28 }
29
30 fn main() {
31     foo();
32     bar();
33
34     print_result!(1u32 + 1);
35
36     // Recall that blocks are expressions too!
37     print_result!({
38         let x = 1u32;
39
40         x * x + 2 * x - 1
41     });
42 }

```

These are some of the available designators:

- `block`
- `expr` is used for expressions
- `ident` is used for variable/function names

- `item`
- `literal` is used for literal constants
- `pat` (*pattern*)
- `path`
- `stmt` (*statement*)
- `tt` (*token tree*)
- `ty` (*type*)
- `vis` (*visibility qualifier*)

For a complete list, see the [Rust Reference](#).

Overload

Macros can be overloaded to accept different combinations of arguments. In that regard, `macro_rules!` can work similarly to a match block:

```
1 // `test!` will compare `$left` and `$right`
2 // in different ways depending on how you invoke it:
3 macro_rules! test {
4     // Arguments don't need to be separated by a comma.
5     // Any template can be used!
6     ($left:expr; and $right:expr) => {
7         println!("{:?} and {:?} is {:?}",
8             stringify!($left),
9             stringify!($right),
10            $left && $right)
11     };
12     // ^ each arm must end with a semicolon.
13     ($left:expr; or $right:expr) => {
14         println!("{:?} or {:?} is {:?}",
15             stringify!($left),
16             stringify!($right),
17            $left || $right)
18     };
19 }
20
21 fn main() {
22     test!(1i32 + 1 == 2i32; and 2i32 * 2 == 4i32);
23     test!(true; or false);
24 }
```

Repeat

Macros can use `+` in the argument list to indicate that an argument may repeat at least once, or `*`, to indicate that the argument may repeat zero or more times.

In the following example, surrounding the matcher with `$(...),+` will match one or more expression, separated by commas. Also note that the semicolon is optional on the last case.

```
1 // `find_min!` will calculate the minimum of any number of arguments.
2 macro_rules! find_min {
3     // Base case:
4     ($x:expr) => ($x);
5     // `$x` followed by at least one `$y`,`
6     ($x:expr, $($y:expr),+) => (
7         // Call `find_min!` on the tail `$y`
8         std::cmp::min($x, find_min!($($y),+))
9     )
10 }
11
12 fn main() {
13     println!("{}", find_min!(1));
14     println!("{}", find_min!(1 + 2, 2));
15     println!("{}", find_min!(5, 2 * 3, 4));
16 }
```

DRY (Don't Repeat Yourself)

Macros allow writing DRY code by factoring out the common parts of functions and/or test suites. Here is an example that implements and tests the `+=`, `*=` and `-=` operators on `Vec<T>`:

```

1 use std::ops::{Add, Mul, Sub};
2
3 macro_rules! assert_equal_len {
4     // The `tt` (token tree) designator is used for
5     // operators and tokens.
6     ($a:expr, $b:expr, $func:ident, $op:tt) => {
7         assert!($a.len() == $b.len(),
8             "{:?}: dimension mismatch: {:?} {:?} {:?}",
9             stringify!($func),
10             ($a.len(),),
11             stringify!($op),
12             ($b.len(),));
13     };
14 }
15
16 macro_rules! op {
17     ($func:ident, $bound:ident, $op:tt, $method:ident) => {
18         fn $func<T: $bound<T, Output=T> + Copy>(xs: &mut Vec<T>, ys: &Vec
19             assert_equal_len!(xs, ys, $func, $op);
20
21         for (x, y) in xs.iter_mut().zip(ys.iter()) {
22             *x = $bound::$method(*x, *y);
23             // *x = x.$method(*y);
24         }
25     }
26 };
27 }
28
29 // Implement `add_assign`, `mul_assign`, and `sub_assign` functions.
30 op!(add_assign, Add, +=, add);
31 op!(mul_assign, Mul, *=, mul);
32 op!(sub_assign, Sub, -=, sub);
33
34 mod test {
35     use std::iter;
36     macro_rules! test {
37         ($func:ident, $x:expr, $y:expr, $z:expr) => {
38             #[test]
39             fn $func() {
40                 for size in 0usize..10 {
41                     let mut x: Vec<_> = iter::repeat($x).take(size).collect();
42                     let y: Vec<_> = iter::repeat($y).take(size).collect();
43                     let z: Vec<_> = iter::repeat($z).take(size).collect();
44
45                     super::$func(&mut x, &y);
46
47                     assert_eq!(x, z);
48                 }
49             }
50         };
51     }
52
53     // Test `add_assign`, `mul_assign`, and `sub_assign`.
54     test!(add_assign, 1u32, 2u32, 3u32);
55     test!(mul_assign, 2u32, 3u32, 6u32);
56     test!(sub_assign, 3u32, 2u32, 1u32);
57 }

```



```
$ rustc --test dry.rs && ./dry
running 3 tests
test test::mul_assign ... ok
test test::add_assign ... ok
test test::sub_assign ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured
```

Domain Specific Languages (DSLs)

A DSL is a mini "language" embedded in a Rust macro. It is completely valid Rust because the macro system expands into normal Rust constructs, but it looks like a small language. This allows you to define concise or intuitive syntax for some special functionality (within bounds).

Suppose that I want to define a little calculator API. I would like to supply an expression and have the output printed to console.

```
1 macro_rules! calculate {
2     (eval $e:expr) => {
3         {
4             let val: usize = $e; // Force types to be integers
5             println!("{}", stringify!{$e}, val);
6         }
7     };
8 }
9
10 fn main() {
11     calculate! {
12         eval 1 + 2 // hehehe `eval` is _not_ a Rust keyword!
13     }
14
15     calculate! {
16         eval (1 + 2) * (3 / 4)
17     }
18 }
```

Output:

```
1 + 2 = 3
(1 + 2) * (3 / 4) = 0
```

This was a very simple example, but much more complex interfaces have been developed, such as [lazy_static](#) or [clap](#).

Also, note the two pairs of braces in the macro. The outer ones are part of the syntax of `macro_rules!`, in addition to `()` or `[]`.

Variadic Interfaces

A *variadic* interface takes an arbitrary number of arguments. For example, `println!` can take an arbitrary number of arguments, as determined by the format string.

We can extend our `calculate!` macro from the previous section to be variadic:

```
1 macro_rules! calculate {
2     // The pattern for a single `eval`
3     (eval $e:expr) => {
4         {
5             let val: usize = $e; // Force types to be integers
6             println!("{}", stringify!{$e}, val);
7         }
8     };
9
10    // Decompose multiple `eval`s recursively
11    (eval $e:expr, $(eval $es:expr),+) => {{
12        calculate! { eval $e }
13        calculate! { $(eval $es),+ }
14    }};
15 }
16
17 fn main() {
18     calculate! { // Look ma! Variadic `calculate`!
19         eval 1 + 2,
20         eval 3 + 4,
21         eval (2 * 3) + 1
22     }
23 }
```

Output:

```
1 + 2 = 3
3 + 4 = 7
(2 * 3) + 1 = 7
```

Error handling

Error handling is the process of handling the possibility of failure. For example, failing to read a file and then continuing to use that *bad* input would clearly be problematic. Noticing and explicitly managing those errors saves the rest of the program from various pitfalls.

There are various ways to deal with errors in Rust, which are described in the following subchapters. They all have more or less subtle differences and different use cases. As a rule of thumb:

An explicit `panic` is mainly useful for tests and dealing with unrecoverable errors. For prototyping it can be useful, for example when dealing with functions that haven't been implemented yet, but in those cases the more descriptive `unimplemented` is better. In tests `panic` is a reasonable way to explicitly fail.

The `option` type is for when a value is optional or when the lack of a value is not an error condition. For example the parent of a directory - `/` and `c:` don't have one. When dealing with `option`s, `unwrap` is fine for prototyping and cases where it's absolutely certain that there is guaranteed to be a value. However `expect` is more useful since it lets you specify an error message in case something goes wrong anyway.

When there is a chance that things do go wrong and the caller has to deal with the problem, use `Result`. You can `unwrap` and `expect` them as well (please don't do that unless it's a test or quick prototype).

For a more rigorous discussion of error handling, refer to the error handling section in the [official book](#).

panic

The simplest error handling mechanism we will see is `panic`. It prints an error message, starts unwinding the stack, and usually exits the program. Here, we explicitly call `panic` on our error condition:

```
1 fn drink(beverage: &str) {  
2     // You shouldn't drink too much sugary beverages.  
3     if beverage == "lemonade" { panic!("AAAaaaaa!!!"); }  
4  
5     println!("Some refreshing {} is all I need.", beverage);  
6 }  
7  
8 fn main() {  
9     drink("water");  
10    drink("lemonade");  
11    drink("still water");  
12 }
```

The first call to `drink` works. The second panics and thus the third is never called.

abort and unwind

The previous section illustrates the error handling mechanism `panic`. Different code paths can be conditionally compiled based on the panic setting. The current values available are `unwind` and `abort`.

Building on the prior lemonade example, we explicitly use the panic strategy to exercise different lines of code.

```
1 |  
2 fn drink(beverage: &str) {  
3     // You shouldn't drink too much sugary beverages.  
4     if beverage == "lemonade" {  
5         if cfg!(panic="abort"){ println!("This is not your party. Run!!!!  
6             else{ println!("Spit it out!!!!");}  
7     }  
8     else{ println!("Some refreshing {} is all I need.", beverage); }  
9 }  
10  
11 fn main() {  
12     drink("water");  
13     drink("lemonade");  
14 }
```

Here is another example focusing on rewriting `drink()` and explicitly use the `unwind` keyword.

```
1 |  
2 #[cfg(panic = "unwind")]  
3 fn ah(){ println!("Spit it out!!!!");}  
4  
5 #[cfg(not(panic="unwind"))]  
6 fn ah(){ println!("This is not your party. Run!!!!");}  
7  
8 fn drink(beverage: &str){  
9     if beverage == "lemonade"{ ah();}  
10    else{println!("Some refreshing {} is all I need.", beverage);}  
11 }  
12  
13 fn main() {  
14     drink("water");  
15     drink("lemonade");  
16 }
```

The panic strategy can be set from the command line by using `abort` or `unwind`.

```
rustc lemonade.rs -C panic=abort
```

Option & unwrap

In the last example, we showed that we can induce program failure at will. We told our program to `panic` if we drink a sugary lemonade. But what if we expect *some* drink but don't receive one? That case would be just as bad, so it needs to be handled!

We *could* test this against the null string (`""`) as we do with a lemonade. Since we're using Rust, let's instead have the compiler point out cases where there's no drink.

An `enum` called `Option<T>` in the `std` library is used when absence is a possibility. It manifests itself as one of two "options":

- `Some(T)` : An element of type `T` was found
- `None` : No element was found

These cases can either be explicitly handled via `match` or implicitly with `unwrap`. Implicit handling will either return the inner element or `panic`.

Note that it's possible to manually customize `panic` with `expect`, but `unwrap` otherwise leaves us with a less meaningful output than explicit handling. In the following example, explicit handling yields a more controlled result while retaining the option to `panic` if desired.

```
1 // The adult has seen it all, and can handle any drink well.
2 // All drinks are handled explicitly using `match`.
3 fn give_adult(drink: Option<&str>) {
4     // Specify a course of action for each case.
5     match drink {
6         Some("lemonade") => println!("Yuck! Too sugary."),
7         Some(inner)     => println!("{}",? How nice.", inner),
8         None             => println!("No drink? Oh well."),
9     }
10 }
11
12 // Others will `panic` before drinking sugary drinks.
13 // All drinks are handled implicitly using `unwrap`.
14 fn drink(drink: Option<&str>) {
15     // `unwrap` returns a `panic` when it receives a `None`.
16     let inside = drink.unwrap();
17     if inside == "lemonade" { panic!("AAaaaaa!!!!"); }
18
19     println!("I love {}s!!!!", inside);
20 }
21
22 fn main() {
23     let water = Some("water");
24     let lemonade = Some("lemonade");
25     let void = None;
26
27     give_adult(water);
28     give_adult(lemonade);
29     give_adult(void);
30
31     let coffee = Some("coffee");
32     let nothing = None;
33
34     drink(coffee);
35     drink(nothing);
36 }
```


Unpacking options with ?

You can unpack `Option`s by using `match` statements, but it's often easier to use the `?` operator. If `x` is an `Option`, then evaluating `x?` will return the underlying value if `x` is `Some`, otherwise it will terminate whatever function is being executed and return `None`.

```
1 fn next_birthday(current_age: Option<u8>) -> Option<String> {
2     // If `current_age` is `None`, this returns `None`.
3     // If `current_age` is `Some`, the inner `u8` gets assigned to `next_a
4     let next_age: u8 = current_age? + 1;
5     Some(format!("Next year I will be {}", next_age))
6 }
```

You can chain many `?`s together to make your code much more readable.

```
1 struct Person {
2     job: Option<Job>,
3 }
4
5 #[derive(Clone, Copy)]
6 struct Job {
7     phone_number: Option<PhoneNumber>,
8 }
9
10 #[derive(Clone, Copy)]
11 struct PhoneNumber {
12     area_code: Option<u8>,
13     number: u32,
14 }
15
16 impl Person {
17
18     // Gets the area code of the phone number of the person's job, if it
19     fn work_phone_area_code(&self) -> Option<u8> {
20         // This would need many nested `match` statements without the `?`
21         // It would take a lot more code - try writing it yourself and se
22         // is easier.
23         self.job?.phone_number?.area_code
24     }
25 }
26
27 fn main() {
28     let p = Person {
29         job: Some(Job {
30             phone_number: Some(PhoneNumber {
31                 area_code: Some(61),
32                 number: 439222222,
33             }),
34         }),
35     };
36
37     assert_eq!(p.work_phone_area_code(), Some(61));
38 }
```

Combinators: map

`match` is a valid method for handling `Option`s. However, you may eventually find heavy usage tedious, especially with operations only valid with an input. In these cases, [combinators](#) can be used to manage control flow in a modular fashion.

`Option` has a built in method called `map()`, a combinator for the simple mapping of `Some -> Some` and `None -> None`. Multiple `map()` calls can be chained together for even more flexibility.

In the following example, `process()` replaces all functions previous to it while staying compact.

```
1  #![allow(dead_code)]
2
3  #[derive(Debug)] enum Food { Apple, Carrot, Potato }
4
5  #[derive(Debug)] struct Peeled(Food);
6  #[derive(Debug)] struct Chopped(Food);
7  #[derive(Debug)] struct Cooked(Food);
8
9  // Peeling food. If there isn't any, then return `None`.
10 // Otherwise, return the peeled food.
11 fn peel(food: Option<Food>) -> Option<Peeled> {
12     match food {
13         Some(food) => Some(Peeled(food)),
14         None       => None,
15     }
16 }
17
18 // Chopping food. If there isn't any, then return `None`.
19 // Otherwise, return the chopped food.
20 fn chop(peeled: Option<Peeled>) -> Option<Chopped> {
21     match peeled {
22         Some(Peeled(food)) => Some(Chopped(food)),
23         None               => None,
24     }
25 }
26
27 // Cooking food. Here, we showcase `map()` instead of `match` for case ha
28 fn cook(chopped: Option<Chopped>) -> Option<Cooked> {
29     chopped.map(|Chopped(food)| Cooked(food))
30 }
31
32 // A function to peel, chop, and cook food all in sequence.
33 // We chain multiple uses of `map()` to simplify the code.
34 fn process(food: Option<Food>) -> Option<Cooked> {
35     food.map(|f| Peeled(f))
36         .map(|Peeled(f)| Chopped(f))
37         .map(|Chopped(f)| Cooked(f))
38 }
39
40 // Check whether there's food or not before trying to eat it!
41 fn eat(food: Option<Cooked>) {
42     match food {
43         Some(food) => println!("Mmm. I love {:?}", food),
44         None       => println!("Oh no! It wasn't edible."),
45     }
46 }
47
48 fn main() {
49     let apple = Some(Food::Apple);
50     let carrot = Some(Food::Carrot);
51     let potato = None;
52
53     let cooked_apple = cook(chop(peel(apple)));
54     let cooked_carrot = cook(chop(peel(carrot)));
55     // Let's try the simpler looking `process()` now.
56     let cooked_potato = process(potato);
57 }
```

See also:

[closures](#), [Option](#), [Option::map\(\)](#)

Combinators: `and_then`

`map()` was described as a chainable way to simplify `match` statements. However, using `map()` on a function that returns an `Option<T>` results in the nested `Option<Option<T>>`. Chaining multiple calls together can then become confusing. That's where another combinator called `and_then()`, known in some languages as `flatMap`, comes in.

`and_then()` calls its function input with the wrapped value and returns the result. If the `Option` is `None`, then it returns `None` instead.

In the following example, `cookable_v3()` results in an `Option<Food>`. Using `map()` instead of `and_then()` would have given an `Option<Option<Food>>`, which is an invalid type for `eat()`.

```
1  #![allow(dead_code)]
2
3  #[derive(Debug)] enum Food { CordonBleu, Steak, Sushi }
4  #[derive(Debug)] enum Day { Monday, Tuesday, Wednesday }
5
6  // We don't have the ingredients to make Sushi.
7  fn have_ingredients(food: Food) -> Option<Food> {
8      match food {
9          Food::Sushi => None,
10         _            => Some(food),
11     }
12 }
13
14 // We have the recipe for everything except Cordon Bleu.
15 fn have_recipe(food: Food) -> Option<Food> {
16     match food {
17         Food::CordonBleu => None,
18         _                 => Some(food),
19     }
20 }
21
22 // To make a dish, we need both the recipe and the ingredients.
23 // We can represent the logic with a chain of `match`es:
24 fn cookable_v1(food: Food) -> Option<Food> {
25     match have_recipe(food) {
26         None      => None,
27         Some(food) => have_ingredients(food),
28     }
29 }
30
31 // This can conveniently be rewritten more compactly with `and_then()`:
32 fn cookable_v3(food: Food) -> Option<Food> {
33     have_recipe(food).and_then(have_ingredients)
34 }
35
36 // Otherwise we'd need to `flatten()` an `Option<Option<Food>>`
37 // to get an `Option<Food>`:
38 fn cookable_v2(food: Food) -> Option<Food> {
39     have_recipe(food).map(have_ingredients).flatten()
40 }
41
42 fn eat(food: Food, day: Day) {
43     match cookable_v3(food) {
44         Some(food) => println!("Yay! On {:?} we get to eat {:?}.", day, food),
45         None       => println!("Oh no. We don't get to eat on {:?}?", day),
46     }
47 }
48
49 fn main() {
50     let (cordon_bleu, steak, sushi) = (Food::CordonBleu, Food::Steak, Food::Sushi);
51
52     eat(cordon_bleu, Day::Monday);
53     eat(steak, Day::Tuesday);
54     eat(sushi, Day::Wednesday);
55 }
```

See also:

[closures](#), [Option](#), [Option::and_then\(\)](#), and [Option::flatten\(\)](#)

Unpacking options and defaults

There is more than one way to unpack an `Option` and fall back on a default if it is `None`. To choose the one that meets our needs, we need to consider the following:

- do we need eager or lazy evaluation?
- do we need to keep the original empty value intact, or modify it in place?

`or()` is chainable, evaluates eagerly, keeps empty value intact

`or()` is chainable and eagerly evaluates its argument, as is shown in the following example. Note that because `or`'s arguments are evaluated eagerly, the variable passed to `or` is moved.

```
1  #[derive(Debug)]
2  enum Fruit { Apple, Orange, Banana, Kiwi, Lemon }
3
4  fn main() {
5      let apple = Some(Fruit::Apple);
6      let orange = Some(Fruit::Orange);
7      let no_fruit: Option<Fruit> = None;
8
9      let first_available_fruit = no_fruit.or(orange).or(apple);
10     println!("first_available_fruit: {:?}", first_available_fruit);
11     // first_available_fruit: Some(Orange)
12
13     // `or` moves its argument.
14     // In the example above, `or(orange)` returned a `Some`, so `or(apple)`
15     // But the variable named `apple` has been moved regardless, and cannot
16     // println!("Variable apple was moved, so this line won't compile: {::
17     // TODO: uncomment the line above to see the compiler error
18 }
```

`or_else()` is chainable, evaluates lazily, keeps empty value intact

Another alternative is to use `or_else`, which is also chainable, and evaluates lazily, as is shown in the following example:


```
1  #[derive(Debug)]
2  enum Fruit { Apple, Orange, Banana, Kiwi, Lemon }
3
4  fn main() {
5      let apple = Some(Fruit::Apple);
6      let no_fruit: Option<Fruit> = None;
7      let get_kiwi_as_fallback = || {
8          println!("Providing kiwi as fallback");
9          Some(Fruit::Kiwi)
10     };
11     let get_lemon_as_fallback = || {
12         println!("Providing lemon as fallback");
13         Some(Fruit::Lemon)
14     };
15
16     let first_available_fruit = no_fruit
17         .or_else(get_kiwi_as_fallback)
18         .or_else(get_lemon_as_fallback);
19     println!("first_available_fruit: {:?}", first_available_fruit);
20     // Providing kiwi as fallback
21     // first_available_fruit: Some(Kiwi)
22 }
```

get_or_insert() evaluates eagerly, modifies empty value in place

To make sure that an `Option` contains a value, we can use `get_or_insert` to modify it in place with a fallback value, as is shown in the following example. Note that `get_or_insert` eagerly evaluates its parameter, so variable `apple` is moved:

```
1  #[derive(Debug)]
2  enum Fruit { Apple, Orange, Banana, Kiwi, Lemon }
3
4  fn main() {
5      let mut my_fruit: Option<Fruit> = None;
6      let apple = Fruit::Apple;
7      let first_available_fruit = my_fruit.get_or_insert(apple);
8      println!("first_available_fruit is: {:?}", first_available_fruit);
9      println!("my_fruit is: {:?}", my_fruit);
10     // first_available_fruit is: Apple
11     // my_fruit is: Some(Apple)
12     //println!("Variable named `apple` is moved: {:?}", apple);
13     // TODO: uncomment the line above to see the compiler error
14 }
```

get_or_insert_with() evaluates lazily, modifies empty value in place

Instead of explicitly providing a value to fall back on, we can pass a closure to `get_or_insert_with`, as follows:

```
1  #[derive(Debug)]
2  enum Fruit { Apple, Orange, Banana, Kiwi, Lemon }
3
4  fn main() {
5      let mut my_fruit: Option<Fruit> = None;
6      let get_lemon_as_fallback = || {
7          println!("Providing lemon as fallback");
8          Fruit::Lemon
9      };
10     let first_available_fruit = my_fruit
11         .get_or_insert_with(get_lemon_as_fallback);
12     println!("first_available_fruit is: {:?}", first_available_fruit);
13     println!("my_fruit is: {:?}", my_fruit);
14     // Providing lemon as fallback
15     // first_available_fruit is: Lemon
16     // my_fruit is: Some(Lemon)
17
18     // If the Option has a value, it is left unchanged, and the closure i
19     let mut my_apple = Some(Fruit::Apple);
20     let should_be_apple = my_apple.get_or_insert_with(get_lemon_as_fallba
21     println!("should_be_apple is: {:?}", should_be_apple);
22     println!("my_apple is unchanged: {:?}", my_apple);
23     // The output is a follows. Note that the closure `get_lemon_as_fallb
24     // should_be_apple is: Apple
25     // my_apple is unchanged: Some(Apple)
26 }
```

See also:

[closures](#), [get_or_insert](#), [get_or_insert_with](#), [moved variables](#), [or](#), [or_else](#)

Result

`Result` is a richer version of the `Option` type that describes possible *error* instead of possible *absence*.

That is, `Result<T, E>` could have one of two outcomes:

- `Ok(T)` : An element `T` was found
- `Err(E)` : An error was found with element `E`

By convention, the expected outcome is `Ok` while the unexpected outcome is `Err`.

Like `Option`, `Result` has many methods associated with it. `unwrap()`, for example, either yields the element `T` or `panic`s. For case handling, there are many combinators between `Result` and `Option` that overlap.

In working with Rust, you will likely encounter methods that return the `Result` type, such as the `parse()` method. It might not always be possible to parse a string into the other type, so `parse()` returns a `Result` indicating possible failure.

Let's see what happens when we successfully and unsuccessfully `parse()` a string:

```
1 fn multiply(first_number_str: &str, second_number_str: &str) -> i32 {
2     // Let's try using `unwrap()` to get the number out. Will it bite us?
3     let first_number = first_number_str.parse::<i32>().unwrap();
4     let second_number = second_number_str.parse::<i32>().unwrap();
5     first_number * second_number
6 }
7
8 fn main() {
9     let twenty = multiply("10", "2");
10    println!("double is {}", twenty);
11
12    let tt = multiply("t", "2");
13    println!("double is {}", tt);
14 }
```

In the unsuccessful case, `parse()` leaves us with an error for `unwrap()` to `panic` on. Additionally, the `panic` exits our program and provides an unpleasant error message.

To improve the quality of our error message, we should be more specific about the return type and consider explicitly handling the error.

Using Result in main

The `Result` type can also be the return type of the `main` function if specified explicitly.

Typically the `main` function will be of the form:

```
fn main() {  
    println!("Hello World!");  
}
```

However `main` is also able to have a return type of `Result`. If an error occurs within the `main` function it will return an error code and print a debug representation of the error (using the `Debug` trait). The following example shows such a scenario and touches on aspects covered in [the following section](#).

```
1 use std::num::ParseIntError;  
2  
3 fn main() -> Result<(), ParseIntError> {  
4     let number_str = "10";  
5     let number = match number_str.parse::<i32>() {  
6         Ok(number) => number,  
7         Err(e) => return Err(e),  
8     };  
9     println!("{}", number);  
10    Ok()  
11 }
```

map for Result

Panicking in the previous example's `multiply` does not make for robust code. Generally, we want to return the error to the caller so it can decide what is the right way to respond to errors.

We first need to know what kind of error type we are dealing with. To determine the `Err` type, we look to `parse()`, which is implemented with the `FromStr` trait for `i32`. As a result, the `Err` type is specified as `ParseIntError`.

In the example below, the straightforward `match` statement leads to code that is overall more cumbersome.

```

1 use std::num::ParseIntError;
2
3 // With the return type rewritten, we use pattern matching without `unwr
4 fn multiply(first_number_str: &str, second_number_str: &str) -> Result<i3
5     match first_number_str.parse::<i32>() {
6         Ok(first_number) => {
7             match second_number_str.parse::<i32>() {
8                 Ok(second_number) => {
9                     Ok(first_number * second_number)
10                },
11                Err(e) => Err(e),
12            }
13        },
14        Err(e) => Err(e),
15    }
16 }
17
18 fn print(result: Result<i32, ParseIntError>) {
19     match result {
20         Ok(n) => println!("n is {}", n),
21         Err(e) => println!("Error: {}", e),
22     }
23 }
24
25 fn main() {
26     // This still presents a reasonable answer.
27     let twenty = multiply("10", "2");
28     print(twenty);
29
30     // The following now provides a much more helpful error message.
31     let tt = multiply("t", "2");
32     print(tt);
33 }
```

Luckily, `Option`'s `map`, `and_then`, and many other combinators are also implemented for `Result`. `Result` contains a complete listing.

```
1 use std::num::ParseIntError;
2
3 // As with `Option`, we can use combinators such as `map()`.
4 // This function is otherwise identical to the one above and reads:
5 // Multiply if both values can be parsed from str, otherwise pass on the
6 fn multiply(first_number_str: &str, second_number_str: &str) -> Result<i32> {
7     first_number_str.parse::<i32>().and_then(|first_number| {
8         second_number_str.parse::<i32>().map(|second_number| first_number
9     })
10 }
11
12 fn print(result: Result<i32, ParseIntError>) {
13     match result {
14         Ok(n) => println!("n is {}", n),
15         Err(e) => println!("Error: {}", e),
16     }
17 }
18
19 fn main() {
20     // This still presents a reasonable answer.
21     let twenty = multiply("10", "2");
22     print(twenty);
23
24     // The following now provides a much more helpful error message.
25     let tt = multiply("t", "2");
26     print(tt);
27 }
```

aliases for Result

How about when we want to reuse a specific `Result` type many times? Recall that Rust allows us to create [aliases](#). Conveniently, we can define one for the specific `Result` in question.

At a module level, creating aliases can be particularly helpful. Errors found in a specific module often have the same `Err` type, so a single alias can succinctly define *all* associated `Results`. This is so useful that the `std` library even supplies one:

`io::Result!`

Here's a quick example to show off the syntax:

```
1 use std::num::ParseIntError;
2
3 // Define a generic alias for a `Result` with the error type `ParseIntError`
4 type AliasedResult<T> = Result<T, ParseIntError>;
5
6 // Use the above alias to refer to our specific `Result` type.
7 fn multiply(first_number_str: &str, second_number_str: &str) -> AliasedResult<i32> {
8     first_number_str.parse::<i32>().and_then(|first_number| {
9         second_number_str.parse::<i32>().map(|second_number| first_number
10     })
11 }
12
13 // Here, the alias again allows us to save some space.
14 fn print(result: AliasedResult<i32>) {
15     match result {
16         Ok(n) => println!("n is {}", n),
17         Err(e) => println!("Error: {}", e),
18     }
19 }
20
21 fn main() {
22     print(multiply("10", "2"));
23     print(multiply("t", "2"));
24 }
```

See also:

`io::Result`

Early returns

In the previous example, we explicitly handled the errors using combinators. Another way to deal with this case analysis is to use a combination of `match` statements and *early returns*.

That is, we can simply stop executing the function and return the error if one occurs. For some, this form of code can be easier to both read and write. Consider this version of the previous example, rewritten using early returns:

```
1 use std::num::ParseIntError;
2
3 fn multiply(first_number_str: &str, second_number_str: &str) -> Result<i32,
4     let first_number = match first_number_str.parse::<i32>() {
5         Ok(first_number) => first_number,
6         Err(e) => return Err(e),
7     };
8
9     let second_number = match second_number_str.parse::<i32>() {
10         Ok(second_number) => second_number,
11         Err(e) => return Err(e),
12     };
13
14     Ok(first_number * second_number)
15 }
16
17 fn print(result: Result<i32, ParseIntError>) {
18     match result {
19         Ok(n) => println!("n is {}", n),
20         Err(e) => println!("Error: {}", e),
21     }
22 }
23
24 fn main() {
25     print(multiply("10", "2"));
26     print(multiply("t", "2"));
27 }
```

At this point, we've learned to explicitly handle errors using combinators and early returns. While we generally want to avoid panicking, explicitly handling all of our errors is cumbersome.

In the next section, we'll introduce `?` for the cases where we simply need to `unwrap` without possibly inducing `panic`.

Introducing ?

Sometimes we just want the simplicity of `unwrap` without the possibility of a `panic`. Until now, `unwrap` has forced us to nest deeper and deeper when what we really wanted was to get the variable *out*. This is exactly the purpose of `?`.

Upon finding an `Err`, there are two valid actions to take:

1. `panic!` which we already decided to try to avoid if possible
2. `return` because an `Err` means it cannot be handled

`?` is *almost*¹ exactly equivalent to an `unwrap` which `return s` instead of `panic` king on `Err s`. Let's see how we can simplify the earlier example that used combinators:

```

1 use std::num::ParseIntError;
2
3 fn multiply(first_number_str: &str, second_number_str: &str) -> Result<i32,
4     let first_number = first_number_str.parse::<i32>()?;
5     let second_number = second_number_str.parse::<i32>()?;
6
7     Ok(first_number * second_number)
8 }
9
10 fn print(result: Result<i32, ParseIntError>) {
11     match result {
12         Ok(n) => println!("n is {}", n),
13         Err(e) => println!("Error: {}", e),
14     }
15 }
16
17 fn main() {
18     print(multiply("10", "2"));
19     print(multiply("t", "2"));
20 }
```

The `try!` macro

Before there was `?`, the same functionality was achieved with the `try!` macro. The `?` operator is now recommended, but you may still find `try!` when looking at older code. The same `multiply` function from the previous example would look like this using `try!`:

```
1 // To compile and run this example without errors, while using Cargo, cha
2 // of the `edition` field, in the `[package]` section of the `Cargo.toml`
3
4 use std::num::ParseIntError;
5
6 fn multiply(first_number_str: &str, second_number_str: &str) -> Result<i32,
7     let first_number = try!(first_number_str.parse::<i32>());
8     let second_number = try!(second_number_str.parse::<i32>());
9
10     Ok(first_number * second_number)
11 }
12
13 fn print(result: Result<i32, ParseIntError>) {
14     match result {
15         Ok(n) => println!("n is {}", n),
16         Err(e) => println!("Error: {}", e),
17     }
18 }
19
20 fn main() {
21     print(multiply("10", "2"));
22     print(multiply("t", "2"));
23 }
```

¹ See [re-enter ?](#) for more details.

Multiple error types

The previous examples have always been very convenient; `Result` s interact with other `Result` s and `Option` s interact with other `Option` s.

Sometimes an `Option` needs to interact with a `Result`, or a `Result<T, Error1>` needs to interact with a `Result<T, Error2>`. In those cases, we want to manage our different error types in a way that makes them composable and easy to interact with.

In the following code, two instances of `unwrap` generate different error types.

`Vec::first` returns an `Option`, while `parse::<i32>()` returns a `Result<i32, ParseIntError>`:

```
1 fn double_first(vec: Vec<&str>) -> i32 {
2     let first = vec.first().unwrap(); // Generate error 1
3     2 * first.parse::<i32>().unwrap() // Generate error 2
4 }
5
6 fn main() {
7     let numbers = vec!["42", "93", "18"];
8     let empty = vec![];
9     let strings = vec!["tofu", "93", "18"];
10
11     println!("The first doubled is {}", double_first(numbers));
12
13     println!("The first doubled is {}", double_first(empty));
14     // Error 1: the input vector is empty
15
16     println!("The first doubled is {}", double_first(strings));
17     // Error 2: the element doesn't parse to a number
18 }
```

Over the next sections, we'll see several strategies for handling these kind of problems.

Pulling Results out of Options

The most basic way of handling mixed error types is to just embed them in each other.

```

1 use std::num::ParseIntError;
2
3 fn double_first(vec: Vec<&str>) -> Option<Result<i32, ParseIntError>> {
4     vec.first().map(|first| {
5         first.parse::<i32>().map(|n| 2 * n)
6     })
7 }
8
9 fn main() {
10     let numbers = vec!["42", "93", "18"];
11     let empty = vec![];
12     let strings = vec!["tofu", "93", "18"];
13
14     println!("The first doubled is {:?}", double_first(numbers));
15
16     println!("The first doubled is {:?}", double_first(empty));
17     // Error 1: the input vector is empty
18
19     println!("The first doubled is {:?}", double_first(strings));
20     // Error 2: the element doesn't parse to a number
21 }
```

There are times when we'll want to stop processing on errors (like with `?`) but keep going when the `Option` is `None`. A couple of combinators come in handy to swap the `Result` and `Option`.

```

1 use std::num::ParseIntError;
2
3 fn double_first(vec: Vec<&str>) -> Result<Option<i32>, ParseIntError> {
4     let opt = vec.first().map(|first| {
5         first.parse::<i32>().map(|n| 2 * n)
6     });
7
8     opt.map_or(Ok(None), |r| r.map(Some))
9 }
10
11 fn main() {
12     let numbers = vec!["42", "93", "18"];
13     let empty = vec![];
14     let strings = vec!["tofu", "93", "18"];
15
16     println!("The first doubled is {:?}", double_first(numbers));
17     println!("The first doubled is {:?}", double_first(empty));
18     println!("The first doubled is {:?}", double_first(strings));
19 }
```

Defining an error type

Sometimes it simplifies the code to mask all of the different errors with a single type of error. We'll show this with a custom error.

Rust allows us to define our own error types. In general, a "good" error type:

- Represents different errors with the same type
- Presents nice error messages to the user
- Is easy to compare with other types
 - Good: `Err(EmptyVec)`
 - Bad: `Err("Please use a vector with at least one element".to_owned())`
- Can hold information about the error
 - Good: `Err(BadChar(c, position))`
 - Bad: `Err("+ cannot be used here".to_owned())`
- Composes well with other errors

```
1 use std::fmt;
2
3 type Result<T> = std::result::Result<T, DoubleError>;
4
5 // Define our error types. These may be customized for our error handling
6 // Now we will be able to write our own errors, defer to an underlying er
7 // implementation, or do something in between.
8 #[derive(Debug, Clone)]
9 struct DoubleError;
10
11 // Generation of an error is completely separate from how it is displayed
12 // There's no need to be concerned about cluttering complex logic with th
13 //
14 // Note that we don't store any extra info about the errors. This means w
15 // which string failed to parse without modifying our types to carry that
16 impl fmt::Display for DoubleError {
17     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
18         write!(f, "invalid first item to double")
19     }
20 }
21
22 fn double_first(vec: Vec<&str>) -> Result<i32> {
23     vec.first()
24         // Change the error to our new type.
25         .ok_or(DoubleError)
26         .and_then(|s| {
27             s.parse::<i32>()
28                 // Update to the new error type here also.
29                 .map_err(|_| DoubleError)
30                 .map(|i| 2 * i)
31         })
32 }
33
34 fn print(result: Result<i32>) {
35     match result {
36         Ok(n) => println!("The first doubled is {}", n),
37         Err(e) => println!("Error: {}", e),
38     }
39 }
40
41 fn main() {
42     let numbers = vec!["42", "93", "18"];
43     let empty = vec![];
44     let strings = vec!["tofu", "93", "18"];
45
46     print(double_first(numbers));
47     print(double_first(empty));
48     print(double_first(strings));
49 }
```

Boxing errors

A way to write simple code while preserving the original errors is to [Box](#) them. The drawback is that the underlying error type is only known at runtime and not [statically determined](#).

The `stdlib` helps in boxing our errors by having `Box` implement conversion from any type that implements the `Error` trait into the trait object `Box<Error>`, via [From](#).

```

1 use std::error;
2 use std::fmt;
3
4 // Change the alias to `Box<error::Error>`.
5 type Result<T> = std::result::Result<T, Box<dyn error::Error>>;
6
7 #[derive(Debug, Clone)]
8 struct EmptyVec;
9
10 impl fmt::Display for EmptyVec {
11     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
12         write!(f, "invalid first item to double")
13     }
14 }
15
16 impl error::Error for EmptyVec {}
17
18 fn double_first(vec: Vec<&str>) -> Result<i32> {
19     vec.first()
20         .ok_or_else(|| EmptyVec.into()) // Converts to Box
21         .and_then(|s| {
22             s.parse::<i32>()
23                 .map_err(|e| e.into()) // Converts to Box
24                 .map(|i| 2 * i)
25         })
26 }
27
28 fn print(result: Result<i32>) {
29     match result {
30         Ok(n) => println!("The first doubled is {}", n),
31         Err(e) => println!("Error: {}", e),
32     }
33 }
34
35 fn main() {
36     let numbers = vec!["42", "93", "18"];
37     let empty = vec![];
38     let strings = vec!["tofu", "93", "18"];
39
40     print(double_first(numbers));
41     print(double_first(empty));
42     print(double_first(strings));
43 }

```

See also:

[Dynamic dispatch](#) and [Error trait](#)

Other uses of `?`

Notice in the previous example that our immediate reaction to calling `parse` is to `map` the error from a library error into a boxed error:

```
.and_then(|s| s.parse::<i32>())  
    .map_err(|e| e.into())
```

Since this is a simple and common operation, it would be convenient if it could be elided. Alas, because `and_then` is not sufficiently flexible, it cannot. However, we can instead use `?`.

`?` was previously explained as either `unwrap` or `return Err(err)`. This is only mostly true. It actually means `unwrap` or `return Err(From::from(err))`. Since `From::from` is a conversion utility between different types, this means that if you `?` where the error is convertible to the return type, it will convert automatically.

Here, we rewrite the previous example using `?`. As a result, the `map_err` will go away when `From::from` is implemented for our error type:

```

1 use std::error;
2 use std::fmt;
3
4 // Change the alias to `Box<dyn error::Error>`.
5 type Result<T> = std::result::Result<T, Box<dyn error::Error>>;
6
7 #[derive(Debug)]
8 struct EmptyVec;
9
10 impl fmt::Display for EmptyVec {
11     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
12         write!(f, "invalid first item to double")
13     }
14 }
15
16 impl error::Error for EmptyVec {}
17
18 // The same structure as before but rather than chain all `Results`
19 // and `Options` along, we `?` to get the inner value out immediately.
20 fn double_first(vec: Vec<&str>) -> Result<i32> {
21     let first = vec.first().ok_or(EmptyVec)?;
22     let parsed = first.parse::<i32>()?;
23     Ok(2 * parsed)
24 }
25
26 fn print(result: Result<i32>) {
27     match result {
28         Ok(n) => println!("The first doubled is {}", n),
29         Err(e) => println!("Error: {}", e),
30     }
31 }
32
33 fn main() {
34     let numbers = vec!["42", "93", "18"];
35     let empty = vec![];
36     let strings = vec!["tofu", "93", "18"];
37
38     print(double_first(numbers));
39     print(double_first(empty));
40     print(double_first(strings));
41 }

```

This is actually fairly clean now. Compared with the original `panic`, it is very similar to replacing the `unwrap` calls with `?` except that the return types are `Result`. As a result, they must be destructured at the top level.

See also:

[From::from](#) and `?`

Wrapping errors

An alternative to boxing errors is to wrap them in your own error type.

```

1 use std::error;
2 use std::error::Error;
3 use std::num::ParseIntError;
4 use std::fmt;
5
6 type Result<T> = std::result::Result<T, DoubleError>;
7
8 #[derive(Debug)]
9 enum DoubleError {
10     EmptyVec,
11     // We will defer to the parse error implementation for their error.
12     // Supplying extra info requires adding more data to the type.
13     Parse(ParseIntError),
14 }
15
16 impl fmt::Display for DoubleError {
17     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
18         match *self {
19             DoubleError::EmptyVec =>
20                 write!(f, "please use a vector with at least one element"
21                     // The wrapped error contains additional information and is a
22                     // via the source() method.
23             DoubleError::Parse(..) =>
24                 write!(f, "the provided string could not be parsed as int
25         }
26     }
27 }
28
29 impl error::Error for DoubleError {
30     fn source(&self) -> Option<&(dyn error::Error + 'static)> {
31         match *self {
32             DoubleError::EmptyVec => None,
33             // The cause is the underlying implementation error type. Is
34             // cast to the trait object `&error::Error`. This works becau
35             // underlying type already implements the `Error` trait.
36             DoubleError::Parse(ref e) => Some(e),
37         }
38     }
39 }
40
41 // Implement the conversion from `ParseIntError` to `DoubleError`.
42 // This will be automatically called by `?` if a `ParseIntError`
43 // needs to be converted into a `DoubleError`.
44 impl From<ParseIntError> for DoubleError {
45     fn from(err: ParseIntError) -> DoubleError {
46         DoubleError::Parse(err)
47     }
48 }
49
50 fn double_first(vec: Vec<&str>) -> Result<i32> {
51     let first = vec.first().ok_or(DoubleError::EmptyVec)?;
52     // Here we implicitly use the `ParseIntError` implementation of `From
53     // we defined above) in order to create a `DoubleError`.
54     let parsed = first.parse::<i32>()?;
55
56     Ok(2 * parsed)
57 }

```

This adds a bit more boilerplate for handling errors and might not be needed in all applications. There are some libraries that can take care of the boilerplate for you.

See also:

[From::from](#) and [Enums](#)

Iterating over Results

An `Iter::map` operation might fail, for example:

```
1 fn main() {
2     let strings = vec!["tofu", "93", "18"];
3     let numbers: Vec<_> = strings
4         .into_iter()
5         .map(|s| s.parse::<i32>())
6         .collect();
7     println!("Results: {:?}", numbers);
8 }
```

Let's step through strategies for handling this.

Ignore the failed items with `filter_map()`

`filter_map` calls a function and filters out the results that are `None`.

```
1 fn main() {
2     let strings = vec!["tofu", "93", "18"];
3     let numbers: Vec<_> = strings
4         .into_iter()
5         .filter_map(|s| s.parse::<i32>().ok())
6         .collect();
7     println!("Results: {:?}", numbers);
8 }
```

Collect the failed items with `map_err()` and `filter_map()`

`map_err` calls a function with the error, so by adding that to the previous `filter_map` solution we can save them off to the side while iterating.

```
1 fn main() {
2     let strings = vec!["42", "tofu", "93", "999", "18"];
3     let mut errors = vec![];
4     let numbers: Vec<_> = strings
5         .into_iter()
6         .map(|s| s.parse::<u8>())
7         .filter_map(|r| r.map_err(|e| errors.push(e)).ok())
8         .collect();
9     println!("Numbers: {:?}", numbers);
10    println!("Errors: {:?}", errors);
11 }
```

Fail the entire operation with `collect()`

`Result` implements `FromIterator` so that a vector of results (`Vec<Result<T, E>>`) can be turned into a result with a vector (`Result<Vec<T>, E>`). Once an `Result::Err` is found, the iteration will terminate.

```
1 fn main() {
2     let strings = vec!["tofu", "93", "18"];
3     let numbers: Result<Vec<_>, _> = strings
4         .into_iter()
5         .map(|s| s.parse::<i32>())
6         .collect();
7     println!("Results: {:?}", numbers);
8 }
```

This same technique can be used with `Option`.

Collect all valid values and failures with `partition()`

```
1 fn main() {
2     let strings = vec!["tofu", "93", "18"];
3     let (numbers, errors): (Vec<_>, Vec<_>) = strings
4         .into_iter()
5         .map(|s| s.parse::<i32>())
6         .partition(Result::is_ok);
7     println!("Numbers: {:?}", numbers);
8     println!("Errors: {:?}", errors);
9 }
```

When you look at the results, you'll note that everything is still wrapped in `Result`. A little more boilerplate is needed for this.

```
1 fn main() {
2     let strings = vec!["tofu", "93", "18"];
3     let (numbers, errors): (Vec<_>, Vec<_>) = strings
4         .into_iter()
5         .map(|s| s.parse::<i32>())
6         .partition(Result::is_ok);
7     let numbers: Vec<_> = numbers.into_iter().map(Result::unwrap).collect();
8     let errors: Vec<_> = errors.into_iter().map(Result::unwrap_err).collect();
9     println!("Numbers: {:?}", numbers);
10    println!("Errors: {:?}", errors);
11 }
```

Std library types

The `std` library provides many custom types which expands drastically on the `primitives`. Some of these include:

- growable `Strings` like: `"hello world"`
- growable vectors: `[1, 2, 3]`
- optional types: `Option<i32>`
- error handling types: `Result<i32, i32>`
- heap allocated pointers: `Box<i32>`

See also:

[primitives](#) and [the std library](#)

Box, stack and heap

All values in Rust are stack allocated by default. Values can be *boxed* (allocated on the heap) by creating a `Box<T>`. A box is a smart pointer to a heap allocated value of type `T`. When a box goes out of scope, its destructor is called, the inner object is destroyed, and the memory on the heap is freed.

Boxed values can be dereferenced using the `*` operator; this removes one layer of indirection.

```
1 use std::mem;
2
3 #[allow(dead_code)]
4 #[derive(Debug, Clone, Copy)]
5 struct Point {
6     x: f64,
7     y: f64,
8 }
9
10 // A Rectangle can be specified by where its top left and bottom right
11 // corners are in space
12 #[allow(dead_code)]
13 struct Rectangle {
14     top_left: Point,
15     bottom_right: Point,
16 }
17
18 fn origin() -> Point {
19     Point { x: 0.0, y: 0.0 }
20 }
21
22 fn boxed_origin() -> Box<Point> {
23     // Allocate this point on the heap, and return a pointer to it
24     Box::new(Point { x: 0.0, y: 0.0 })
25 }
26
27 fn main() {
28     // (all the type annotations are superfluous)
29     // Stack allocated variables
30     let point: Point = origin();
31     let rectangle: Rectangle = Rectangle {
32         top_left: origin(),
33         bottom_right: Point { x: 3.0, y: -4.0 }
34     };
35
36     // Heap allocated rectangle
37     let boxed_rectangle: Box<Rectangle> = Box::new(Rectangle {
38         top_left: origin(),
39         bottom_right: Point { x: 3.0, y: -4.0 },
40     });
41
42     // The output of functions can be boxed
43     let boxed_point: Box<Point> = Box::new(origin());
44
45     // Double indirection
46     let box_in_a_box: Box<Box<Point>> = Box::new(boxed_origin());
47
48     println!("Point occupies {} bytes on the stack",
49             mem::size_of_val(&point));
50     println!("Rectangle occupies {} bytes on the stack",
51             mem::size_of_val(&rectangle));
52
53     // box size == pointer size
54     println!("Boxed point occupies {} bytes on the stack",
55             mem::size_of_val(&boxed_point));
56     println!("Boxed rectangle occupies {} bytes on the stack",
57             mem::size_of_val(&boxed_rectangle));
```


Vectors

Vectors are re-sizable arrays. Like slices, their size is not known at compile time, but they can grow or shrink at any time. A vector is represented using 3 parameters:

- pointer to the data
- length
- capacity

The capacity indicates how much memory is reserved for the vector. The vector can grow as long as the length is smaller than the capacity. When this threshold needs to be surpassed, the vector is reallocated with a larger capacity.

```
1 fn main() {
2     // Iterators can be collected into vectors
3     let collected_iterator: Vec<i32> = (0..10).collect();
4     println!("Collected (0..10) into: {:?}", collected_iterator);
5
6     // The `vec!` macro can be used to initialize a vector
7     let mut xs = vec![1i32, 2, 3];
8     println!("Initial vector: {:?}", xs);
9
10    // Insert new element at the end of the vector
11    println!("Push 4 into the vector");
12    xs.push(4);
13    println!("Vector: {:?}", xs);
14
15    // Error! Immutable vectors can't grow
16    collected_iterator.push(0);
17    // FIXME ^ Comment out this line
18
19    // The `len` method yields the number of elements currently stored in
20    println!("Vector length: {}", xs.len());
21
22    // Indexing is done using the square brackets (indexing starts at 0)
23    println!("Second element: {}", xs[1]);
24
25    // `pop` removes the last element from the vector and returns it
26    println!("Pop last element: {:?}", xs.pop());
27
28    // Out of bounds indexing yields a panic
29    println!("Fourth element: {}", xs[3]);
30    // FIXME ^ Comment out this line
31
32    // `Vector`s can be easily iterated over
33    println!("Contents of xs:");
34    for x in xs.iter() {
35        println!("> {}", x);
36    }
37
38    // A `Vector` can also be iterated over while the iteration
39    // count is enumerated in a separate variable (`i`)
40    for (i, x) in xs.iter().enumerate() {
41        println!("In position {} we have value {}", i, x);
42    }
43
44    // Thanks to `iter_mut`, mutable `Vector`s can also be iterated
45    // over in a way that allows modifying each value
46    for x in xs.iter_mut() {
47        *x *= 3;
48    }
49    println!("Updated vector: {:?}", xs);
50 }
```

More `vec` methods can be found under the [std::vec](#) module

Strings

There are two types of strings in Rust: `String` and `&str`.

A `String` is stored as a vector of bytes (`Vec<u8>`), but guaranteed to always be a valid UTF-8 sequence. `String` is heap allocated, growable and not null terminated.

`&str` is a slice (`&[u8]`) that always points to a valid UTF-8 sequence, and can be used to view into a `String`, just like `&[T]` is a view into `Vec<T>`.

```

1 fn main() {
2     // (all the type annotations are superfluous)
3     // A reference to a string allocated in read only memory
4     let pangram: &'static str = "the quick brown fox jumps over the lazy dog";
5     println!("Pangram: {}", pangram);
6
7     // Iterate over words in reverse, no new string is allocated
8     println!("Words in reverse");
9     for word in pangram.split_whitespace().rev() {
10         println!("> {}", word);
11     }
12
13     // Copy chars into a vector, sort and remove duplicates
14     let mut chars: Vec<char> = pangram.chars().collect();
15     chars.sort();
16     chars.dedup();
17
18     // Create an empty and growable `String`
19     let mut string = String::new();
20     for c in chars {
21         // Insert a char at the end of string
22         string.push(c);
23         // Insert a string at the end of string
24         string.push_str(", ");
25     }
26
27     // The trimmed string is a slice to the original string, hence no new
28     // allocation is performed
29     let chars_to_trim: &[char] = &[' ', ','];
30     let trimmed_str: &str = string.trim_matches(chars_to_trim);
31     println!("Used characters: {}", trimmed_str);
32
33     // Heap allocate a string
34     let alice = String::from("I like dogs");
35     // Allocate new memory and store the modified string there
36     let bob: String = alice.replace("dog", "cat");
37
38     println!("Alice says: {}", alice);
39     println!("Bob says: {}", bob);
40 }

```

More `str` / `String` methods can be found under the [std::str](#) and [std::string](#) modules

Literals and escapes

There are multiple ways to write string literals with special characters in them. All result in a similar `&str` so it's best to use the form that is the most convenient to write. Similarly there are multiple ways to write byte string literals, which all result in `&[u8; N]`.

Generally special characters are escaped with a backslash character: `\`. This way you can add any character to your string, even unprintable ones and ones that you don't know how to type. If you want a literal backslash, escape it with another one: `\\`.

String or character literal delimiters occurring within a literal must be escaped: `"\"`, `'\'`.

```

1 fn main() {
2     // You can use escapes to write bytes by their hexadecimal values...
3     let byte_escape = "I'm writing \x52\x75\x73\x74!";
4     println!("What are you doing\x3F (\\x3F means ?) {}", byte_escape);
5
6     // ...or Unicode code points.
7     let unicode_codepoint = "\u{211D}";
8     let character_name = "\"DOUBLE-STRUCK CAPITAL R\"";
9
10    println!("Unicode character {} (U+211D) is called {}",
11             unicode_codepoint, character_name );
12
13
14    let long_string = "String literals
15                      can span multiple lines.
16                      The linebreak and indentation here ->\
17                      <- can be escaped too!";
18    println!("{}", long_string);
19 }
```

Sometimes there are just too many characters that need to be escaped or it's just much more convenient to write a string out as-is. This is where raw string literals come into play.

```

1 fn main() {
2     let raw_str = r"Escapes don't work here: \x3F \u{211D}";
3     println!("{}", raw_str);
4
5     // If you need quotes in a raw string, add a pair of #s
6     let quotes = r#"And then I said: "There is no escape!"#;
7     println!("{}", quotes);
8
9     // If you need "#" in your string, just use more #s in the delimiter.
10    // You can use up to 65535 #s.
11    let longer_delimiter = r####"A string with "#" in it. And even "##!"###
12    println!("{}", longer_delimiter);
13 }
```

Want a string that's not UTF-8? (Remember, `str` and `String` must be valid UTF-8). Or

maybe you want an array of bytes that's mostly text? Byte strings to the rescue!

```

1 use std::str;
2
3 fn main() {
4     // Note that this is not actually a `&str`
5     let bytestring: &[u8; 21] = b"this is a byte string";
6
7     // Byte arrays don't have the `Display` trait, so printing them is a
8     println!("A byte string: {:?}", bytestring);
9
10    // Byte strings can have byte escapes...
11    let escaped = b"\x52\x75\x73\x74 as bytes";
12    // ...but no unicode escapes
13    // let escaped = b"\u{211D} is not allowed";
14    println!("Some escaped bytes: {:?}", escaped);
15
16
17    // Raw byte strings work just like raw strings
18    let raw_bytestring = br"\u{211D} is not escaped here";
19    println!("{:?}", raw_bytestring);
20
21    // Converting a byte array to `str` can fail
22    if let Ok(my_str) = str::from_utf8(raw_bytestring) {
23        println!("And the same as text: '{}'", my_str);
24    }
25
26    let _quotes = br#"You can also use "fancier" formatting, \
27                  like with normal raw strings"#;
28
29    // Byte strings don't have to be UTF-8
30    let shift_jis = b"\x82\xe6\x82\xa8\x82\xb1\x82\xbb"; // "ようこそ" in
31
32    // But then they can't always be converted to `str`
33    match str::from_utf8(shift_jis) {
34        Ok(my_str) => println!("Conversion successful: '{}'", my_str),
35        Err(e) => println!("Conversion failed: {:?}", e),
36    };
37 }

```

For conversions between character encodings check out the [encoding](#) crate.

A more detailed listing of the ways to write string literals and escape characters is given in the ['Tokens' chapter](#) of the Rust Reference.

Option

Sometimes it's desirable to catch the failure of some parts of a program instead of calling `panic!`; this can be accomplished using the `Option` enum.

The `Option<T>` enum has two variants:

- `None`, to indicate failure or lack of value, and
- `Some(value)`, a tuple struct that wraps a `value` with type `T`.

```

1 // An integer division that doesn't `panic!`
2 fn checked_division(dividend: i32, divisor: i32) -> Option<i32> {
3     if divisor == 0 {
4         // Failure is represented as the `None` variant
5         None
6     } else {
7         // Result is wrapped in a `Some` variant
8         Some(dividend / divisor)
9     }
10 }
11
12 // This function handles a division that may not succeed
13 fn try_division(dividend: i32, divisor: i32) {
14     // `Option` values can be pattern matched, just like other enums
15     match checked_division(dividend, divisor) {
16         None => println!("{}", divisor, "failed!", dividend, divisor),
17         Some(quotient) => {
18             println!("{}", divisor, " = {}", dividend, divisor, quotient)
19         },
20     }
21 }
22
23 fn main() {
24     try_division(4, 2);
25     try_division(1, 0);
26
27     // Binding `None` to a variable needs to be type annotated
28     let none: Option<i32> = None;
29     let _equivalent_none = None::;
30
31     let optional_float = Some(0f32);
32
33     // Unwrapping a `Some` variant will extract the value wrapped.
34     println!("{}", optional_float, optional_float.unwrap());
35
36     // Unwrapping a `None` variant will `panic!`
37     println!("{}", none, none.unwrap());
38 }

```

Result

We've seen that the `Option` enum can be used as a return value from functions that may fail, where `None` can be returned to indicate failure. However, sometimes it is important to express *why* an operation failed. To do this we have the `Result` enum.

The `Result<T, E>` enum has two variants:

- `Ok(value)` which indicates that the operation succeeded, and wraps the `value` returned by the operation. (`value` has type `T`)
- `Err(why)` , which indicates that the operation failed, and wraps `why` , which (hopefully) explains the cause of the failure. (`why` has type `E`)

```
1 mod checked {
2     // Mathematical "errors" we want to catch
3     #[derive(Debug)]
4     pub enum MathError {
5         DivisionByZero,
6         NonPositiveLogarithm,
7         NegativeSquareRoot,
8     }
9
10    pub type MathResult = Result<f64, MathError>;
11
12    pub fn div(x: f64, y: f64) -> MathResult {
13        if y == 0.0 {
14            // This operation would `fail`, instead let's return the reason
15            // the failure wrapped in `Err`
16            Err(MathError::DivisionByZero)
17        } else {
18            // This operation is valid, return the result wrapped in `Ok`
19            Ok(x / y)
20        }
21    }
22
23    pub fn sqrt(x: f64) -> MathResult {
24        if x < 0.0 {
25            Err(MathError::NegativeSquareRoot)
26        } else {
27            Ok(x.sqrt())
28        }
29    }
30
31    pub fn ln(x: f64) -> MathResult {
32        if x <= 0.0 {
33            Err(MathError::NonPositiveLogarithm)
34        } else {
35            Ok(x.ln())
36        }
37    }
38 }
39
40 // `op(x, y)` === `sqrt(ln(x / y))`
41 fn op(x: f64, y: f64) -> f64 {
42     // This is a three level match pyramid!
43     match checked::div(x, y) {
44         Err(why) => panic!("{:?}", why),
45         Ok(ratio) => match checked::ln(ratio) {
46             Err(why) => panic!("{:?}", why),
47             Ok(ln) => match checked::sqrt(ln) {
48                 Err(why) => panic!("{:?}", why),
49                 Ok(sqrt) => sqrt,
50             },
51         },
52     }
53 }
54
55 fn main() {
56     // Will this fail?
57     println!("{}", op(1.0, 10.0));
```


?

Chaining results using `match` can get pretty untidy; luckily, the `?` operator can be used to make things pretty again. `?` is used at the end of an expression returning a `Result`, and is equivalent to a `match` expression, where the `Err(err)` branch expands to an early `return Err(From::from(err))`, and the `Ok(ok)` branch expands to an `ok` expression.

```
1 mod checked {
2     #[derive(Debug)]
3     enum MathError {
4         DivisionByZero,
5         NonPositiveLogarithm,
6         NegativeSquareRoot,
7     }
8
9     type MathResult = Result<f64, MathError>;
10
11     fn div(x: f64, y: f64) -> MathResult {
12         if y == 0.0 {
13             Err(MathError::DivisionByZero)
14         } else {
15             Ok(x / y)
16         }
17     }
18
19     fn sqrt(x: f64) -> MathResult {
20         if x < 0.0 {
21             Err(MathError::NegativeSquareRoot)
22         } else {
23             Ok(x.sqrt())
24         }
25     }
26
27     fn ln(x: f64) -> MathResult {
28         if x <= 0.0 {
29             Err(MathError::NonPositiveLogarithm)
30         } else {
31             Ok(x.ln())
32         }
33     }
34
35     // Intermediate function
36     fn op_(x: f64, y: f64) -> MathResult {
37         // if `div` "fails", then `DivisionByZero` will be `return`ed
38         let ratio = div(x, y)?;
39
40         // if `ln` "fails", then `NonPositiveLogarithm` will be `return`ed
41         let ln = ln(ratio)?;
42
43         sqrt(ln)
44     }
45
46     pub fn op(x: f64, y: f64) {
47         match op_(x, y) {
48             Err(why) => panic!("{}", match why {
49                 MathError::NonPositiveLogarithm
50                     => "logarithm of non-positive number",
51                 MathError::DivisionByZero
52                     => "division by zero",
53                 MathError::NegativeSquareRoot
54                     => "square root of negative number",
55             }),
56             Ok(value) => println!("{}", value),
57         }
58     }
59 }
```

Be sure to check the [documentation](#), as there are many methods to map/compose `Result`.

panic!

The `panic!` macro can be used to generate a panic and start unwinding its stack. While unwinding, the runtime will take care of freeing all the resources *owned* by the thread by calling the destructor of all its objects.

Since we are dealing with programs with only one thread, `panic!` will cause the program to report the panic message and exit.

```

1 // Re-implementation of integer division (//)
2 fn division(dividend: i32, divisor: i32) -> i32 {
3     if divisor == 0 {
4         // Division by zero triggers a panic
5         panic!("division by zero");
6     } else {
7         dividend / divisor
8     }
9 }
10
11 // The `main` task
12 fn main() {
13     // Heap allocated integer
14     let _x = Box::new(0i32);
15
16     // This operation will trigger a task failure
17     division(3, 0);
18
19     println!("This point won't be reached!");
20
21     // `_x` should get destroyed at this point
22 }
```

Let's check that `panic!` doesn't leak memory.

```

$ rustc panic.rs && valgrind ./panic
==4401== Memcheck, a memory error detector
==4401== Copyright (C) 2002-2013, and GNU GPL'd, by Julian Seward et al.
==4401== Using Valgrind-3.10.0.SVN and LibVEX; rerun with -h for copyright
info
==4401== Command: ./panic
==4401==
thread '<main>' panicked at 'division by zero', panic.rs:5
==4401==
==4401== HEAP SUMMARY:
==4401==     in use at exit: 0 bytes in 0 blocks
==4401==   total heap usage: 18 allocs, 18 frees, 1,648 bytes allocated
==4401==
==4401== All heap blocks were freed -- no leaks are possible
==4401==
==4401== For counts of detected and suppressed errors, rerun with: -v
==4401== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```


HashMap

Where vectors store values by an integer index, `HashMap`s store values by key. `HashMap` keys can be booleans, integers, strings, or any other type that implements the `Eq` and `Hash` traits. More on this in the next section.

Like vectors, `HashMap`s are growable, but `HashMap`s can also shrink themselves when they have excess space. You can create a `HashMap` with a certain starting capacity using `HashMap::with_capacity(uint)`, or use `HashMap::new()` to get a `HashMap` with a default initial capacity (recommended).

```
1 use std::collections::HashMap;
2
3 fn call(number: &str) -> &str {
4     match number {
5         "798-1364" => "We're sorry, the call cannot be completed as dialed.
6             Please hang up and try again.",
7         "645-7689" => "Hello, this is Mr. Awesome's Pizza. My name is Freddie.
8             What can I get for you today?",
9         _ => "Hi! Who is this again?"
10    }
11 }
12
13 fn main() {
14     let mut contacts = HashMap::new();
15
16     contacts.insert("Daniel", "798-1364");
17     contacts.insert("Ashley", "645-7689");
18     contacts.insert("Katie", "435-8291");
19     contacts.insert("Robert", "956-1745");
20
21     // Takes a reference and returns Option<&V>
22     match contacts.get(&"Daniel") {
23         Some(&number) => println!("Calling Daniel: {}", call(number)),
24         _ => println!("Don't have Daniel's number."),
25     }
26
27     // `HashMap::insert()` returns `None`
28     // if the inserted value is new, `Some(value)` otherwise
29     contacts.insert("Daniel", "164-6743");
30
31     match contacts.get(&"Ashley") {
32         Some(&number) => println!("Calling Ashley: {}", call(number)),
33         _ => println!("Don't have Ashley's number."),
34     }
35
36     contacts.remove(&"Ashley");
37
38     // `HashMap::iter()` returns an iterator that yields
39     // (&'a key, &'a value) pairs in arbitrary order.
40     for (contact, &number) in contacts.iter() {
41         println!("Calling {}: {}", contact, call(number));
42     }
43 }
```

For more information on how hashing and hash maps (sometimes called hash tables) work, have a look at [Hash Table Wikipedia](#)

Alternate/custom key types

Any type that implements the `Eq` and `Hash` traits can be a key in `HashMap`. This includes:

- `bool` (though not very useful since there is only two possible keys)
- `int`, `uint`, and all variations thereof
- `String` and `&str` (protip: you can have a `HashMap` keyed by `String` and call `.get()` with an `&str`)

Note that `f32` and `f64` do *not* implement `Hash`, likely because [floating-point precision errors](#) would make using them as hashmap keys horribly error-prone.

All collection classes implement `Eq` and `Hash` if their contained type also respectively implements `Eq` and `Hash`. For example, `Vec<T>` will implement `Hash` if `T` implements `Hash`.

You can easily implement `Eq` and `Hash` for a custom type with just one line:

```
#[derive(PartialEq, Eq, Hash)]
```

The compiler will do the rest. If you want more control over the details, you can implement `Eq` and/or `Hash` yourself. This guide will not cover the specifics of implementing `Hash`.

To play around with using a `struct` in `HashMap`, let's try making a very simple user logon system:

```
1 use std::collections::HashMap;
2
3 // Eq requires that you derive PartialEq on the type.
4 #[derive(PartialEq, Eq, Hash)]
5 struct Account<'a>{
6     username: &'a str,
7     password: &'a str,
8 }
9
10 struct AccountInfo<'a>{
11     name: &'a str,
12     email: &'a str,
13 }
14
15 type Accounts<'a> = HashMap<Account<'a>, AccountInfo<'a>>;
16
17 fn try_logon<'a>(accounts: &Accounts<'a>,
18     username: &'a str, password: &'a str){
19     println!("Username: {}", username);
20     println!("Password: {}", password);
21     println!("Attempting logon...");
22
23     let logon = Account {
24         username,
25         password,
26     };
27
28     match accounts.get(&logon) {
29         Some(account_info) => {
30             println!("Successful logon!");
31             println!("Name: {}", account_info.name);
32             println!("Email: {}", account_info.email);
33         },
34         _ => println!("Login failed!"),
35     }
36 }
37
38 fn main(){
39     let mut accounts: Accounts = HashMap::new();
40
41     let account = Account {
42         username: "j.everyman",
43         password: "password123",
44     };
45
46     let account_info = AccountInfo {
47         name: "John Everyman",
48         email: "j.everyman@email.com",
49     };
50
51     accounts.insert(account, account_info);
52
53     try_logon(&accounts, "j.everyman", "psasword123");
54
55     try_logon(&accounts, "j.everyman", "password123");
56 }
```

HashSet

Consider a `HashSet` as a `HashMap` where we just care about the keys (`HashSet<T>` is, in actuality, just a wrapper around `HashMap<T, ()>`).

"What's the point of that?" you ask. "I could just store the keys in a `Vec`."

A `HashSet` 's unique feature is that it is guaranteed to not have duplicate elements. That's the contract that any set collection fulfills. `HashSet` is just one implementation. (see also: [BTreeSet](#))

If you insert a value that is already present in the `HashSet` , (i.e. the new value is equal to the existing and they both have the same hash), then the new value will replace the old.

This is great for when you never want more than one of something, or when you want to know if you've already got something.

But sets can do more than that.

Sets have 4 primary operations (all of the following calls return an iterator):

- `union` : get all the unique elements in both sets.
- `difference` : get all the elements that are in the first set but not the second.
- `intersection` : get all the elements that are only in *both* sets.
- `symmetric_difference` : get all the elements that are in one set or the other, but *not* both.

Try all of these in the following example:

```
1 use std::collections::HashSet;
2
3 fn main() {
4     let mut a: HashSet<i32> = vec![1i32, 2, 3].into_iter().collect();
5     let mut b: HashSet<i32> = vec![2i32, 3, 4].into_iter().collect();
6
7     assert!(a.insert(4));
8     assert!(a.contains(&4));
9
10    // `HashSet::insert()` returns false if
11    // there was a value already present.
12    assert!(b.insert(4), "Value 4 is already in set B!");
13    // FIXME ^ Comment out this line
14
15    b.insert(5);
16
17    // If a collection's element type implements `Debug`,
18    // then the collection implements `Debug`.
19    // It usually prints its elements in the format `[elem1, elem2, ...]`
20    println!("A: {:?}", a);
21    println!("B: {:?}", b);
22
23    // Print [1, 2, 3, 4, 5] in arbitrary order
24    println!("Union: {:?}", a.union(&b).collect::
```

(Examples are adapted from the [documentation](#).)

Rc

When multiple ownership is needed, `Rc` (Reference Counting) can be used. `Rc` keeps track of the number of the references which means the number of owners of the value wrapped inside an `Rc`.

Reference count of an `Rc` increases by 1 whenever an `Rc` is cloned, and decreases by 1 whenever one cloned `Rc` is dropped out of the scope. When an `Rc`'s reference count becomes zero (which means there are no remaining owners), both the `Rc` and the value are all dropped.

Cloning an `Rc` never performs a deep copy. Cloning creates just another pointer to the wrapped value, and increments the count.

```
1 use std::rc::Rc;
2
3 fn main() {
4     let rc_examples = "Rc examples".to_string();
5     {
6         println!("--- rc_a is created ---");
7
8         let rc_a: Rc<String> = Rc::new(rc_examples);
9         println!("Reference Count of rc_a: {}", Rc::strong_count(&rc_a));
10
11         {
12             println!("--- rc_a is cloned to rc_b ---");
13
14             let rc_b: Rc<String> = Rc::clone(&rc_a);
15             println!("Reference Count of rc_b: {}", Rc::strong_count(&rc_b));
16             println!("Reference Count of rc_a: {}", Rc::strong_count(&rc_a));
17
18             // Two `Rc`s are equal if their inner values are equal
19             println!("rc_a and rc_b are equal: {}", rc_a.eq(&rc_b));
20
21             // We can use methods of a value directly
22             println!("Length of the value inside rc_a: {}", rc_a.len());
23             println!("Value of rc_b: {}", rc_b);
24
25             println!("--- rc_b is dropped out of scope ---");
26         }
27
28         println!("Reference Count of rc_a: {}", Rc::strong_count(&rc_a));
29
30         println!("--- rc_a is dropped out of scope ---");
31     }
32
33     // Error! `rc_examples` already moved into `rc_a`
34     // And when `rc_a` is dropped, `rc_examples` is dropped together
35     // println!("rc_examples: {}", rc_examples);
36     // TODO ^ Try uncommenting this line
37 }
```

See also:

[std::rc](#) and [std::sync::arc](#).

Arc

When shared ownership between threads is needed, `Arc` (Atomically Reference Counted) can be used. This struct, via the `clone` implementation can create a reference pointer for the location of a value in the memory heap while increasing the reference counter. As it shares ownership between threads, when the last reference pointer to a value is out of scope, the variable is dropped.

```
1 use std::time::Duration;
2 use std::sync::Arc;
3 use std::thread;
4
5 fn main() {
6     // This variable declaration is where its value is specified.
7     let apple = Arc::new("the same apple");
8
9     for _ in 0..10 {
10        // Here there is no value specification as it is a pointer to a
11        // reference in the memory heap.
12        let apple = Arc::clone(&apple);
13
14        thread::spawn(move || {
15            // As Arc was used, threads can be spawned using the value at
16            // in the Arc variable pointer's location.
17            println!("{:?}", apple);
18        });
19    }
20
21    // Make sure all Arc instances are printed from spawned threads.
22    thread::sleep(Duration::from_secs(1));
23 }
```

Std misc

Many other types are provided by the std library to support things such as:

- Threads
- Channels
- File I/O

These expand beyond what the [primitives](#) provide.

See also:

[primitives](#) and [the std library](#)

Threads

Rust provides a mechanism for spawning native OS threads via the `spawn` function, the argument of this function is a moving closure.

```
1 use std::thread;
2
3 const NTHREADS: u32 = 10;
4
5 // This is the `main` thread
6 fn main() {
7     // Make a vector to hold the children which are spawned.
8     let mut children = vec![];
9
10    for i in 0..NTHREADS {
11        // Spin up another thread
12        children.push(thread::spawn(move || {
13            println!("this is thread number {}", i);
14        }));
15    }
16
17    for child in children {
18        // Wait for the thread to finish. Returns a result.
19        let _ = child.join();
20    }
21 }
```

These threads will be scheduled by the OS.

Testcase: map-reduce

Rust makes it very easy to parallelise data processing, without many of the headaches traditionally associated with such an attempt.

The standard library provides great threading primitives out of the box. These, combined with Rust's concept of Ownership and aliasing rules, automatically prevent data races.

The aliasing rules (one writable reference XOR many readable references) automatically prevent you from manipulating state that is visible to other threads. (Where synchronisation is needed, there are synchronisation primitives like `Mutex`s or `Channels`.)

In this example, we will calculate the sum of all digits in a block of numbers. We will do this by parcelling out chunks of the block into different threads. Each thread will sum its tiny block of digits, and subsequently we will sum the intermediate sums produced by each thread.

Note that, although we're passing references across thread boundaries, Rust understands that we're only passing read-only references, and that thus no unsafety or data races can occur. Also because the references we're passing have `'static` lifetimes, Rust understands that our data won't be destroyed while these threads are still running. (When you need to share non-`static` data between threads, you can use a smart pointer like `Arc` to keep the data alive and avoid non-`static` lifetimes.)

```
1 use std::thread;
2
3 // This is the `main` thread
4 fn main() {
5
6     // This is our data to process.
7     // We will calculate the sum of all digits via a threaded map-reduce
8     // Each whitespace separated chunk will be handled in a different thr
9     //
10    // TODO: see what happens to the output if you insert spaces!
11    let data = "86967897737416471853297327050364959
1211861322575564723963297542624962850
1370856234701860851907960690014725639
1438397966707106094172783238747669219
1552380795257888236525459303330302837
1658495327135744041048897885734297812
1769920216438980873548808413720956532
1816278424637452589860345374828574668";
19
20    // Make a vector to hold the child-threads which we will spawn.
21    let mut children = vec![];
22
23    /*****
24     * "Map" phase
25     *
26     * Divide our data into segments, and apply initial processing
27     *****/
28
29    // split our data into segments for individual calculation
30    // each chunk will be a reference (&str) into the actual data
31    let chunked_data = data.split_whitespace();
32
33    // Iterate over the data segments.
34    // .enumerate() adds the current loop index to whatever is iterated
35    // the resulting tuple "(index, element)" is then immediately
36    // "destructured" into two variables, "i" and "data_segment" with a
37    // "destructuring assignment"
38    for (i, data_segment) in chunked_data.enumerate() {
39        println!("data segment {} is \"{}\"", i, data_segment);
40
41        // Process each data segment in a separate thread
42        //
43        // spawn() returns a handle to the new thread,
44        // which we MUST keep to access the returned value
45        //
46        // 'move || -> u32' is syntax for a closure that:
47        // * takes no arguments ('||')
48        // * takes ownership of its captured variables ('move') and
49        // * returns an unsigned 32-bit integer ('-> u32')
50        //
51        // Rust is smart enough to infer the '-> u32' from
52        // the closure itself so we could have left that out.
53        //
54        // TODO: try removing the 'move' and see what happens
55        children.push(thread::spawn(move || -> u32 {
56            // Calculate the intermediate sum of this segment:
57            let result = data_segment
```

Assignments

It is not wise to let our number of threads depend on user inputted data. What if the user decides to insert a lot of spaces? Do we *really* want to spawn 2,000 threads? Modify the program so that the data is always chunked into a limited number of chunks, defined by a static constant at the beginning of the program.

See also:

- [Threads](#)
- [vectors](#) and [iterators](#)
- [closures](#), [move](#) semantics and [move closures](#)
- [destructuring](#) assignments

- [turbofish notation](#) to help type inference
- [unwrap vs. expect](#)
- [enumerate](#)

Channels

Rust provides asynchronous `channels` for communication between threads. Channels allow a unidirectional flow of information between two end-points: the `Sender` and the `Receiver`.

```
1 use std::sync::mpsc::{Sender, Receiver};
2 use std::sync::mpsc;
3 use std::thread;
4
5 static NTHREADS: i32 = 3;
6
7 fn main() {
8     // Channels have two endpoints: the `Sender<T>` and the `Receiver<T>`
9     // where `T` is the type of the message to be transferred
10    // (type annotation is superfluous)
11    let (tx, rx): (Sender<i32>, Receiver<i32>) = mpsc::channel();
12    let mut children = Vec::new();
13
14    for id in 0..NTHREADS {
15        // The sender endpoint can be copied
16        let thread_tx = tx.clone();
17
18        // Each thread will send its id via the channel
19        let child = thread::spawn(move || {
20            // The thread takes ownership over `thread_tx`
21            // Each thread queues a message in the channel
22            thread_tx.send(id).unwrap();
23
24            // Sending is a non-blocking operation, the thread will continue
25            // immediately after sending its message
26            println!("thread {} finished", id);
27        });
28
29        children.push(child);
30    }
31
32    // Here, all the messages are collected
33    let mut ids = Vec::with_capacity(NTHREADS as usize);
34    for _ in 0..NTHREADS {
35        // The `recv` method picks a message from the channel
36        // `recv` will block the current thread if there are no messages
37        ids.push(rx.recv());
38    }
39
40    // Wait for the threads to complete any remaining work
41    for child in children {
42        child.join().expect("oops! the child thread panicked");
43    }
44
45    // Show the order in which the messages were sent
46    println!("{:?}", ids);
47 }
```


Path

The `Path` struct represents file paths in the underlying filesystem. There are two flavors of `Path`: `posix::Path`, for UNIX-like systems, and `windows::Path`, for Windows. The prelude exports the appropriate platform-specific `Path` variant.

A `Path` can be created from an `OsStr`, and provides several methods to get information from the file/directory the path points to.

A `Path` is immutable. The owned version of `Path` is `PathBuf`. The relation between `Path` and `PathBuf` is similar to that of `str` and `String`: a `PathBuf` can be mutated in-place, and can be dereferenced to a `Path`.

Note that a `Path` is *not* internally represented as an UTF-8 string, but instead is stored as an `OsString`. Therefore, converting a `Path` to a `&str` is *not* free and may fail (an `Option` is returned). However, a `Path` can be freely converted to an `OsString` or `&OsStr` using `into_os_string` and `as_os_str`, respectively.

```
1 use std::path::Path;
2
3 fn main() {
4     // Create a `Path` from an `&'static str`
5     let path = Path::new(".");
6
7     // The `display` method returns a `Display`able structure
8     let _display = path.display();
9
10    // `join` merges a path with a byte container using the OS specific
11    // separator, and returns a `PathBuf`
12    let mut new_path = path.join("a").join("b");
13
14    // `push` extends the `PathBuf` with a `&Path`
15    new_path.push("c");
16    new_path.push("myfile.tar.gz");
17
18    // `set_file_name` updates the file name of the `PathBuf`
19    new_path.set_file_name("package.tgz");
20
21    // Convert the `PathBuf` into a string slice
22    match new_path.to_str() {
23        None => panic!("new path is not a valid UTF-8 sequence"),
24        Some(s) => println!("new path is {}", s),
25    }
26 }
27
```

Be sure to check at other `Path` methods (`posix::Path` or `windows::Path`) and the `Metadata` struct.

See also:

[OsStr](#) and [Metadata](#).

File I/O

The `File` struct represents a file that has been opened (it wraps a file descriptor), and gives read and/or write access to the underlying file.

Since many things can go wrong when doing file I/O, all the `File` methods return the `io::Result<T>` type, which is an alias for `Result<T, io::Error>`.

This makes the failure of all I/O operations *explicit*. Thanks to this, the programmer can see all the failure paths, and is encouraged to handle them in a proactive manner.

open

The `open` function can be used to open a file in read-only mode.

A `File` owns a resource, the file descriptor and takes care of closing the file when it is dropped.

```
1 use std::fs::File;
2 use std::io::prelude::*;
3 use std::path::Path;
4
5 fn main() {
6     // Create a path to the desired file
7     let path = Path::new("hello.txt");
8     let display = path.display();
9
10    // Open the path in read-only mode, returns `io::Result<File>`
11    let mut file = match File::open(&path) {
12        Err(why) => panic!("couldn't open {}: {}", display, why),
13        Ok(file) => file,
14    };
15
16    // Read the file contents into a string, returns `io::Result<usize>`
17    let mut s = String::new();
18    match file.read_to_string(&mut s) {
19        Err(why) => panic!("couldn't read {}: {}", display, why),
20        Ok(_) => print!("{}", contains:\n{}", display, s),
21    }
22
23    // `file` goes out of scope, and the "hello.txt" file gets closed
24 }
```

Here's the expected successful output:

```
$ echo "Hello World!" > hello.txt
$ rustc open.rs && ./open
hello.txt contains:
Hello World!
```

(You are encouraged to test the previous example under different failure conditions: `hello.txt` doesn't exist, or `hello.txt` is not readable, etc.)

create

The `create` function opens a file in write-only mode. If the file already existed, the old content is destroyed. Otherwise, a new file is created.

```
static LOREM_IPSUM: &str =
    "Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
    quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
    consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
    cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non
    proident, sunt in culpa qui officia deserunt mollit anim id est laborum."
    ";

use std::fs::File;
use std::io::prelude::*;
use std::path::Path;

fn main() {
    let path = Path::new("lorem_ipsum.txt");
    let display = path.display();

    // Open a file in write-only mode, returns `io::Result<File>`
    let mut file = match File::create(&path) {
        Err(why) => panic!("couldn't create {}: {}", display, why),
        Ok(file) => file,
    };

    // Write the `LOREM_IPSUM` string to `file`, returns `io::Result<()>`
    match file.write_all(LOREM_IPSUM.as_bytes()) {
        Err(why) => panic!("couldn't write to {}: {}", display, why),
        Ok(_) => println!("successfully wrote to {} ", display),
    }
}
```

Here's the expected successful output:

```
$ rustc create.rs && ./create
successfully wrote to lorem_ipsum.txt
$ cat lorem_ipsum.txt
Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non
proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
```

(As in the previous example, you are encouraged to test this example under failure conditions.)

The `OpenOptions` struct can be used to configure how a file is opened.

read_lines

A naive approach

This might be a reasonable first attempt for a beginner's first implementation for reading lines from a file.

```
use std::fs::read_to_string;

fn read_lines(filename: &str) -> Vec<String> {
    let mut result = Vec::new();

    for line in read_to_string(filename).unwrap().lines() {
        result.push(line.to_string())
    }

    result
}
```

Since the method `lines()` returns an iterator over the lines in the file, we can also perform a map inline and collect the results, yielding a more concise and fluent expression.

```
use std::fs::read_to_string;

fn read_lines(filename: &str) -> Vec<String> {
    read_to_string(filename)
        .unwrap() // panic on possible file-reading errors
        .lines()  // split the string into an iterator of string slices
        .map(String::from) // make each slice into a string
        .collect() // gather them together into a vector
}
```

Note that in both examples above, we must convert the `&str` reference returned from `lines()` to the owned type `String`, using `.to_string()` and `String::from` respectively.

A more efficient approach

Here we pass ownership of the open `File` to a `BufReader` struct. `BufReader` uses an internal buffer to reduce intermediate allocations.

We also update `read_lines` to return an iterator instead of allocating new `String`

objects in memory for each line.

```
use std::fs::File;
use std::io::{self, BufRead};
use std::path::Path;

fn main() {
    // File hosts.txt must exist in the current path
    if let Ok(lines) = read_lines("./hosts.txt") {
        // Consumes the iterator, returns an (Optional) String
        for line in lines {
            if let Ok(ip) = line {
                println!("{}", ip);
            }
        }
    }
}

// The output is wrapped in a Result to allow matching on errors
// Returns an Iterator to the Reader of the lines of the file.
fn read_lines<P>(filename: P) -> io::Result<io::Lines<io::BufReader<File>>>
where P: AsRef<Path>, {
    let file = File::open(filename)?;
    Ok(io::BufReader::new(file).lines())
}
```

Running this program simply prints the lines individually.

```
$ echo -e "127.0.0.1\n192.168.0.1\n" > hosts.txt
$ rustc read_lines.rs && ./read_lines
127.0.0.1
192.168.0.1
```

(Note that since `File::open` expects a generic `AsRef<Path>` as argument, we define our generic `read_lines()` method with the same generic constraint, using the `where` keyword.)

This process is more efficient than creating a `String` in memory with all of the file's contents. This can especially cause performance issues when working with larger files.

Child processes

The `process::Output` struct represents the output of a finished child process, and the `process::Command` struct is a process builder.

```
1 use std::process::Command;
2
3 fn main() {
4     let output = Command::new("rustc")
5         .arg("--version")
6         .output().unwrap_or_else(|e| {
7             panic!("failed to execute process: {}", e)
8         });
9
10    if output.status.success() {
11        let s = String::from_utf8_lossy(&output.stdout);
12
13        print!("rustc succeeded and stdout was:\n{}", s);
14    } else {
15        let s = String::from_utf8_lossy(&output.stderr);
16
17        print!("rustc failed and stderr was:\n{}", s);
18    }
19 }
```

(You are encouraged to try the previous example with an incorrect flag passed to `rustc`)

Pipes

The `std::child` struct represents a running child process, and exposes the `stdin`, `stdout` and `stderr` handles for interaction with the underlying process via pipes.

```
use std::io::prelude::*;
use std::process::{Command, Stdio};

static PANGRAM: &'static str =
    "the quick brown fox jumped over the lazy dog\n";

fn main() {
    // Spawn the `wc` command
    let process = match Command::new("wc")
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn() {
        Err(why) => panic!("couldn't spawn wc: {}", why),
        Ok(process) => process,
    };

    // Write a string to the `stdin` of `wc`.
    //
    // `stdin` has type `Option<ChildStdin>`, but since we know this instance
    // must have one, we can directly `unwrap` it.
    match process.stdin.unwrap().write_all(PANGRAM.as_bytes()) {
        Err(why) => panic!("couldn't write to wc stdin: {}", why),
        Ok(_) => println!("sent pangram to wc"),
    }

    // Because `stdin` does not live after the above calls, it is `drop`ed,
    // and the pipe is closed.
    //
    // This is very important, otherwise `wc` wouldn't start processing the
    // input we just sent.

    // The `stdout` field also has type `Option<ChildStdout>` so must be
    unwrapped.
    let mut s = String::new();
    match process.stdout.unwrap().read_to_string(&mut s) {
        Err(why) => panic!("couldn't read wc stdout: {}", why),
        Ok(_) => print!("wc responded with:\n{}", s),
    }
}
```

Wait

If you'd like to wait for a `process::Child` to finish, you must call `child::wait`, which will return a `process::ExitStatus`.

```
use std::process::Command;

fn main() {
    let mut child = Command::new("sleep").arg("5").spawn().unwrap();
    let _result = child.wait().unwrap();

    println!("reached end of main");
}
```

```
$ rustc wait.rs && ./wait
# `wait` keeps running for 5 seconds until the `sleep 5` command finishes
reached end of main
```

Filesystem Operations

The `std::fs` module contains several functions that deal with the filesystem.

```
use std::fs;
use std::fs::{File, OpenOptions};
use std::io;
use std::io::prelude::*;
use std::os::unix;
use std::path::Path;

// A simple implementation of `% cat path`
fn cat(path: &Path) -> io::Result<String> {
    let mut f = File::open(path)?;
    let mut s = String::new();
    match f.read_to_string(&mut s) {
        Ok(_) => Ok(s),
        Err(e) => Err(e),
    }
}

// A simple implementation of `% echo s > path`
fn echo(s: &str, path: &Path) -> io::Result<()> {
    let mut f = File::create(path)?;

    f.write_all(s.as_bytes())
}

// A simple implementation of `% touch path` (ignores existing files)
fn touch(path: &Path) -> io::Result<()> {
    match OpenOptions::new().create(true).write(true).open(path) {
        Ok(_) => Ok(()),
        Err(e) => Err(e),
    }
}

fn main() {
    println!("`mkdir a`");
    // Create a directory, returns `io::Result<()>`
    match fs::create_dir("a") {
        Err(why) => println!("! {:?}", why.kind()),
        Ok(_) => {},
    }

    println!("`echo hello > a/b.txt`");
    // The previous match can be simplified using the `unwrap_or_else` method
    echo("hello", &Path::new("a/b.txt")).unwrap_or_else(|why| {
        println!("! {:?}", why.kind());
    });

    println!("`mkdir -p a/c/d`");
    // Recursively create a directory, returns `io::Result<()>`
    fs::create_dir_all("a/c/d").unwrap_or_else(|why| {
        println!("! {:?}", why.kind());
    });

    println!("`touch a/c/e.txt`");
    touch(&Path::new("a/c/e.txt")).unwrap_or_else(|why| {
        println!("! {:?}", why.kind());
    });
}
```

```
println!("`ln -s ../b.txt a/c/b.txt`");
// Create a symbolic link, returns `io::Result<()>`
if cfg!(target_family = "unix") {
    unix::fs::symlink("../b.txt", "a/c/b.txt").unwrap_or_else(|why| {
        println!("! {:?}", why.kind());
    });
}

println!("`cat a/c/b.txt`");
match cat(&Path::new("a/c/b.txt")) {
    Err(why) => println!("! {:?}", why.kind()),
    Ok(s) => println!("> {}", s),
}

println!("`ls a`");
// Read the contents of a directory, returns `io::Result<Vec<Path>>`
match fs::read_dir("a") {
    Err(why) => println!("! {:?}", why.kind()),
    Ok(paths) => for path in paths {
        println!("> {:?}", path.unwrap().path());
    },
}

println!("`rm a/c/e.txt`");
// Remove a file, returns `io::Result<()>`
fs::remove_file("a/c/e.txt").unwrap_or_else(|why| {
    println!("! {:?}", why.kind());
});

println!("`rmdir a/c/d`");
// Remove an empty directory, returns `io::Result<()>`
fs::remove_dir("a/c/d").unwrap_or_else(|why| {
    println!("! {:?}", why.kind());
});
}
```

Here's the expected successful output:

```
$ rustc fs.rs && ./fs
`mkdir a`
`echo hello > a/b.txt`
`mkdir -p a/c/d`
`touch a/c/e.txt`
`ln -s ../b.txt a/c/b.txt`
`cat a/c/b.txt`
> hello
`ls a`
> "a/b.txt"
> "a/c"
`rm a/c/e.txt`
`rmdir a/c/d`
```

And the final state of the `a` directory is:

```
$ tree a
a
|-- b.txt
`-- c
    |-- b.txt -> ../b.txt
```

1 directory, 2 files

An alternative way to define the function `cat` is with `?` notation:

```
fn cat(path: &Path) -> io::Result<String> {
    let mut f = File::open(path)?;
    let mut s = String::new();
    f.read_to_string(&mut s)?;
    Ok(s)
}
```

See also:

[cfg!](#)

Program arguments

Standard Library

The command line arguments can be accessed using `std::env::args`, which returns an iterator that yields a `String` for each argument:

```
1 use std::env;
2
3 fn main() {
4     let args: Vec<String> = env::args().collect();
5
6     // The first argument is the path that was used to call the program.
7     println!("My path is {}. ", args[0]);
8
9     // The rest of the arguments are the passed command line parameters.
10    // Call the program like this:
11    // $ ./args arg1 arg2
12    println!("I got {:?} arguments: {:?}. ", args.len() - 1, &args[1..]);
13 }
```

```
$ ./args 1 2 3
My path is ./args.
I got 3 arguments: ["1", "2", "3"].
```

Crates

Alternatively, there are numerous crates that can provide extra functionality when creating command-line applications. The [Rust Cookbook](#) exhibits best practices on how to use one of the more popular command line argument crates, `clap`.

Argument parsing

Matching can be used to parse simple arguments:


```
1 use std::env;
2
3 fn increase(number: i32) {
4     println!("{}", number + 1);
5 }
6
7 fn decrease(number: i32) {
8     println!("{}", number - 1);
9 }
10
11 fn help() {
12     println!("usage:
13 match_args <string>
14     Check whether given string is the answer.
15 match_args {{increase|decrease}} <integer>
16     Increase or decrease given integer by one.");
17 }
18
19 fn main() {
20     let args: Vec<String> = env::args().collect();
21
22     match args.len() {
23         // no arguments passed
24         1 => {
25             println!("My name is 'match_args'. Try passing some arguments
26         },
27         // one argument passed
28         2 => {
29             match args[1].parse() {
30                 Ok(42) => println!("This is the answer!"),
31                 _ => println!("This is not the answer."),
32             }
33         },
34         // one command and one argument passed
35         3 => {
36             let cmd = &args[1];
37             let num = &args[2];
38             // parse the number
39             let number: i32 = match num.parse() {
40                 Ok(n) => {
41                     n
42                 },
43                 Err(_) => {
44                     eprintln!("error: second argument not an integer");
45                     help();
46                     return;
47                 },
48             };
49             // parse the command
50             match &cmd[..] {
51                 "increase" => increase(number),
52                 "decrease" => decrease(number),
53                 _ => {
54                     eprintln!("error: invalid command");
55                     help();
56                 },
57             }
58         }
59     }
60 }
```

```
$ ./match_args Rust
This is not the answer.
$ ./match_args 42
This is the answer!
$ ./match_args do something
error: second argument not an integer
usage:
match_args <string>
    Check whether given string is the answer.
match_args {increase|decrease} <integer>
    Increase or decrease given integer by one.
$ ./match_args do 42
error: invalid command
usage:
match_args <string>
    Check whether given string is the answer.
match_args {increase|decrease} <integer>
    Increase or decrease given integer by one.
$ ./match_args increase 42
43
```

Foreign Function Interface

Rust provides a Foreign Function Interface (FFI) to C libraries. Foreign functions must be declared inside an `extern` block annotated with a `#[link]` attribute containing the name of the foreign library.

```
use std::fmt;

// this extern block links to the libm library
#[link(name = "m")]
extern {
    // this is a foreign function
    // that computes the square root of a single precision complex number
    fn csqrtf(z: Complex) -> Complex;

    fn ccosf(z: Complex) -> Complex;
}

// Since calling foreign functions is considered unsafe,
// it's common to write safe wrappers around them.
fn cos(z: Complex) -> Complex {
    unsafe { ccosf(z) }
}

fn main() {
    // z = -1 + 0i
    let z = Complex { re: -1., im: 0. };

    // calling a foreign function is an unsafe operation
    let z_sqrt = unsafe { csqrtf(z) };

    println!("the square root of {:?} is {:?}", z, z_sqrt);

    // calling safe API wrapped around unsafe operation
    println!("cos({:?}) = {:?}", z, cos(z));
}

// Minimal implementation of single precision complex numbers
#[repr(C)]
#[derive(Clone, Copy)]
struct Complex {
    re: f32,
    im: f32,
}

impl fmt::Debug for Complex {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        if self.im < 0. {
            write!(f, "{}-{}i", self.re, -self.im)
        } else {
            write!(f, "{}+{}i", self.re, self.im)
        }
    }
}
```

Testing

Rust is a programming language that cares a lot about correctness and it includes support for writing software tests within the language itself.

Testing comes in three styles:

- [Unit](#) testing.
- [Doc](#) testing.
- [Integration](#) testing.

Also Rust has support for specifying additional dependencies for tests:

- [Dev-dependencies](#)

See Also

- [The Book](#) chapter on testing
- [API Guidelines](#) on doc-testing

Unit testing

Tests are Rust functions that verify that the non-test code is functioning in the expected manner. The bodies of test functions typically perform some setup, run the code we want to test, then assert whether the results are what we expect.

Most unit tests go into a `tests mod` with the `#[cfg(test)]` [attribute](#). Test functions are marked with the `#[test]` attribute.

Tests fail when something in the test function [panics](#). There are some helper [macros](#):

- `assert!(expression)` - panics if expression evaluates to `false`.
- `assert_eq!(left, right)` and `assert_ne!(left, right)` - testing left and right expressions for equality and inequality respectively.

```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

// This is a really bad adding function, its purpose is to fail in this
// example.
#[allow(dead_code)]
fn bad_add(a: i32, b: i32) -> i32 {
    a - b
}

#[cfg(test)]
mod tests {
    // Note this useful idiom: importing names from outer (for mod tests)
    scope.
    use super::*;

    #[test]
    fn test_add() {
        assert_eq!(add(1, 2), 3);
    }

    #[test]
    fn test_bad_add() {
        // This assert would fire and test will fail.
        // Please note, that private functions can be tested too!
        assert_eq!(bad_add(1, 2), 3);
    }
}
```

Tests can be run with `cargo test`.

```
$ cargo test
```

```
running 2 tests
test tests::test_bad_add ... FAILED
test tests::test_add ... ok
```

```
failures:
```

```
---- tests::test_bad_add stdout ----
      thread 'tests::test_bad_add' panicked at 'assertion failed: `(left ==
right)`
  left: `-1`,
 right: `3`, src/lib.rs:21:8
note: Run with `RUST_BACKTRACE=1` for a backtrace.
```

```
failures:
```

```
tests::test_bad_add
```

```
test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered
out
```

Tests and ?

None of the previous unit test examples had a return type. But in Rust 2018, your unit tests can return `Result<(), String>`, which lets you use `?` in them! This can make them much more concise.

```
1 fn sqrt(number: f64) -> Result<f64, String> {
2     if number >= 0.0 {
3         Ok(number.powf(0.5))
4     } else {
5         Err("negative floats don't have square roots".to_owned())
6     }
7 }
8
9 #[cfg(test)]
10 mod tests {
11     use super::*;
12
13     #[test]
14     fn test_sqrt() -> Result<(), String> {
15         let x = 4.0;
16         assert_eq!(sqrt(x)?.powf(2.0), x);
17         Ok(())
18     }
19 }
```

See ["The Edition Guide"](#) for more details.

Testing panics

To check functions that should panic under certain circumstances, use attribute `#[should_panic]`. This attribute accepts optional parameter `expected =` with the text of the panic message. If your function can panic in multiple ways, it helps make sure your test is testing the correct panic.

```
pub fn divide_non_zero_result(a: u32, b: u32) -> u32 {
    if b == 0 {
        panic!("Divide-by-zero error");
    } else if a < b {
        panic!("Divide result is zero");
    }
    a / b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_divide() {
        assert_eq!(divide_non_zero_result(10, 2), 5);
    }

    #[test]
    #[should_panic]
    fn test_any_panic() {
        divide_non_zero_result(1, 0);
    }

    #[test]
    #[should_panic(expected = "Divide result is zero")]
    fn test_specific_panic() {
        divide_non_zero_result(1, 10);
    }
}
```

Running these tests gives us:


```
$ cargo test

running 3 tests
test tests::test_any_panic ... ok
test tests::test_divide ... ok
test tests::test_specific_panic ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Doc-tests tmp-test-should-panic

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Running specific tests

To run specific tests one may specify the test name to `cargo test` command.

```
$ cargo test test_any_panic
running 1 test
test tests::test_any_panic ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 2 filtered out

Doc-tests tmp-test-should-panic

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

To run multiple tests one may specify part of a test name that matches all the tests that should be run.

```
$ cargo test panic
running 2 tests
test tests::test_any_panic ... ok
test tests::test_specific_panic ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 1 filtered out

Doc-tests tmp-test-should-panic

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Ignoring tests

Tests can be marked with the `#[ignore]` attribute to exclude some tests. Or to run them with command `cargo test -- --ignored`

```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_add() {
        assert_eq!(add(2, 2), 4);
    }

    #[test]
    fn test_add_hundred() {
        assert_eq!(add(100, 2), 102);
        assert_eq!(add(2, 100), 102);
    }

    #[test]
    #[ignore]
    fn ignored_test() {
        assert_eq!(add(0, 0), 0);
    }
}
```

```
$ cargo test
running 3 tests
test tests::ignored_test ... ignored
test tests::test_add ... ok
test tests::test_add_hundred ... ok

test result: ok. 2 passed; 0 failed; 1 ignored; 0 measured; 0 filtered out

Doc-tests tmp-ignore

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

$ cargo test -- --ignored
running 1 test
test tests::ignored_test ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Doc-tests tmp-ignore

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Documentation testing

The primary way of documenting a Rust project is through annotating the source code. Documentation comments are written in [CommonMark Markdown specification](#) and support code blocks in them. Rust takes care about correctness, so these code blocks are compiled and used as documentation tests.

```
/// First line is a short summary describing function.
///
/// The next lines present detailed documentation. Code blocks start with
/// triple backquotes and have implicit `fn main()` inside
/// and `extern crate <cratename>`. Assume we're testing `doccomments` crate:
/// ```
/// let result = doccomments::add(2, 3);
/// assert_eq!(result, 5);
/// ```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

/// Usually doc comments may include sections "Examples", "Panics" and
/// "Failures".
///
/// The next function divides two numbers.
///
/// # Examples
/// ```
/// let result = doccomments::div(10, 2);
/// assert_eq!(result, 5);
/// ```
///
/// # Panics
///
/// The function panics if the second argument is zero.
///
/// ```rust,should_panic
/// // panics on division by zero
/// doccomments::div(10, 0);
/// ```
pub fn div(a: i32, b: i32) -> i32 {
    if b == 0 {
        panic!("Divide-by-zero error");
    }

    a / b
}
```

Code blocks in documentation are automatically tested when running the regular `cargo test` command:

```
$ cargo test
running 0 tests
```

```
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

```
Doc-tests doccomments
```

```
running 3 tests
test src/lib.rs - add (line 7) ... ok
test src/lib.rs - div (line 21) ... ok
test src/lib.rs - div (line 31) ... ok
```

```
test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Motivation behind documentation tests

The main purpose of documentation tests is to serve as examples that exercise the functionality, which is one of the most important [guidelines](#). It allows using examples from docs as complete code snippets. But using `?` makes compilation fail since `main` returns `unit`. The ability to hide some source lines from documentation comes to the rescue: one may write `fn try_main() -> Result<(), ErrorType>`, hide it and `unwrap` it in `hidden main`. Sounds complicated? Here's an example:

```
/// Using hidden `try_main` in doc tests.
///
/// ```
/// # // hidden lines start with `#` symbol, but they're still compilable!
/// # fn try_main() -> Result<(), String> { // line that wraps the body shown
in doc
/// let res = doccomments::try_div(10, 2)?;
/// # Ok(()) // returning from try_main
/// # }
/// # fn main() { // starting main that'll unwrap()
/// #     try_main().unwrap(); // calling try_main and unwrapping
/// #                                     // so that test will panic in case of error
/// # }
/// ```
pub fn try_div(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        Err(String::from("Divide-by-zero"))
    } else {
        Ok(a / b)
    }
}
```

See Also

- [RFC505](#) on documentation style
- [API Guidelines](#) on documentation guidelines

Integration testing

[Unit tests](#) are testing one module in isolation at a time: they're small and can test private code. Integration tests are external to your crate and use only its public interface in the same way any other code would. Their purpose is to test that many parts of your library work correctly together.

Cargo looks for integration tests in `tests` directory next to `src`.

File `src/lib.rs`:

```
// Define this in a crate called `adder`.
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

File with test: `tests/integration_test.rs`:

```
#[test]
fn test_add() {
    assert_eq!(adder::add(3, 2), 5);
}
```

Running tests with `cargo test` command:

```
$ cargo test
running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

    Running target/debug/deps/integration_test-bcd60824f5fbfe19

running 1 test
test test_add ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Doc-tests adder

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Each Rust source file in the `tests` directory is compiled as a separate crate. In order to share some code between integration tests we can make a module with public functions, importing and using it within tests.

File `tests/common/mod.rs`:

```
pub fn setup() {  
    // some setup code, like creating required files/directories, starting  
    // servers, etc.  
}
```

File with test: `tests/integration_test.rs`

```
// importing common module.  
mod common;  
  
#[test]  
fn test_add() {  
    // using common code.  
    common::setup();  
    assert_eq!(adder::add(3, 2), 5);  
}
```

Creating the module as `tests/common.rs` also works, but is not recommended because the test runner will treat the file as a test crate and try to run tests inside it.

Development dependencies

Sometimes there is a need to have dependencies for tests (or examples, or benchmarks) only. Such dependencies are added to `Cargo.toml` in the `[dev-dependencies]` section. These dependencies are not propagated to other packages which depend on this package.

One such example is [pretty_assertions](#), which extends standard `assert_eq!` and `assert_ne!` macros, to provide colorful diff.

File `Cargo.toml`:

```
# standard crate data is left out
[dev-dependencies]
pretty_assertions = "1"
```

File `src/lib.rs`:

```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;
    use pretty_assertions::assert_eq; // crate for test-only use. Cannot be
    used in non-test code.

    #[test]
    fn test_add() {
        assert_eq!(add(2, 3), 5);
    }
}
```

See Also

[Cargo](#) docs on specifying dependencies.

Unsafe Operations

As an introduction to this section, to borrow from [the official docs](#), "one should try to minimize the amount of unsafe code in a code base." With that in mind, let's get started! Unsafe annotations in Rust are used to bypass protections put in place by the compiler; specifically, there are four primary things that unsafe is used for:

- dereferencing raw pointers
- calling functions or methods which are `unsafe` (including calling a function over FFI, see [a previous chapter](#) of the book)
- accessing or modifying static mutable variables
- implementing unsafe traits

Raw Pointers

Raw pointers `*` and references `&T` function similarly, but references are always safe because they are guaranteed to point to valid data due to the borrow checker. Dereferencing a raw pointer can only be done through an unsafe block.

```
1 fn main() {  
2     let raw_p: *const u32 = &10;  
3  
4     unsafe {  
5         assert!(*raw_p == 10);  
6     }  
7 }
```

Calling Unsafe Functions

Some functions can be declared as `unsafe`, meaning it is the programmer's responsibility to ensure correctness instead of the compiler's. One example of this is `std::slice::from_raw_parts` which will create a slice given a pointer to the first element and a length.

```
1 use std::slice;
2
3 fn main() {
4     let some_vector = vec![1, 2, 3, 4];
5
6     let pointer = some_vector.as_ptr();
7     let length = some_vector.len();
8
9     unsafe {
10         let my_slice: &[u32] = slice::from_raw_parts(pointer, length);
11
12         assert_eq!(some_vector.as_slice(), my_slice);
13     }
14 }
```

For `slice::from_raw_parts`, one of the assumptions which *must* be upheld is that the pointer passed in points to valid memory and that the memory pointed to is of the correct type. If these invariants aren't upheld then the program's behaviour is undefined and there is no knowing what will happen.

Inline assembly

Rust provides support for inline assembly via the `asm!` macro. It can be used to embed handwritten assembly in the assembly output generated by the compiler. Generally this should not be necessary, but might be where the required performance or timing cannot be otherwise achieved. Accessing low level hardware primitives, e.g. in kernel code, may also demand this functionality.

Note: the examples here are given in x86/x86-64 assembly, but other architectures are also supported.

Inline assembly is currently supported on the following architectures:

- x86 and x86-64
- ARM
- AArch64
- RISC-V

Basic usage

Let us start with the simplest possible example:

```
use std::arch::asm;

unsafe {
    asm!("nop");
}
```

This will insert a NOP (no operation) instruction into the assembly generated by the compiler. Note that all `asm!` invocations have to be inside an `unsafe` block, as they could insert arbitrary instructions and break various invariants. The instructions to be inserted are listed in the first argument of the `asm!` macro as a string literal.

Inputs and outputs

Now inserting an instruction that does nothing is rather boring. Let us do something that actually acts on data:

```
use std::arch::asm;

let x: u64;
unsafe {
    asm!("mov {}, 5", out(reg) x);
}
assert_eq!(x, 5);
```

This will write the value `5` into the `u64` variable `x`. You can see that the string literal we use to specify instructions is actually a template string. It is governed by the same rules as Rust [format strings](#). The arguments that are inserted into the template however look a bit different than you may be familiar with. First we need to specify if the variable is an input or an output of the inline assembly. In this case it is an output. We declared this by writing `out`. We also need to specify in what kind of register the assembly expects the variable. In this case we put it in an arbitrary general purpose register by specifying `reg`. The compiler will choose an appropriate register to insert into the template and will read the variable from there after the inline assembly finishes executing.

Let us see another example that also uses an input:

```
use std::arch::asm;

let i: u64 = 3;
let o: u64;
unsafe {
    asm!(
        "mov {}, {}",
        "add {}, 5",
        out(reg) o,
        in(reg) i,
    );
}
assert_eq!(o, 8);
```

This will add `5` to the input in variable `i` and write the result to variable `o`. The particular way this assembly does this is first copying the value from `i` to the output, and then adding `5` to it.

The example shows a few things:

First, we can see that `asm!` allows multiple template string arguments; each one is treated as a separate line of assembly code, as if they were all joined together with newlines between them. This makes it easy to format assembly code.

Second, we can see that inputs are declared by writing `in` instead of `out`.

Third, we can see that we can specify an argument number, or name as in any format string. For inline assembly templates this is particularly useful as arguments are often used more than once. For more complex inline assembly using this facility is generally

recommended, as it improves readability, and allows reordering instructions without changing the argument order.

We can further refine the above example to avoid the `mov` instruction:

```
use std::arch::asm;

let mut x: u64 = 3;
unsafe {
    asm!("add {0}, 5", inout(reg) x);
}
assert_eq!(x, 8);
```

We can see that `inout` is used to specify an argument that is both input and output. This is different from specifying an input and output separately in that it is guaranteed to assign both to the same register.

It is also possible to specify different variables for the input and output parts of an `inout` operand:

```
use std::arch::asm;

let x: u64 = 3;
let y: u64;
unsafe {
    asm!("add {0}, 5", inout(reg) x => y);
}
assert_eq!(y, 8);
```

Late output operands

The Rust compiler is conservative with its allocation of operands. It is assumed that an `out` can be written at any time, and can therefore not share its location with any other argument. However, to guarantee optimal performance it is important to use as few registers as possible, so they won't have to be saved and reloaded around the inline assembly block. To achieve this Rust provides a `lateout` specifier. This can be used on any output that is written only after all inputs have been consumed. There is also a `inlateout` variant of this specifier.

Here is an example where `inlateout` *cannot* be used in `release` mode or other optimized cases:

```

use std::arch::asm;

let mut a: u64 = 4;
let b: u64 = 4;
let c: u64 = 4;
unsafe {
    asm!(
        "add {0}, {1}",
        "add {0}, {2}",
        inout(reg) a,
        in(reg) b,
        in(reg) c,
    );
}
assert_eq!(a, 12);

```

The above could work well in unoptimized cases (`Debug` mode), but if you want optimized performance (`release` mode or other optimized cases), it could not work.

That is because in optimized cases, the compiler is free to allocate the same register for inputs `b` and `c` since it knows they have the same value. However it must allocate a separate register for `a` since it uses `inout` and not `inlateout`. If `inlateout` was used, then `a` and `c` could be allocated to the same register, in which case the first instruction to overwrite the value of `c` and cause the assembly code to produce the wrong result.

However the following example can use `inlateout` since the output is only modified after all input registers have been read:

```

use std::arch::asm;

let mut a: u64 = 4;
let b: u64 = 4;
unsafe {
    asm!("add {0}, {1}", inlateout(reg) a, in(reg) b);
}
assert_eq!(a, 8);

```

As you can see, this assembly fragment will still work correctly if `a` and `b` are assigned to the same register.

Explicit register operands

Some instructions require that the operands be in a specific register. Therefore, Rust inline assembly provides some more specific constraint specifiers. While `reg` is generally available on any architecture, explicit registers are highly architecture specific. E.g. for x86 the general purpose registers `eax`, `ebx`, `ecx`, `edx`, `ebp`, `esi`, and `edi` among others

can be addressed by their name.

```
use std::arch::asm;

let cmd = 0xd1;
unsafe {
    asm!("out 0x64, eax", in("eax") cmd);
}
```

In this example we call the `out` instruction to output the content of the `cmd` variable to port `0x64`. Since the `out` instruction only accepts `eax` (and its sub registers) as operand we had to use the `eax` constraint specifier.

Note: unlike other operand types, explicit register operands cannot be used in the template string: you can't use `{}` and should write the register name directly instead. Also, they must appear at the end of the operand list after all other operand types.

Consider this example which uses the x86 `mul` instruction:

```
use std::arch::asm;

fn mul(a: u64, b: u64) -> u128 {
    let lo: u64;
    let hi: u64;

    unsafe {
        asm!(
            // The x86 mul instruction takes rax as an implicit input and
            writes // the 128-bit result of the multiplication to rax:rdx.
            "mul {}",
            in(reg) a,
            inlateout("rax") b => lo,
            lateout("rdx") hi
        );
    }

    ((hi as u128) << 64) + lo as u128
}
```

This uses the `mul` instruction to multiply two 64-bit inputs with a 128-bit result. The only explicit operand is a register, that we fill from the variable `a`. The second operand is implicit, and must be the `rax` register, which we fill from the variable `b`. The lower 64 bits of the result are stored in `rax` from which we fill the variable `lo`. The higher 64 bits are stored in `rdx` from which we fill the variable `hi`.

Clobbered registers

In many cases inline assembly will modify state that is not needed as an output. Usually this is either because we have to use a scratch register in the assembly or because instructions modify state that we don't need to further examine. This state is generally referred to as being "clobbered". We need to tell the compiler about this since it may need to save and restore this state around the inline assembly block.

```
use std::arch::asm;

fn main() {
    // three entries of four bytes each
    let mut name_buf = [0_u8; 12];
    // String is stored as ascii in ebx, edx, ecx in order
    // Because ebx is reserved, the asm needs to preserve the value of it.
    // So we push and pop it around the main asm.
    // 64 bit mode on 64 bit processors does not allow pushing/popping of
    // 32 bit registers (like ebx), so we have to use the extended rbx
    register instead.

    unsafe {
        asm!(
            "push rbx",
            "cpuid",
            "mov [rdi], ebx",
            "mov [rdi + 4], edx",
            "mov [rdi + 8], ecx",
            "pop rbx",
            // We use a pointer to an array for storing the values to
            // the Rust code at the cost of a couple more asm instructions
            // This is more explicit with how the asm works however, as
            // to explicit register outputs such as `out("ecx") val`
            // The *pointer itself* is only an input even though it's written
            in("rdi") name_buf.as_mut_ptr(),
            // select cpuid 0, also specify eax as clobbered
            inout("eax") 0 => _,
            // cpuid clobbers these registers too
            out("ecx") _,
            out("edx") _,
        );

        let name = core::str::from_utf8(&name_buf).unwrap();
        println!("CPU Manufacturer ID: {}", name);
    }
}
```

In the example above we use the `cpuid` instruction to read the CPU manufacturer ID. This instruction writes to `eax` with the maximum supported `cpuid` argument and `ebx`,

`edx`, and `ecx` with the CPU manufacturer ID as ASCII bytes in that order.

Even though `eax` is never read we still need to tell the compiler that the register has been modified so that the compiler can save any values that were in these registers before the asm. This is done by declaring it as an output but with `_` instead of a variable name, which indicates that the output value is to be discarded.

This code also works around the limitation that `ebx` is a reserved register by LLVM. That means that LLVM assumes that it has full control over the register and it must be restored to its original state before exiting the asm block, so it cannot be used as an input or output **except** if the compiler uses it to fulfill a general register class (e.g. `in(reg)`). This makes `reg` operands dangerous when using reserved registers as we could unknowingly corrupt our input or output because they share the same register.

To work around this we use `rdi` to store the pointer to the output array, save `ebx` via `push`, read from `ebx` inside the asm block into the array and then restore `ebx` to its original state via `pop`. The `push` and `pop` use the full 64-bit `rbx` version of the register to ensure that the entire register is saved. On 32 bit targets the code would instead use `ebx` in the `push` / `pop`.

This can also be used with a general register class to obtain a scratch register for use inside the asm code:

```
use std::arch::asm;

// Multiply x by 6 using shifts and adds
let mut x: u64 = 4;
unsafe {
    asm!(
        "mov {tmp}, {x}",
        "shl {tmp}, 1",
        "shl {x}, 2",
        "add {x}, {tmp}",
        x = inout(reg) x,
        tmp = out(reg) _,
    );
}
assert_eq!(x, 4 * 6);
```

Symbol operands and ABI clobbers

By default, `asm!` assumes that any register not specified as an output will have its contents preserved by the assembly code. The `clobber_abi` argument to `asm!` tells the compiler to automatically insert the necessary clobber operands according to the given calling convention ABI: any register which is not fully preserved in that ABI will be treated

as clobbered. Multiple `clobber_abi` arguments may be provided and all clobbers from all specified ABIs will be inserted.

```
use std::arch::asm;

extern "C" fn foo(arg: i32) -> i32 {
    println!("arg = {}", arg);
    arg * 2
}

fn call_foo(arg: i32) -> i32 {
    unsafe {
        let result;
        asm!(
            "call {}",
            // Function pointer to call
            in(reg) foo,
            // 1st argument in rdi
            in("rdi") arg,
            // Return value in rax
            out("rax") result,
            // Mark all registers which are not preserved by the "C" calling
            // convention as clobbered.
            clobber_abi("C"),
        );
        result
    }
}
```

Register template modifiers

In some cases, fine control is needed over the way a register name is formatted when inserted into the template string. This is needed when an architecture's assembly language has several names for the same register, each typically being a "view" over a subset of the register (e.g. the low 32 bits of a 64-bit register).

By default the compiler will always choose the name that refers to the full register size (e.g. `rax` on x86-64, `eax` on x86, etc).

This default can be overridden by using modifiers on the template string operands, just like you would with format strings:

```

use std::arch::asm;

let mut x: u16 = 0xab;

unsafe {
    asm!("mov {0:h}, {0:l}", inout(reg_abcd) x);
}

assert_eq!(x, 0xabab);

```

In this example, we use the `reg_abcd` register class to restrict the register allocator to the 4 legacy x86 registers (`ax` , `bx` , `cx` , `dx`) of which the first two bytes can be addressed independently.

Let us assume that the register allocator has chosen to allocate `x` in the `ax` register. The `h` modifier will emit the register name for the high byte of that register and the `l` modifier will emit the register name for the low byte. The asm code will therefore be expanded as `mov ah, al` which copies the low byte of the value into the high byte.

If you use a smaller data type (e.g. `u16`) with an operand and forget to use template modifiers, the compiler will emit a warning and suggest the correct modifier to use.

Memory address operands

Sometimes assembly instructions require operands passed via memory addresses/memory locations. You have to manually use the memory address syntax specified by the target architecture. For example, on x86/x86_64 using Intel assembly syntax, you should wrap inputs/outputs in `[]` to indicate they are memory operands:

```

use std::arch::asm;

fn load_fpu_control_word(control: u16) {
    unsafe {
        asm!("fldcw [{0}]", in(reg) &control, options(nostack));
    }
}

```

Labels

Any reuse of a named label, local or otherwise, can result in an assembler or linker error or may cause other strange behavior. Reuse of a named label can happen in a variety of ways including:

- explicitly: using a label more than once in one `asm!` block, or multiple times across blocks.
- implicitly via inlining: the compiler is allowed to instantiate multiple copies of an `asm!` block, for example when the function containing it is inlined in multiple places.
- implicitly via LTO: LTO can cause code from *other crates* to be placed in the same codegen unit, and so could bring in arbitrary labels.

As a consequence, you should only use GNU assembler **numeric local labels** inside inline assembly code. Defining symbols in assembly code may lead to assembler and/or linker errors due to duplicate symbol definitions.

Moreover, on x86 when using the default Intel syntax, due to [an LLVM bug](#), you shouldn't use labels exclusively made of 0 and 1 digits, e.g. 0, 11 or 101010, as they may end up being interpreted as binary values. Using `options(att_syntax)` will avoid any ambiguity, but that affects the syntax of the *entire* `asm!` block. (See [Options](#), below, for more on `options`.)

```
use std::arch::asm;

let mut a = 0;
unsafe {
    asm!(
        "mov {0}, 10",
        "2:",
        "sub {0}, 1",
        "cmp {0}, 3",
        "jle 2f",
        "jmp 2b",
        "2:",
        "add {0}, 2",
        out(reg) a
    );
}
assert_eq!(a, 5);
```

This will decrement the `{0}` register value from 10 to 3, then add 2 and store it in `a`.

This example shows a few things:

- First, that the same number can be used as a label multiple times in the same inline block.
- Second, that when a numeric label is used as a reference (as an instruction operand, for example), the suffixes `"b"` ("backward") or `"f"` ("forward") should be added to the numeric label. It will then refer to the nearest label defined by this number in this direction.

Options

By default, an inline assembly block is treated the same way as an external FFI function call with a custom calling convention: it may read/write memory, have observable side effects, etc. However, in many cases it is desirable to give the compiler more information about what the assembly code is actually doing so that it can optimize better.

Let's take our previous example of an `add` instruction:

```
use std::arch::asm;

let mut a: u64 = 4;
let b: u64 = 4;
unsafe {
    asm!(
        "add {0}, {1}",
        inlateout(reg) a, in(reg) b,
        options(pure, nomem, nostack),
    );
}
assert_eq!(a, 8);
```

Options can be provided as an optional final argument to the `asm!` macro. We specified three options here:

- `pure` means that the asm code has no observable side effects and that its output depends only on its inputs. This allows the compiler optimizer to call the inline asm fewer times or even eliminate it entirely.
- `nomem` means that the asm code does not read or write to memory. By default the compiler will assume that inline assembly can read or write any memory address that is accessible to it (e.g. through a pointer passed as an operand, or a global).
- `nostack` means that the asm code does not push any data onto the stack. This allows the compiler to use optimizations such as the stack red zone on x86-64 to avoid stack pointer adjustments.

These allow the compiler to better optimize code using `asm!`, for example by eliminating `pure asm!` blocks whose outputs are not needed.

See the [reference](#) for the full list of available options and their effects.

Compatibility

The Rust language is fastly evolving, and because of this certain compatibility issues can arise, despite efforts to ensure forwards-compatibility wherever possible.

- [Raw identifiers](#)

Raw identifiers

Rust, like many programming languages, has the concept of "keywords". These identifiers mean something to the language, and so you cannot use them in places like variable names, function names, and other places. Raw identifiers let you use keywords where they would not normally be allowed. This is particularly useful when Rust introduces new keywords, and a library using an older edition of Rust has a variable or function with the same name as a keyword introduced in a newer edition.

For example, consider a crate `foo` compiled with the 2015 edition of Rust that exports a function named `try`. This keyword is reserved for a new feature in the 2018 edition, so without raw identifiers, we would have no way to name the function.

```
extern crate foo;

fn main() {
    foo::try();
}
```

You'll get this error:

```
error: expected identifier, found keyword `try`
--> src/main.rs:4:4
   |
4 | foo::try();
   |      ^^^ expected identifier, found keyword
```

You can write this with a raw identifier:

```
extern crate foo;

fn main() {
    foo::r#try();
}
```


Meta

Some topics aren't exactly relevant to how you program but provide you tooling or infrastructure support which just makes things better for everyone. These topics include:

- [Documentation](#): Generate library documentation for users via the included `rustdoc` .
- [Playground](#): Integrate the Rust Playground in your documentation.

Documentation

Use `cargo doc` to build documentation in `target/doc`.

Use `cargo test` to run all tests (including documentation tests), and `cargo test --doc` to only run documentation tests.

These commands will appropriately invoke `rustdoc` (and `rustc`) as required.

Doc comments

Doc comments are very useful for big projects that require documentation. When running `rustdoc`, these are the comments that get compiled into documentation. They are denoted by a `///`, and support [Markdown](#).

```
1  #![crate_name = "doc"]
2
3  /// A human being is represented here
4  pub struct Person {
5      /// A person must have a name, no matter how much Juliet may hate it
6      name: String,
7  }
8
9  impl Person {
10     /// Returns a person with the name given them
11     ///
12     /// # Arguments
13     ///
14     /// * `name` - A string slice that holds the name of the person
15     ///
16     /// # Examples
17     ///
18     /// ```
19     /// // You can have rust code between fences inside the comments
20     /// // If you pass --test to `rustdoc`, it will even test it for you!
21     /// use doc::Person;
22     /// let person = Person::new("name");
23     /// ```
24     pub fn new(name: &str) -> Person {
25         Person {
26             name: name.to_string(),
27         }
28     }
29
30     /// Gives a friendly hello!
31     ///
32     /// Says "Hello, [name](Person::name)" to the `Person` it is called o
33     pub fn hello(& self) {
34         println!("Hello, {}!", self.name);
35     }
36 }
37
38 fn main() {
39     let john = Person::new("John");
40
41     john.hello();
42 }
```

To run the tests, first build the code as a library, then tell `rustdoc` where to find the library so it can link it into each doctest program:

```
$ rustc doc.rs --crate-type lib
$ rustdoc --test --extern doc="libdoc.rlib" doc.rs
```

Doc attributes

Below are a few examples of the most common `#[doc]` attributes used with `rustdoc`.

inline

Used to inline docs, instead of linking out to separate page.

```
#[doc(inline)]
pub use bar::Bar;

/// bar docs
mod bar {
    /// the docs for Bar
    pub struct Bar;
}
```

no_inline

Used to prevent linking out to separate page or anywhere.

```
// Example from libcore/prelude
#[doc(no_inline)]
pub use crate::mem::drop;
```

hidden

Using this tells `rustdoc` not to include this in documentation:

```
1 // Example from the futures-rs library
2 #[doc(hidden)]
3 pub use self::async_await::*;
```

For documentation, `rustdoc` is widely used by the community. It's what is used to generate the [std library docs](#).

See also:

- [The Rust Book: Making Useful Documentation Comments](#)
- [The rustdoc Book](#)
- [The Reference: Doc comments](#)
- [RFC 1574: API Documentation Conventions](#)
- [RFC 1946: Relative links to other items from doc comments \(intra-rustdoc links\)](#)
- [Is there any documentation style guide for comments? \(reddit\)](#)

Playground

The [Rust Playground](#) is a way to experiment with Rust code through a web interface.

Using it with mdbook

In [mdbook](#), you can make code examples playable and editable.

```
1 fn main() {  
2     println!("Hello World!");  
3 }
```

This allows the reader to both run your code sample, but also modify and tweak it. The key here is the adding the word `editable` to your code fence block separated by a comma.

```
```rust,editable  
//...place your code here
```
```

Additionally, you can add `ignore` if you want `mdbook` to skip your code when it builds and tests.

```
```rust,editable,ignore  
//...place your code here
```
```

Using it with docs

You may have noticed in some of the [official Rust docs](#) a button that says "Run", which opens the code sample up in a new tab in Rust Playground. This feature is enabled if you use the `#[doc]` attribute called `html_playground_url`.

See also:

- [The Rust Playground](#)
- [rust-playground](#)
- [The rustdoc Book](#)